

卒業論文

High Performance Computing 向け AI Scientist v2の提案

名古屋大学 情報学部 コンピュータ科学科
情報システム専攻

学籍番号：102230147

樹神 宇徳

概要

従来の大規模言語モデル（LLM）を用いた自律研究システムは、アイデア生成から実験実行、論文執筆までを自動化しうる一方で、高性能計算（HPC）環境に固有の制約（実行環境の分離、依存解決、コンパイル型言語、資源管理、長期的な実験履歴の保持）に十分には適合していない。特に Sakana AI 社が提案した AI Scientist v2 は、Python 中心の実行を前提としており、HPC で一般的な「環境構築→ビルド→実行→解析」の反復を安全かつ再現可能に扱うには設計上のギャップが残る。

本研究は、AI Scientist v2 を基盤として HPC 環境に適合させた自律研究システム HPC-AutoResearch を提案する。提案の主眼は、(1) システムアーキテクチャの設計、および (2) Split-Phase 実行モデルの提案である。Split-Phase 実行モデルは、研究実行を Phase 0（計画）から Phase 4（実行）までの 5 段階に分割し、HPC で一般的な「環境構築→ビルド→実行→解析」の反復を安全かつ再現可能に扱う。フェーズ分割によりフェーズ間で参照すべき履歴・判断根拠が増え、プロンプトに含める情報を取捨選択するだけでは重要情報の欠落が生じうるため、LLM が前実験で重要だと判断した知見を記録・再利用する MemGPT 型 3 層階層記憶（Core/Recall/Archival）と分岐継承を追加し、文脈長制約下でも反復実験に必要な情報を保持する。加えて、Singularity/Apptainer によるコンテナ分離実行と反復的依存解決、および AI Scientist v2 に実装されている評価関数に基づく Best-First 木探索を継承し、提案機構と統合して用いる。

Himeno Benchmark を題材に、提案手法と既存の LLM コーディングエージェント（Claude Code, Codex, Gemini CLI）との比較実験を実施し、各エージェントの出力を提案システムのポスト処理パイプライン（論文生成＋LLM レビュー）に通して評価した。その結果、提案手法（HPC-AutoResearch）は環境構築から結果取得まで全段階を完遂し、LLM レビューにおいて全 7 評価項目で最高水準（各 3/4）を達成、Overall 7/10・Accept 判定を獲得した。Claude Code および Gemini CLI も全段階を完遂し Accept 判定を得たが、独創性や貢献度では提案手法が優位であった。一方、Codex は環境構築段階で失敗し実験結果を取得できず、Overall 3/10・Reject 判定となった。これらの結果は、Split-Phase 実行モデルによるフェーズ分離と反復的修復、および階層記憶機構による文脈管理が、HPC 環境での自律研究遂行において有効に機能することを示している。本論文では、上記の各機構を「HPC での自律研究」を成立させるための設計要件として再定式化し、アルゴリズム、データ構造、フェーズ遷移、記憶操作、資源注入規則を形式的に記述する。

目次

第 1 章	はじめに	3
1.1	本研究のアプローチ：HPC-AutoResearch	3
1.2	AI Scientist v2 からの拡張	4
1.3	本研究の貢献	5
1.4	本論文の構成	5
第 2 章	AI Scientist v2	6
2.1	AI Scientist v2 の概要	6
2.2	AI Scientist v2 の問題点	7
2.3	Best-First 木探索	11
第 3 章	提案システム	17
3.1	システムアーキテクチャ	17
3.2	Split-Phase 実行モデル	22
3.3	階層記憶機構の提案	29
3.4	ポスト処理パイプラインの提案	42
3.5	実装と検証	48
第 4 章	性能評価	50
4.1	実験概要	50
4.2	実験条件	50
4.3	実験環境	51
4.4	評価指標	52
4.5	実験結果	52
4.6	分析	72
第 5 章	関連研究	74
5.1	LLM を用いた研究自動化	74
5.2	LLM エージェントの記憶機構	75
5.3	LLM ソフトウェアエージェント	76
5.4	コード生成における探索アルゴリズム	76
5.5	本研究の位置づけ	77

第 6 章	おわりに	79
6.1	まとめ	79
6.2	今後の課題	80
6.3	今後の展開	80
6.4	提案手法の限界	81
6.5	設計上のトレードオフ	81
謝辞		83
Data Availability		84
参考文献		85
付録 A	実験設定ファイル	88
付録 B	実験題材：Himeno Benchmark 性能解析	92
付録 C	アイデア創出機構の生成例	94
付録 D	論文評価プロンプト	97
D.1	LLM レビュー プロンプト	97
D.2	VLM レビュー プロンプト	101
付録 E	エージェント比較実験の入力ファイル	103
付録 F	各エージェントの生成論文	107
F.1	Claude Code の生成論文	108
F.2	Codex の生成論文	118
F.3	Gemini CLI の生成論文	121

第 1 章

はじめに

大規模言語モデル (LLM) の急速な発展により、プログラム生成や科学的推論における能力が飛躍的に向上している。この進歩を受け、LLM を用いて研究プロセス全体を自動化しようとする試みが活発化している。

Sakana AI 社が提案した AI Scientist v2 は、LLM エージェントを用いて研究プロセス（アイデア生成、実験実行、論文草稿生成など）を自律的に遂行することを目的としたシステムであるが、高性能計算 (HPC) 環境での運用を考慮した設計になっていない。本研究では、AI Scientist v2 を基盤として HPC 環境に適合させた自律研究システム HPC-AutoResearch を提案する。

1.1 本研究のアプローチ：HPC-AutoResearch

本研究では、AI Scientist v2 の問題点を解決し、HPC 環境で動作する自律研究システム「HPC-AutoResearch」を提案する。HPC-AutoResearch は、AI Scientist v2 の GitHub リポジトリ^{*1}として公開されているコードベースをフォークし、これを基盤として開発を行った。オリジナルの木探索機構や実験管理の枠組みを継承しつつ、HPC 環境への適合に必要な拡張を加えている。HPC-AutoResearch は、以下の 3 つの主要コンポーネントから構成される。

1.1.1 Split-Phase 実行モデル

研究プロセスを 5 つのフェーズ（計画・環境構築・コーディング・コンパイル・実行）に分割し、各フェーズで適切な抽象度の問題を LLM に与える：

- **Phase 0 (計画)**：環境情報の収集と実行計画の策定
- **Phase 1 (環境構築)**：Singularity コンテナ内での依存解決
- **Phase 2 (コーディング)**：LLM によるソースコード生成
- **Phase 3 (コンパイル)**：gcc/g++/nvcc 等でのビルド
- **Phase 4 (実行)**：プログラム実行と結果収集

これにより、複雑なタスクを管理可能な部分問題に分解し、各フェーズでの失敗を診断・修復可能にす

^{*1} <https://github.com/SakanaAI/AI-Scientist-v2>

る．一方で、フェーズ分割は「各フェーズで何を入力として与えるべきか」という選択問題を新たに生み、反復の進行に伴って文脈（履歴・判断根拠）が肥大化しやすい．本研究ではこの問題に対して、次節の階層記憶機構により、重要情報の欠落を抑制する．

1.1.2 MemGPT 型階層記憶機構

MemGPT[1] の概念を拡張した 3 層階層記憶機構を新規に実装した：

- **コア記憶**：常に参照される重要情報（研究目標、設定）
- **リコール記憶**：時間的に近い出来事（最近の実験結果）
- **アーカイブ記憶**：検索可能な長期記憶（過去の知見）

LLM による知的な圧縮と記憶操作により、有限の文脈長の中で長期的な実験履歴を効率的に管理する．

1.1.3 Best-First 木探索

評価関数に基づく最良優先探索により、実装空間を効率的に探索する：

- 各ノードはコード実装状態を表現
- 評価値に基づいて次に展開するノードを選択
- 有望な実装に計算資源を集中
- 4 つのステージ（初期実装・ベースライン調整・創造的改良・アブレーション）で体系的に探索

1.2 AI Scientist v2 からの拡張

1.2.1 Singularity コンテナによる実行環境

HPC 環境では Docker が使用できない（root デーモンが必要であるため）ケースが一般的であり、代わりに Singularity（Apptainer）が標準的に使用される．Singularity はユーザー空間で動作し、root 権限なしにコンテナを実行できるため、共有クラスタ環境に適している．

本システムでは、Singularity コンテナを用いた実行環境を新規に実装した：

- **Singularity コンテナ内での分離実行環境**：LLM が生成したコード（`apt-get install` やソースビルドを含む）をホスト環境から隔離して実行し、他ユーザーや共有環境への影響を排除する
- **GPU パススルー**：`--nv` フラグにより NVIDIA GPU をコンテナ内から利用可能にし、CUDA/OpenCL ワークロードを実行する
- **ワーカーごとの SIF 構築**：Phase 1 でインストールされた依存関係をワーカーごとの SIF イメージに焼き込み、Phase 2 以降の実行で再インストールを不要にする．これにより、依存解決の結果が確実に再現される
- **書き込みモード**：`writable-tmpfs`（高速だがメモリ制限あり）、`overlay`（永続的だが低速）、`auto`（状況に応じて選択）の 3 モードを提供し、Phase 1 での柔軟な依存解決を実現する

主要な実行環境パラメータとして、Singularity イメージパス、GPU 使用有無、ワークスペースマウントポイント、Phase 1 の最大イテレーション数（デフォルト 100）、実行タイムアウト（デフォルト 10800 秒）などを設定ファイルにて指定する。

1.2.2 その他の拡張

上記の主要拡張に加え、以下の機能を追加した：

- **リソースファイル機構**：データセット、テンプレート、ドキュメント、Hugging Face モデルを宣言的に管理
- **ペルソナシステム**：役割に応じたプロンプトカスタマイズ
- **トークン追跡**：LLM 使用量とコストのモニタリング
- **マルチシード評価**：結果の堅牢性検証のための複数シード実行
- **最終メモリ生成**：論文執筆用の構造化された実験記録出力

1.3 本研究の貢献

本研究の学術的貢献を以下にまとめる。

1. **HPC 制約下の自律研究を要件として定式化**：実行環境分離、反復的依存解決、コンパイル型言語対応、資源管理、長期記憶という HPC 固有の要求を、自律研究ワークフローの要件として整理した。
2. **Split-Phase 実行モデルの提示**：計画・環境構築・コーディング・コンパイル・実行を明確に分離し、各フェーズの入出力（計画、成果物、ログ）を定義することで、失敗診断と修復を可能にした。
3. **MemGPT 型階層記憶の HPC 木探索への統合**：Core/Recall/Archival の 3 層に加え、分岐継承と書き込み隔離（sibling isolation）を導入し、探索木に沿った知識の蓄積と再利用を可能にした。
4. **資源注入（Resource System）による外部依存の宣言的管理**：ローカルデータ、GitHub、Hugging Face 等の外部資源を宣言的に定義し、フェーズごとの注入規則によりプロンプト・コンテナ双方へ安全に注入する枠組みを示した。

1.4 本論文の構成

本論文の構成は以下の通りである。第 2 章では、本研究の基盤である AI Scientist v2 の概要と問題点、および AI Scientist v2 に実装されている Best-First 木探索について述べる。第 3 章では、提案システムとして、システムアーキテクチャ、Split-Phase 実行モデル、階層記憶機構の提案、ポスト処理パイプラインの提案、および実装と検証について述べる。第 4 章では、性能評価として実験の概要と結果を示す。第 5 章では、関連研究を概観し、本研究の位置づけを明確にする。第 6 章では、考察と結論を述べる。

第 2 章

AI Scientist v2

2.1 AI Scientist v2 の概要

大規模言語モデル（LLM）の急速な発展により、プログラム生成や科学的推論における能力が飛躍的に向上している。この進歩を受け、LLM を用いて研究プロセス全体を自動化しようとする試みが活発化している。

AI Scientist v2[2] は、LLM エージェントを用いて研究プロセス（アイデア生成、実験実行、論文草稿生成など）を自律的に遂行することを目的としたシステムである。図 2.1 に、文献で示される AI Scientist v2 のワークフローを示す。

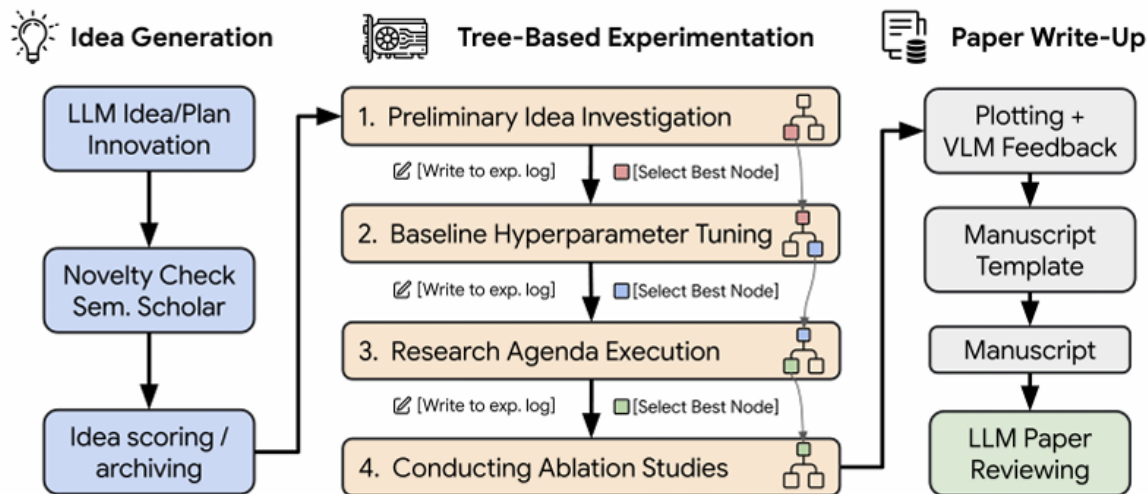


図 2.1 AI Scientist v2 のワークフロー ([2] より引用)。

図 2.1 に示すように、AI Scientist v2 はアイデア生成からコード実装、実験実行、論文執筆、レビューまでの研究サイクル全体を LLM エージェントが自律的に遂行する。木探索（AIDE[3] ベース）により複数の実装候補を探索し、評価関数に基づいて最良の実装を選択する。本研究はこのワークフローを基盤として、HPC 環境への適応（Split-Phase 実行、Singularity コンテナ、階層記憶機構）を追加したものである。

AI Scientist v2 の主要な特徴は以下の通りである：

- **木探索によるコード空間探索**：複数のコード実装候補を木構造で管理し，評価値に基づいて有望な実装を選択
- **一貫したワークフロー**：アイデア生成から実験実行，論文執筆までを自動化
- **LLM エージェントによる自律的実行**：人間の介入なしに研究プロセスを遂行
- **汎用的なコードベース**：トピック固有のテンプレートではなく，汎用的なコードベースを用いた実験を対象として設計

AI Scientist v2 の文献上の説明では，研究プロセスを「アイデア生成」「実験実行」「論文生成（執筆）」といった段階に分解し，段階ごとに異なるプロンプトや処理を割り当てる点が中核として述べられる．ただし実装上は，木探索や後処理（メトリクス抽出・可視化等）など複数のサブステップが含まれるため，本論文では以降，必要に応じて内部処理を分解して扱う．

2.2 AI Scientist v2 の問題点

AI Scientist v2 は画期的なシステムであるが，高性能計算（HPC）環境での運用を考慮した設計になっていない．HPC 環境で AI Scientist v2 を適用しようとした場合，以下の根本的な問題点が存在する．

2.2.1 問題点 1：実行環境の分離機構がない

AI Scientist v2 は，ホストマシン上で Python の任意の関数をローカル実行する設計となっている．ただし，シェルスクリプトの実行やパッケージのダウンロード・インストールといったシステムレベルの操作は行えない．この設計は，研究者個人のローカル環境やクラウドインスタンスでは十分に機能するが，HPC 環境では以下のセキュリティ上・運用上の制約と衝突する：

- **root 権限の不在**：HPC 共有クラスタではユーザーに root 権限が付与されず，`apt-get install` や `pip install` によるシステムレベルのパッケージインストールが不可能である．AI Scientist v2 はシェルスクリプトを実行できないため，ユーザー空間でのパッケージ導入も自律的に行えない
- **ネットワーク制限**：計算ノードから外部ネットワークへのアクセスが制限される場合が多く，実行時に PyPI や GitHub からの依存取得が失敗する．なお，このネットワーク制限の問題は HPC 環境固有の課題であり，本研究でも完全には解決していないが，Singularity コンテナ内に事前に依存関係を封入することで影響を軽減する設計としている
- **マルチテナント環境への影響**：共有クラスタでは，LLM が生成したコードがホスト環境を直接変更すると，他のユーザーのジョブや共有ライブラリに意図しない影響を与える可能性がある．実行環境をコンテナ等で分離する機構が不可欠であるが，AI Scientist v2 にはこのような分離機構が存在しない

2.2.2 問題点 2：明示的な長期記憶の取り扱いが限定的である

AI Scientist v2 では，木探索において親ノードのコードとフィードバック要約のみが子ノードに継承されるが，祖先ノードの知見は参照できず，研究の反復過程で得られる知見（試行の履歴，失敗要因，実験設

定、計測結果など）を、外部の長期記憶として構造化・検索可能に管理する仕組みが体系化されていない。研究実験は多くの場合、単一の LLM 呼び出しでは完結せず、試行錯誤の結果が次の実験設計に影響する反復的プロセスである。

そのため、探索が深まるにつれ初期の知見が失われ、過去の実験経験が構造的に再利用されない。結果として以下の問題が生じる：

- 過去の実験結果・失敗パターンが十分に再利用されず、探索が非効率になる
- 既出のエラーや回避策を再発見するなど、同種の失敗を繰り返す可能性がある
- 長期にわたる実験履歴の整合的な要約や引用（論文執筆時の根拠付け）が難しくなる

2.2.3 問題点 3：Python 中心の実行モデル

AI Scientist v2 は、Experiment Progress Manager により 4 つのステージ（予備調査、ハイパーパラメータチューニング、研究アジェンダ実行、アブレーション研究）に分割された実験管理を行う。しかし、各ステージ内でのコード生成と実行は Python スクリプトを前提としており、HPC 環境で一般的な以下のワークフローに対応できない：

- **コンパイル型言語への非対応**：HPC 研究では性能が重要であり、C/C++、Fortran、CUDA などのコンパイル型言語が広く使用される。AI Scientist v2 は Python の関数実行のみを想定しているため、ソースコードのコンパイル、リンク、最適化フラグの指定といったビルドプロセスを扱えない
- **複雑な依存関係の未対応**：HPC 環境では、MPI 実装（OpenMPI、MPICH）、数値計算ライブラリ（BLAS、LAPACK、FFTW）、GPU ツールキット（CUDA、ROCm）など、バージョンや設定が相互に依存するライブラリ群を正確に構成する必要がある。AI Scientist v2 にはこれらのビルド時依存関係を解決する仕組みがない
- **並列プログラミングモデルの欠如**：MPI によるプロセス間通信、OpenMP によるスレッド並列、CUDA による GPU 並列など、HPC 研究で不可欠な並列実行モデルの起動・制御（例：`mpirun -np 4` による並列実行）がサポートされていない

2.2.4 問題点 4：実行環境情報へのアクセス機能の欠如

HPC 環境では、プログラムの性能がハードウェア構成に大きく依存するため、実行環境の詳細情報に基づいた最適化が不可欠である。AI Scientist v2 は、このような実行環境の内部情報を取得・活用する機能を持たない：

- **ハードウェア情報の取得不可**：CPU 情報（コア数、アーキテクチャ、キャッシュサイズ、SIMD 対応状況）、GPU 情報（デバイス名、VRAM 容量、Compute Capability）、メモリ容量や NUMA トポロジといった情報を自動的に取得する機構がない。これにより、LLM が実行環境の能力を把握できず、ハードウェアに適したコードを生成することが困難である
- **ソフトウェアスタックの検出不可**：利用可能なコンパイラ（gcc、nvcc、icc 等）のバージョン、インストール済みライブラリ（MPI 実装、BLAS、CUDA ツールキット等）、module システムで提供さ

れる環境などを検出する機能がない。そのため、LLM は存在しないコンパイラやライブラリを前提としたコードを生成し、ビルドが失敗する可能性がある

- **環境適応型最適化の欠如**：実行環境の情報が得られないため、AVX-512 命令の利用可否に応じた SIMD 最適化、NUMA ノード配置を考慮したメモリアクセスパターン、GPU メモリ容量に応じたバッチサイズ調整など、環境に適応した最適化戦略を LLM が選択できない

2.2.5 問題点 5：リソース管理機能の欠如

HPC 研究では、大規模データセット、参照実装、事前学習モデル、セットアップスクリプトなど多様な外部リソースを実験で使うことが一般的である。AI Scientist v2 は、これらの外部リソースを体系的に管理する機構を持たない：

- 実験に必要な外部リソース（データセット、テンプレートコード、設定ファイル、外部ライブラリ等）を宣言的に定義・管理する機構がなく、リソースの追加・変更のたびにコードの修正が必要となる
- リソースの種別（テンプレート、ドキュメント、セットアップスクリプト、ライブラリ等）に応じて、実験のどの段階でどのように注入するかを制御する規則が存在しない。そのため、不要な情報がプロンプトに含まれたり、必要なリソースが欠落したりする可能性がある
- ローカルファイル、GitHub リポジトリ、Hugging Face モデルといった異なるソースからのリソース取得を統一的に扱うインターフェースがなく、リソースの出自に応じた個別の処理が必要となる

2.2.6 問題点 6：Python に限定された言語サポート

AI Scientist v2 は、Python スクリプトの実行のみを想定した設計となっている。HPC 研究で使われる多様な言語やビルドシステムに対応できない：

- **コンパイル型言語の非対応**：HPC 分野では、数値計算性能を追求するため C/C++、Fortran、CUDA が広く使われるが、AI Scientist v2 はこれらの言語で記述されたソースコードのコンパイル・実行を行う機構を持たない。特に、Fortran は気象・海洋シミュレーションなどのレガシーコードで依然として主要な言語であり、CUDA は GPU コンピューティングに不可欠である
- **ビルドシステムの非対応**：HPC プロジェクトでは、Makefile、CMake、Autotools などのビルドシステムが標準的に使われる。AI Scientist v2 にはこれらのビルドシステムを生成・実行する機能がなく、複数のソースファイルから成るプロジェクトの自動ビルドが困難である
- **シェルスクリプト実行の不可**：AI Scientist v2 は Python 関数の実行のみが可能であり、シェルスクリプトの実行やパッケージのダウンロード・インストールができない。そのため、環境構築の自動化や、`module load` による HPC 環境設定、`make` によるビルド実行といった、HPC 研究に不可欠な操作を自律的に行えない

本研究では、Python に加えて C/C++、Fortran、CUDA などのコンパイル型言語にも対応する。

2.2.7 問題点 7：機械学習実験に特化したプロンプト設計

AI Scientist v2 は汎用的なコードベースを用いる設計であるが、プロンプトは機械学習実験を前提とした表現（例：「訓練ループ」「損失関数」「エポック」といった用語や、PyTorch のコード構造を想定した指示）で構成されており、HPC 実験を含む他の研究領域にそのまま適用することが難しい：

- **HPC 研究領域の未考慮**：数値シミュレーション（有限要素法，有限差分法），流体力学（CFD），分子動力学（MD），量子化学計算など，HPC で主要な研究領域では，機械学習とは根本的に異なる評価指標（収束性，精度，スケーラビリティ）や実験設計が求められる．AI Scientist v2 のプロンプトはこれらの領域を想定していない
- **性能最適化観点の欠如**：HPC 研究では計算性能の最適化が中心的課題であり，SIMD 命令の活用，キャッシュ局所性の最適化，メモリアクセスパターンの改善，通信と計算のオーバーラップなどが重要な設計判断となる．AI Scientist v2 のプロンプトにはこれらの性能最適化に関する知識や観点が含まれていない
- **ベンチマーク手法の未対応**：HPC 研究では，弱スケーリング・強スケーリングによるスケーラビリティ評価，FLOPS やメモリバンド幅によるルーフラインモデル分析，異なる問題サイズでの性能比較など，機械学習の精度評価とは異なるベンチマーク手法が使用される．これらの手法がプロンプトに想定されていない

本研究では，特定の研究領域に特化するのではなく，HPC 実験にも適用可能な中立的な表現のプロンプトを実装し，ペルソナシステムにより対象ドメインを切り替え可能とした．

2.2.8 問題点 8：限定的なデータソースアクセス

AI Scientist v2 は，データソースとして Hugging Face データセットの読み込みを主に想定しており，HPC 研究で一般的な多様なデータソースへの対応が不十分である：

- **データソースの限定**：AI Scientist v2 は Hugging Face からのデータセット取得を主要なデータ入力手段としている．しかし，HPC 研究では，ローカルファイルシステム上の大規模シミュレーションデータ，GitHub リポジトリに格納された参照実装やベンチマークスイート，組織内のデータサーバなど，Hugging Face 以外の多様なデータソースを使用することが一般的である
- **大規模データの管理不在**：HPC 研究で扱うデータは数 GB～数 TB に及ぶことがあり，メモリに全データを展開する方式では対応できない．ストリーミング読み込み，分散ファイルシステム（Lustre，GPFS 等）上のデータへの効率的なアクセス，ノード間でのデータ分配といった大規模データ固有の管理が考慮されていない
- **研究固有のデータ形式への未対応**：HPC 研究では，HDF5，NetCDF，VTK，FITS，バイナリ格子データなど，分野固有のデータ形式が広く使用される．AI Scientist v2 はこれらの形式への対応を想定しておらず，データの読み込み・前処理を LLM が自律的に行うことが困難である

表 2.1 に，AI Scientist v2 の問題点と本研究での解決策の対応を示す．

表 2.1 AI Scientist v2 の問題点と本研究の解決策

問題点	AI Scientist v2 の状況	本研究の解決策
実行環境分離	なし	Singularity コンテナ
記憶機構	コードとフィードバック継承のみ	3 層階層記憶
文脈圧縮	なし	LLM による知的圧縮
実行モデル	4 ステージ (Python 中心)	4 ステージ + Split-Phase (5 フェーズ, 多言語対応)
環境情報取得	なし	Phase 0 で自動収集
依存解決	事前構築前提	反復的自律解決
並列実行	エージェント木探索の並列化	マルチワーカー対応
リソース管理	限定的	リソースファイル機構
言語サポート	Python のみ	C/C++/Fortran/CUDA 対応
プロンプト	ML 実験特化	中立的表現 + ペルソナシステム
データソース	Hugging Face 中心	多様なソース対応

2.3 Best-First 木探索

本章で述べる Best-First 木探索は, AI Scientist v2 に実装されている機能である. 本研究ではこの探索機構をそのまま継承し, Split-Phase 実行モデルおよび階層記憶機構と統合して使用する. 以下では, AI Scientist v2 における木探索の設計と動作を説明する.

2.3.1 設計動機

なぜ木探索が必要か

研究アイデアの実装には多くの設計決定 (アルゴリズム選択, データ構造, 最適化戦略, パラメータ設定) が伴い, その組み合わせは広大な空間を形成する. HPC 環境では 1 回の試行コストが高いため, 無作為な試行は非効率である.

木探索により, 複数の実装候補を体系的に探索し, 評価に基づいて有望な方向に計算資源を集中できる. 各ノードの実行結果と記憶を構造化して保持することで, 過去の試行から学習し, 効率的に探索を進められる.

なぜ Best-First か

探索アルゴリズムの選択において, 以下の選択肢を検討した:

- **深さ優先探索**: メモリ効率は良いが, 局所最適解に陥りやすい
- **幅優先探索**: 網羅的だが, 計算コストが高い
- **Best-First 探索**: 評価関数に基づき有望なノードを優先するため, 効率的

研究実験の文脈では, 各ノードの実行に大きなコストがかかるため, 有望なノードへの資源集中が重要である. したがって, Best-First 探索を採用した.

2.3.2 探索空間の定義

定義 2.1 (探索木). 探索木 $\mathcal{T} = (V, E)$ は以下のように定義される：

- V ：ノードの集合. 各ノード $v \in V$ は実装状態を表す
- $E \subseteq V \times V$ ：エッジの集合. 親ノードから子ノードへの派生を表す
- ルートノード $r \in V$ ：初期状態（テンプレート）
- 評価関数 $f: V \rightarrow \mathbb{R}$ ：各ノードのスコア

2.3.3 探索ステージ

探索は 4 つのステージを経て進行する. 各ステージは異なる目的を持つ.

表 2.2 に、各ステージの内部名称、目的、および具体的な操作を示す.

表 2.2 探索ステージの定義

Stage	内部名称	目的	主な操作
1	initial_implementation	初期実装生成	Draft/Debug ノード
2	baseline_tuning	ベースライン調整	Hyperparam ノード
3	creative_research	創造的改良	Improve ノード
4	ablation_studies	アブレーション	Ablation ノード

Stage 1：初期実装（initial_implementation）

目的：研究アイデアの複数の初期実装を生成し、機能性を検証する

操作：ルートノードから n 個の子ノード（Draft ノード）を生成. 失敗したノードには Debug ノードを生成してエラー修正を試みる.

根拠：同じアイデアでも、異なるアルゴリズムやデータ構造の選択により、性能が大きく異なる可能性がある. 複数の初期案を試すことで、有望な方向を早期に発見できる.

Stage 2：ベースライン調整（baseline_tuning）

目的：有望な実装のハイパーパラメータを最適化し、追加データセットでの評価を行う

操作：高スコアノードを選択し、異なるパラメータ設定の Hyperparam ノードを生成

根拠：アルゴリズムの性能はハイパーパラメータに大きく依存する. 探索により最適な設定を発見できる.

Stage 3：創造的改良（creative_research）

Stage 3 は、実装を反復的に改良し、実験計画を実行するステージである. 成功ノードに対してはさらなる性能向上を試みる Improve ノードを、失敗ノードに対してはデバッグを試みる Debug ノードを生成する. 初期実装は完璧ではないことが多く、反復的な改良により品質を段階的に向上させることが本ステージ

の意義である。

Stage 4：アブレーション（ablation_studies）

Stage 4 は、実装の各コンポーネントの寄与を検証し、リスク要因を確認するステージである。最良ノードから特定のコンポーネントを削除または変更した Ablation ノードを生成する。研究論文において、提案手法の各要素がどの程度寄与しているかを示すアブレーション実験は不可欠であり、本ステージはその体系的な実施を自動化する。

2.3.4 ノードの構造

探索木の各ノードは、実装状態とその評価結果を保持するデータ構造である。表 2.3 に、ノードの主要な属性を示す。

表 2.3 ノードの主要属性

カテゴリ	属性	説明
基本情報	id, step, ctime	一意 ID, ステップ番号, 作成時刻
関係	parent, children, branch_id	親ノード, 子ノード, 記憶ブランチ
コード	code, plan, phase_artifacts	生成コード, 計画, フェーズ成果物
実行	term_out, exec_time, exc_type	出力, 実行時間, 例外情報
評価	metric, analysis, is_buggy	メトリクス, 分析結果, バグフラグ
VLM	plot_analyses, vlm_feedback	プロット分析, VLM フィードバック
特殊	ablation_name, is_seed_node	アブレーション名, シードノードフラグ
継承	inherited_from_node_id	継承元ノード ID

ノードの種類

表 2.4 に、探索で生成されるノードの種類とその役割を示す。

表 2.4 ノードの種類

種類	目的	親からの入力
Draft	初期実装を生成	アイデア記述
Debug	エラーを修正	失敗コード + エラーログ
Improve	性能を改善	動作コード + メトリクス
Hyperparam	パラメータ調整	ベストコード + 調整履歴
Ablation	構成要素の効果検証	最終コード

2.3.5 ワーカーとノードのライフサイクル

並列実行において、ワーカー（プロセス）とノード（探索木の実装状態）は多対多の関係にある。各ワーカーのライフサイクルは以下の通りである。まず、AgentManager が Best-First 方針でノードを選択しワーカーに割り当て、GPU リソースのマッピングを行う。次に、親ノードのブランチから子ブランチを作成して記憶の継承を確立し、ワーカー固有の Singularity コンテナを起動する。前ステージの成果物をワークスペースに継承した後、Split-Phase 実行モデルに基づく Phase 0-4 の実行とポスト処理（メトリクス抽出、プロット生成、VLM 分析、ノードサマリー）を行う。最後に、ワークスペースの成果物を永続化する。実行がタイムアウトした場合は、中間結果を保存して部分的な結果を活用する。

この設計により、ワーカーの状態とノードの成果物が分離され、ワーカーの再利用が可能になる。

2.3.6 マルチシード評価

各主要ステージの完了時に、最良ノードはマルチシード評価を受ける。これは、単一の乱数シードに依存しない堅牢な評価を行うための機構である。HPC 実験では乱数初期化や測定ノイズにより結果が変動するため、複数シードでの再現性確認は結果の信頼性を担保する上で不可欠である。

評価フロー

1. 最良ノードを選択
2. 複数の異なるシード値（デフォルト 3 個）で同一コードを実行
3. 各シードでのメトリクスを収集
4. シード集約ノードを作成し、平均・標準偏差・統計プロットを生成

マルチシード評価により、結果の再現性と統計的有意性を確保できる。主要な設定パラメータとして、各ステージの最大反復回数（デフォルト各 40 回）、初期ドラフト数（デフォルト 3）、デバッグ探索の最大深度（デフォルト 3）、評価シード数（デフォルト 3）などが設定ファイルにて調整可能である。

2.3.7 探索アルゴリズム

2.3.8 ノード選択戦略

最良ノードの選択は、単純なスコア比較だけでなく、以下の要素を考慮する：

ステージ適合性

各ノードは特定のステージで生成され、そのステージに適した操作のみが適用される。例えば、Stage 1 で生成されたノードは、Stage 2（ハイパーパラメータ調整）の対象となりうる。

探索の多様性

高スコアノードへの過度な集中を避けるため、確率的な選択を導入する。具体的には、デバッグ探索（失敗ノードの修復）と改良探索（成功ノードの発展）を確率的に選択する。

Algorithm 1 Best-First 木探索 (`AgentManager.step + _select_parallel_nodes`)

入力: 研究アイデア I , 評価関数 f , ステージ設定 $\{S_1, S_2, S_3, S_4\}$, ワーカー数 W , デバッグ確率 p_d

出力: 最良ノード v^*

```
1:  $journal \leftarrow \text{Journal}()$ 
2: for  $stage$  in  $[S_1, S_2, S_3, S_4]$  do
3:   for  $iter = 1$  to  $stage.max\_iters$  do
4:                                     ▷ 並列ノード選択 (_select_parallel_nodes)
5:      $selected \leftarrow []$ 
6:     while  $|selected| < W$  do
7:       if  $|journal.draft\_nodes| < num\_drafts$  then
8:          $selected.add(\text{None})$                                      ▷ 新規ルートノード作成シグナル
9:       else if  $\text{rand}() < p_d$  and  $\exists$  バグ付きリーフ then
10:         $selected.add(\text{RandomChoice}(\text{buggy\_leaves}))$            ▷ Debug
11:       else if  $stage = S_4$  then
12:         $selected.add(\text{best\_stage3\_node})$                        ▷ Ablation
13:       else if  $stage = S_2$  then
14:         $selected.add(\text{best\_stage1\_node})$                        ▷ Hyperparam
15:       else
16:         $selected.add(journal.get\_best\_node())$                    ▷ Improve
17:       end if
18:     end while
19:
20:                                     ▷ 並列実行 (各ワーカーが Singularity コンテナ内で Phase 0–4 を実行)
21:     for  $parent$  in  $selected$  parallel do
22:        $b_{child} \leftarrow \text{ForkBranch}(parent.branch\_id)$          ▷ 記憶ブランチ分離
23:        $plan \leftarrow \text{PHASE0PLANNING}(I, parent, b_{child})$ 
24:                                     ▷ ノード種別に応じたコード生成
25:       if  $parent = \text{None}$  then
26:          $child \leftarrow \text{DRAFT}(I)$ 
27:       else if  $parent.is\_buggy$  then
28:          $child \leftarrow \text{DEBUG}(parent)$ 
29:       else
30:          $child \leftarrow \text{NODETYPEDISPATCH}(parent, stage)$      ▷ Improve/Hyperparam/Ablation
31:       end if
32:        $child.score \leftarrow f(\text{SPLITPHASEEXECUTE}(child))$ 
33:        $journal.append(child)$ 
34:     end for
35:   end for
36: end for
37:  $v^* \leftarrow \text{argmax}_{v \in journal} f(v)$ 
38: return  $v^*$ 
```

2.3.9 評価関数

評価関数は、実装の品質を数値化する。本システムでは、以下の要素を組み合わせる：

1. **実行成功**：コンパイル・実行が成功したか
2. **性能指標**：実行時間、精度、メモリ使用量など（タスク依存）
3. **コード品質**：LLM による評価（可読性、保守性など）

図 2.2 に、Best-First 木探索の例を示す。図中の色はステージ（Stage 1: 初期実装, Stage 2: ベースライン調整, Stage 3: 創造的改良, Stage 4: アブレーション）を表し、太線は最良パス（最高評価値を達成したノードからルートへの経路）を示している。

各ノードは種類（Draft/Debug/Improve/Hyperparam/Ablation）と Phase 0-4 の実行結果を保持する。ステージ内では評価値に基づく Best-First 選択が行われ、最も有望なノードが優先的に展開される。ステージ昇格時には、現在のステージの最良ノードを親として、次ステージのノードが生成される。この仕組みにより、段階的に実装を洗練しながら、効率的に探索空間を探索できる。

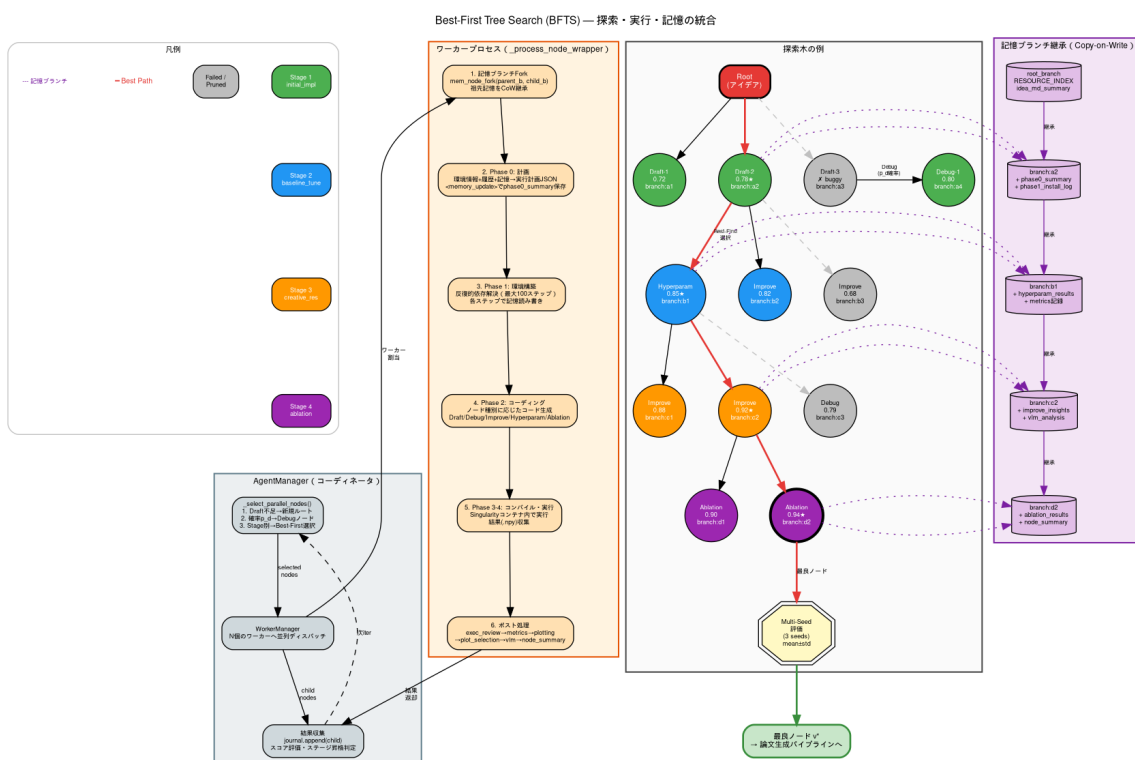


図 2.2 Best-First 木探索の例 (Stage 色分け, 最良パス表示)

第 3 章

提案システム

本章では、AI Scientist v2 の問題点を解決するために提案する HPC-AutoResearch システムの設計と実装について述べる。

3.1 システムアーキテクチャ

3.1.1 設計原理

本システムの設計は、以下の 4 つの原理に基づいている。

原理 1：関心の分離（Separation of Concerns）

複雑なシステムを理解・保守・拡張可能にするため、異なる責務を持つコンポーネントを明確に分離する。本システムでは、以下の分離を行う：

- **計画と実行の分離**：何をすべきかを決定するコンポーネント（プランナー）と、実際に実行するコンポーネント（エグゼキュータ）を分離する。これにより、計画の修正と実行の最適化を独立に行える。
- **記憶と推論の分離**：情報の保存・検索を担当するコンポーネント（記憶管理）と、その情報を用いて判断を行うコンポーネント（LLM 推論）を分離する。これにより、記憶機構の改善が LLM の動作に透過的に反映される。
- **探索と評価の分離**：探索空間を展開するコンポーネント（木探索エンジン）と、各ノードの品質を評価するコンポーネント（評価器）を分離する。これにより、異なる評価指標への切り替えが容易になる。

原理 2：段階的抽象化（Progressive Abstraction）

複雑なタスクを一度に解決するのではなく、段階的に詳細化していく。本システムでは、研究プロセスを以下の抽象度で扱う：

1. **アイデアレベル**：研究の目的と仮説（最も抽象的）
2. **計画レベル**：実現方法の概要と必要な依存関係
3. **実装レベル**：具体的なコードと実行手順（最も具体的）

この段階的抽象化により、Chain-of-Thought[4] の知見と同様に、LLM は各段階で適切な粒度の問題に集中でき、複雑なタスクの成功率が向上する。

原理 3：失敗からの学習（Learning from Failures）

エラーは避けるべきものではなく、システムを改善するための情報源として活用する。本システムでは、各フェーズの失敗情報を構造化して記憶に記録し、類似状況で参照可能にする。さらに、失敗したノードから派生するデバッグノードを生成し、記録された失敗パターンを参照しながら修正を試みることで、同種のエラーの再発を抑制する。

原理 4：環境適応性（Environment Adaptability）

異なる HPC 環境での動作を保証するため、環境依存の処理を抽象化し、実行時に適応する。具体的には、Phase 0 において環境情報（CPU、メモリ、コンパイラ、ライブラリ等）を実行時に自動収集して LLM に提供し、Singularity[5] コンテナ技術によりホスト環境からの独立性を確保する。また、依存解決を反復的なプロセスとして設計することで、環境固有の制約（利用可能なパッケージマネージャ、ネットワークアクセスの有無等）に動的に対応する。

3.1.2 アーキテクチャ概要

図 3.1 に、システム全体のアーキテクチャを示す。システムは大きく 4 つのレイヤーから構成される。

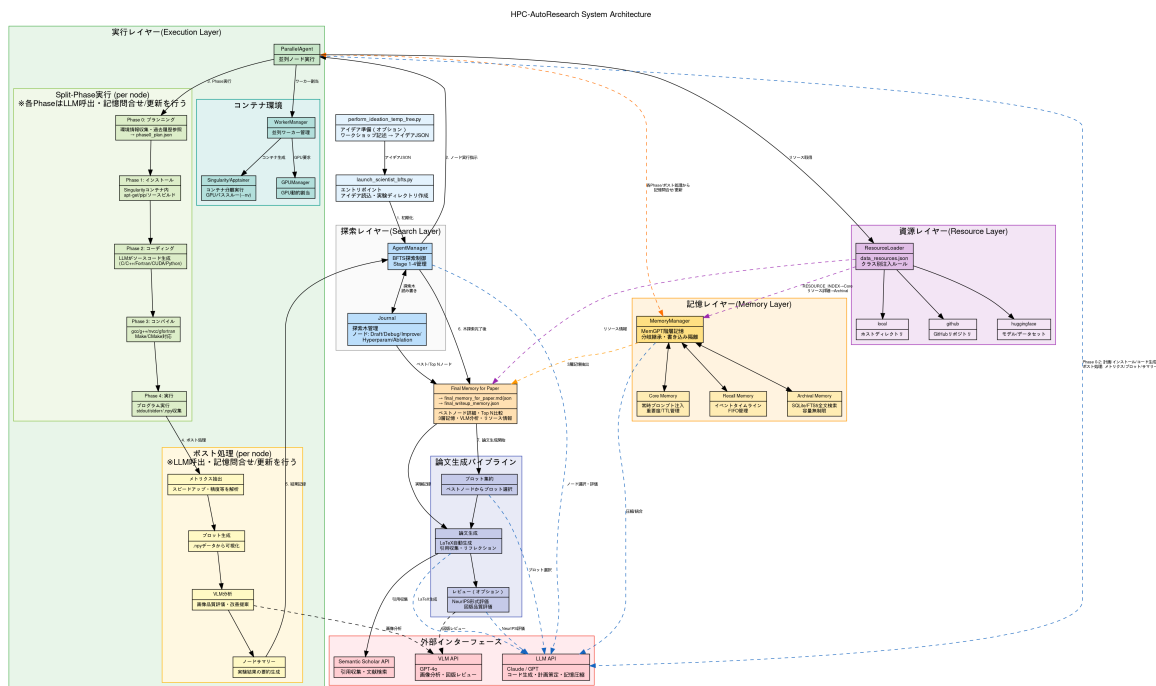


図 3.1 システムアーキテクチャの概要。

図 3.1 にシステム全体のアーキテクチャを示す。実線矢印はデータ・制御の主要フローを、青色破線は LLM/VLM API 呼出を、橙色破線は記憶レイヤーへの問合せ・更新を、紫色破線はリソース関連の接続をそ

れぞれ示す。エントリポイントから AgentManager が探索木 (Journal) を管理しつつ ParallelAgent へノード実行を指示し、ParallelAgent は Split-Phase (Phase 0-4) およびポスト処理を順次実行する。各 Phase およびポスト処理は LLM API を呼び出してコード生成・計画策定・メトリクス解析等を行い、記憶が有効な場合は MemoryManager を介して Core/Recall/Archival 記憶の問合せ・更新を行う。ResourceLoader は外部データソース (local/GitHub/HuggingFace) を取得し、リソースインデックスを Core 記憶へ、詳細情報を Archival 記憶へ注入する。木探索完了後、AgentManager・Journal・MemoryManager・ResourceLoader の情報が Final Memory for Paper に集約され、論文生成パイプライン (プロット集約→LaTeX 生成→レビュー) へ引き渡される。

3.1.3 全体ワークフロー

本システムの全体ワークフローは、アイデア準備からレビューまでの 6 つの主要ステップから構成される。

ワークフロー概要

1. **アイデア準備** (オプション): ワークショップ記述から LLM がアイデア JSON を生成
2. **実験ランチャー**: アイデア JSON を読み込み、実験ディレクトリを作成し、BFTS 実験を開始
3. **BFTS 実験**: 木探索による実験の自動実行 (Stage 1-4)
4. **プロット集約**: 複数ノードから生成されたプロットを選択・集約
5. **論文生成**: 実験結果から LaTeX 論文を自動生成
6. **レビュー** (オプション): LLM による論文レビューおよび VLM による図版レビュー

各ステップの詳細

BFTS 実験では、各ノードにおいて Split-Phase 実行モデルに基づく 5 段階の処理 (Phase 0: プランニング, Phase 1: 環境構築, Phase 2: コーディング, Phase 3: コンパイル, Phase 4: 実行) が順次実行され、その後メトリクス抽出、プロット生成、VLM 分析、ノードサマリー生成のポスト処理が行われる。

アイデア生成プロセス

ワークフローの最初のステップであるアイデア準備は、ワークショップ記述 (Markdown ファイル) を入力として、LLM が JSON 形式の研究アイデアを生成するオプションなフェーズである。既存のアイデア JSON を直接指定する場合はスキップ可能である。

アイデア生成は `perform_ideation_temp_free.py` により実行され、以下のプロセスで構成される:

1. **生成**: ワークショップ記述と既存のアイデアアーカイブを入力として、LLM が新規研究提案を生成する
2. **文献調査**: Semantic Scholar API[6] を用いた関連文献の検索ツールが利用可能であり、LLM は生成した提案の新規性と位置づけを確認できる
3. **リフレクション**: Reflexion[7] に着想を得た自己改善機構により、設定された回数 (デフォルト 5 ラウンド) にわたり、LLM が提案の質・新規性・実現可能性を自己評価し改善する

4. **確定**：LLM が `FinalizeIdea` ツールを呼び出し、構造化されたアイデア JSON を出力する

生成されるアイデア JSON は以下のフィールドを含む：

- **Name**：短い識別子（小文字，アンダースコア区切り）
- **Title**：研究タイトル
- **Short Hypothesis**：研究仮説の簡潔な記述
- **Related Work**：関連研究との差別化
- **Abstract**：約 250 語のアブストラクト
- **Experiments**：検証実験の詳細（評価指標を含む）
- **Risk Factors and Limitations**：リスク要因と制約

このプロセスにより、最大 `max_num_generations` 個のアイデアが生成され、JSON ファイルに保存される。実験ランチャーは `--load_ideas` オプションでこのファイルを読み込み、`--idea_idx` で指定されたアイデアに基づいて BFTS 実験を開始する。

なお、アイデア生成フェーズでは MemGPT 記憶機構は使用されない。これはアイデア生成が BFTS 実験の前段階として独立して実行され、記憶の継承対象となるノードがまだ存在しないためである。

システムレイヤー構成

本システムは 4 つのレイヤーから構成される。**探索レイヤー**は、Best-First 木探索アルゴリズムに基づき、評価値の高いノードを優先的に展開しながら実装空間を探索する。探索は 4 つのステージ（初期実装・ベースライン調整・創造的改良・アブレーション）を経て進行し、各ステージで異なる探索戦略を適用する。**実行レイヤー**は、各ノードの実装を Split-Phase 実行モデルに基づいて Singularity コンテナ内で実行し、評価値を算出する。**記憶レイヤー**は、階層記憶機構によりコア記憶・リコール記憶・アーカイブ記憶の 3 層で実験履歴と知見を管理し、LLM 呼び出し時に適切な記憶を注入して長期的な一貫性を維持する。**資源レイヤー**は、複数ワーカーの並列実行、GPU 資源の動的割当、データセットやライブラリの宣言的管理を担当する。

3.1.4 リソースファイル機構

HPC 研究では、大規模データセット、参照実装、事前学習モデルなどの外部リソースを実験で使うことが一般的である。これらを LLM 生成コード内にハードコードすると、パスの変更やリソースの更新に弱くなる。リソースファイル機構は、外部データ、リポジトリ、モデルを**実験設定として宣言的に管理**し、プロンプトとコンテナの両方に一貫して注入するための機構である。

リソース定義

リソースは `data_resources.json` ファイルにて定義される。4 種類のリソースタイプをサポートする：

1. **local**：ホスト上のローカルディレクトリ（読み取り専用でバインドマウント）
2. **github**：GitHub リポジトリからのクローン（固定 ref/commit/tag を推奨）
3. **huggingface**：Hugging Face モデルまたはデータセットのダウンロード

4. items：上記リソース内の特定ファイル/ディレクトリを分類

リソースクラスと注入ルール

各 item には class 属性が付与され、フェーズごとの注入ルールが適用される。表 3.1 に、クラス別の注入ルールを示す。

表 3.1 リソースクラス別の注入ルール

Class	Phase 0	Phase 1	Phase 2	Phase 3/4
template	○	○	○	-
document	○	○	○	○
setup	○	○	-	-
library	meta only	○	-	-
dataset	meta only	○	-	-
model	meta only	○	-	-

- **template**：テンプレートコード。Phase 0-2 でプロンプトに内容を注入
- **document**：ドキュメント。全フェーズで参照可能
- **setup**：セットアップスクリプト。Phase 0-1 で参照
- **library/dataset/model**：Phase 0 ではメタ情報のみ、Phase 1 でマウント

マウントとフェッチの動作

リソースタイプによって実行時の動作が異なる。ローカルリソースはホストディレクトリを Singularity コンテナにバインドマウントすることで利用可能となる。GitHub および Hugging Face リソースは、Phase 1（環境構築）においてコンテナ内にフェッチされる。すべてのリソースパスはワークスペース配下に制約され、ホスト環境への影響を防止する。

ローカルリソースのうち、template, setup, document クラスに分類されるものは、実行前にステージングディレクトリにコピーされ、表 3.1 に示す注入ルールに従って LLM プロンプトに含められる。

3.1.5 ペルソナシステム

AI Scientist v2 は汎用的な設計を志向しているが、プロンプトや評価基準は主に機械学習実験を想定しており、HPC 特有の研究領域（数値シミュレーション、性能最適化等）に対応していない。ペルソナシステムは、プロンプト全体の専門領域を切り替えることで、LLM の出力を対象ドメインに適合させるための機構である。設定ファイルにて役割記述（例：“HPC Researcher”）を指定すると、すべてのプロンプト内のペルソナトークンが指定された役割に再帰的に置換される。これにより、同一のシステム基盤を用いながら、異なる研究領域への適応が可能となる。

3.2 Split-Phase 実行モデル

3.2.1 設計動機

なぜフェーズ分割が必要か

LLM に複雑なタスクを与えた場合、以下の問題が発生しやすい：

1. **注意の分散**：複数の懸念事項を同時に処理しようとする、重要な詳細が見落とされる
2. **エラーの連鎖**：初期段階のエラーが後続の処理に伝播し、デバッグが困難になる
3. **文脈の肥大化**：すべての情報を一度に与えると、文脈長の制限に達する

これらの問題に対処するため、研究プロセスを複数のフェーズに分割し、各フェーズで単一の責務に集中させる設計を採用した。

なぜ5つのフェーズか

フェーズの数は、以下の考慮に基づいて決定した：

- **少なすぎる場合**：各フェーズの責務が大きくなり、分割の効果が薄れる
- **多すぎる場合**：フェーズ間のオーバーヘッド（文脈の引き継ぎ、状態の保存）が増大する

5つのフェーズ（計画・環境構築・コーディング・コンパイル・実行）は、HPC 環境での研究プロセスの自然な分割点に対応している：

1. 計画と実行は本質的に異なる認知タスクである
2. 環境構築は、コーディングの前提条件を満たす独立したタスクである
3. コンパイル型言語では、コーディングとコンパイルは分離可能である
4. 実行は、コンパイル成功後にのみ意味を持つ

3.2.2 フェーズの定義

定義 3.1 (Split-Phase 実行モデル). *Split-Phase* 実行モデルは、研究実行プロセスを 5つのフェーズ $\mathcal{P} = \{P_0, P_1, P_2, P_3, P_4\}$ に分割する。各フェーズは以下のように定義される：

- P_0 (計画)：環境情報と研究アイデアから実行計画を生成
- P_1 (環境構築)：計画に基づき依存関係を反復的に解決
- P_2 (コーディング)：計画に基づきソースコードを生成
- P_3 (コンパイル)：生成されたコードをビルド
- P_4 (実行)：ビルド成果物を実行し結果を収集

図 3.2 に、Split-Phase 実行モデルのフローを示す。

継承しており、Phase 0 はその記憶を参照してノード固有の計画を策定する。

入力として研究アイデア記述、環境情報、過去の実行履歴、および継承された記憶コンテキストを受け取り、出力として依存関係リスト、実装戦略、各フェーズへのガイダンスを含む構造化された実行計画を生成する。

環境情報の収集

LLM が適切な計画を立てるためには、実行環境の詳細な情報が必要である。Phase 0 では、Singularity コンテナ内で以下のコマンドを実行し、環境情報を自動収集する。

基本システム情報：

- OS 情報：`cat /etc/os-release`
- CPU 情報：`lscpu` (モデル名, ソケット数, コア数, スレッド数, キャッシュサイズ, NUMA 情報)
- メモリ情報：`free -h` および `cat /proc/meminfo` (総 RAM, 利用可能メモリ)
- CPU ガバナー：`cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor`
- NUMA 構成：`numactl --hardware` (NUMA ノード数, メモリ配置, 距離行列)

コンパイラ・ライブラリ情報：

- 利用可能コンパイラ：`which gcc/g++/clang/clang++/nvcc/gfortran/mpicc` および `--version`
- 共有ライブラリ：`ldconfig -p | grep <lib_name>` (libopenblas, libmpi, libcuda 等)
- システムパッケージ：`dpkg-query` (Debian/Ubuntu) または該当パッケージマネージャ

性能解析ツール：

- 利用可能ツール：`which numactl/perf/taskset/likwid-pin/hwloc-ls`
- 各ツールの動作確認：バージョンフラグによる機能検証

GPU・ネットワーク情報：

- GPU 情報：`nvidia-smi -L` (GPU モデル, 割当 GPU ID, CUDA_VISIBLE_DEVICES)
- ネットワーク：`getent hosts github.com` (外部アクセス可否)
- 環境変数：`env | sort` (PATH, LD_LIBRARY_PATH 等)

これらの情報は、表 3.2 に示すように LLM プロンプトに構造化して注入される。LLM はこの情報を基に、環境に適した実装戦略を選択する。例えば、NUMA ノードが複数存在する場合は NUMA-aware なメモリ配置を、`perf` が利用可能であればハードウェアカウンタによる性能計測を計画に組み込む。

フェーズガイダンスの生成

Phase 0 の重要な出力として、後続フェーズへのガイダンスがある。これは各フェーズに対して以下の情報を提供する：

- **目標**：そのフェーズで達成すべき具体的な目標
- **推奨コマンド**：実行すべきコマンドの候補

表 3.2 環境情報の注入項目

項目	内容例
cpu_info	Intel Xeon E5-2680 v4, 28 cores, 2 sockets
numa_config	2 NUMA nodes, 64GB per node
memory_info	128GB total, 120GB available
available_compilers	gcc 11.4.0, nvcc 12.0, mpicc
system_performance_tools	perf, numactl, taskset, likwid-pin
gpu_info	NVIDIA A100 40GB, CUDA 12.0

- **完了条件**：フェーズ完了を判定する条件

このガイダンスにより、後続フェーズの LLM 呼び出しで必要な文脈量を削減できる。

3.2.4 Phase 1：環境構築フェーズ

反復的依存解決の必要性

HPC 環境での依存解決は、以下の理由から複雑である：

1. root 権限がないため、システムレベルのパッケージ管理が制限される
2. 一部の依存関係は他の依存関係に依存する（推移的依存関係）
3. エラーメッセージから必要な依存関係を推測する必要がある

これらに対処するため、Phase 1 は ReAct[8] パターン（推論→行動→観察の反復）に基づく反復的なアルゴリズムとして設計されている。

アルゴリズム

Phase 1 は記憶機構と連携した反復的アルゴリズムとして実装されている。各イテレーションで LLM は記憶の読み書きを行い、過去の経験を活用して依存解決を進める。

記憶機構との連携

Phase 1 では、記憶機構の有効/無効に応じて異なるプロンプトが選択される。記憶有効時には、LLM は各イテレーションにおいて記憶更新（記憶の読み書き操作）とコマンド出力の 2 部構成で応答する。これにより、過去のインストール経験を検索・活用しながら依存解決を進めることが可能となる。

Phase 1 における記憶操作

Phase 1 の各イテレーションにおいて、LLM はコア記憶への書き込み・読み取り、およびアーカイブ記憶への書き込み・検索を実行できる。典型的な使用パターンとして、インストール完了後のパッケージバージョンや Git コミット SHA のコア記憶への記録、失敗したアプローチとその回避策のアーカイブへの保存、および同様のエラーに対する過去の解決策のアーカイブ検索が挙げられる。これらの操作により、複数の実行を通じて依存解決のベストプラクティスが蓄積される。

Algorithm 2 記憶連携型反復依存解決のアルゴリズム (Phase 1 : phase1_iterative_driver)

入力: Phase 0 計画 π (ガイダンス含む), 最大ステップ数 N (phase1_max_steps), 環境情報 E , 記憶マネージャ $\mathcal{M}$, ブランチ ID b

出力: 依存解決の成否, 更新された記憶

```
1:  $history \leftarrow []$ 
2:                                     ▷ 記憶有効時は phase1_installer_with_memory.txt を使用
3:  $intro \leftarrow \text{SELECTPROMPT}(\mathcal{M}.enabled)$ 
4: for  $step = 0$  to  $N - 1$  do
5:     ▷ プロンプト構築: 導入 + タスク記述 + Phase 0 ガイダンス + 進捗履歴 + 記憶コンテキスト
6:      $mem\_ctx \leftarrow \text{INJECTMEMORY}(\mathcal{M}, b, \text{"phase1\_iterative"}, budget\_chars)$ 
7:      $prompt \leftarrow \text{BUILDPROMPT}(intro, \pi.guidance, history, E, mem\_ctx)$ 
8:      $response \leftarrow \text{LLM}(prompt)$ 
9:     if  $\mathcal{M}.enabled$  then
10:                                     ▷ <memory_update>ブロックの抽出と適用
11:          $mem\_ops \leftarrow \text{EXTRACTMEMORYUPDATES}(response)$ 
12:          $results \leftarrow \text{APPLYLLMMEMORYUPDATES}(\mathcal{M}, b, mem\_ops)$ 
13:         if  $\text{HASMEMORYREADRESULTS}(results)$  then
14:             ▷ 読み取り操作の結果を注入して再クエリ (最大 max_memory_read_rounds 回)
15:              $\text{RUNMEMORYUPDATEPHASE}(prompt, \mathcal{M}, b, \text{"phase1\_iterative"})$ 
16:         end if
17:     end if
18:      $cmd \leftarrow \text{PARSEPHASE1RESPONSE}(response)$                                      ▷ {command, done, notes}
19:     if  $cmd.done = \text{true}$  then
20:         return SUCCESS
21:     end if
22:      $output \leftarrow \text{EXECUTEINCONTAINER}(cmd.command)$                                      ▷ Singularity 内で実行
23:      $history.append((cmd, output))$ 
24: end for
25: return FAILURE
```

読み取り操作の再クエリフロー

LLM が読み取り操作を含む記憶更新を出力した場合, システムはマルチターン再クエリを実行する. 具体的には, 書き込み操作を永続化した上で読み取り結果を収集し, その結果をプロンプトに注入して LLM を再クエリする. この処理は最大ラウンド数 (デフォルト 5 回) に達するまで繰り返される. この機構により, LLM は過去の Phase 1 実行で得られた知見 (例: 特定のパッケージのインストール方法, エラー回避策) を検索し, 現在の依存解決に活用できる.

Phase 1 で実行されるコマンドは, パッケージインストール (apt-get, pip, conda 等), ソースビルド,

データセットやモデルのダウンロード，および依存関係の検証に大別される．LLM は各ステップで最も適切なコマンドを選択し，エラーが発生した場合は代替手段を試みる．

3.2.5 Phase 2/3/4：コーディング・コンパイル・実行

統合 LLM 呼び出し

Phase 2（コーディング），Phase 3（コンパイル），Phase 4（実行）は，効率性の観点から単一の LLM 呼び出しで処理される．これは以下の理由による：

1. これら 3 つのフェーズは密接に関連しており，コードの構造がコンパイル方法と実行方法に影響する
2. 3 回の個別呼び出しは，文脈の重複によりトークン消費が増大する
3. 統合することで，LLM が全体の整合性を考慮した出力を生成できる

出力形式

LLM は以下の構造化された JSON 出力を生成する．この出力は `phase_artifacts` としてノード属性に保存され，各フェーズの実行に使用される：

1. **coding**：生成するファイルの一覧と内容（ファイルパスとソースコードの辞書）
2. **compile**：ビルドに必要なコマンド列（シェルコマンドのリスト）
3. **run**：実行コマンドと期待される出力ファイル（`{experiment_name}_data.npy` 形式の動的ファイル名）

出力ファイル名は実験名に基づいて動的に生成される．例えば実験名が `stability_oriented_autotuning_v2` の場合，出力ファイルは `stability_oriented_autotuning_v2_data.npy` となる．この規約により，複数の実験結果を一意に識別できる．

失敗時の処理

コンパイルまたは実行が失敗した場合，システムはエラーメッセージ，失敗したコマンドと終了コード，生成されたファイルの状態を収集する．これらの情報は記憶に保存され，デバッグノードの生成時に参照される．

3.2.6 記憶との連携

Split-Phase 実行モデルは，階層記憶機構と密接に連携する．表 3.3 に，各フェーズにおける記憶の利用方法を示す．

フェーズ別記憶連携

表 3.3 フェーズ別記憶連携

フェーズ	記憶注入	LLM 記憶操作	特記事項
Phase 0	○	○	計画要約をコア記憶に保存
Phase 1	○	○	イテレーション毎に読み書き可能
Phase 2/3/4	○	○	統合 LLM 呼び出しで処理

Phase 1 の特殊性

Phase 1 は他のフェーズと異なり、**反復的な LLM 呼び出し**を行う唯一のフェーズである。各イテレーションで：

1. 記憶コンテキストがプロンプトに注入される (`_inject_memory()`)
2. LLM が`<memory_update>`ブロックで記憶を操作
3. 読み取り操作がある場合、再クエリループを実行
4. コマンドを実行し、結果を履歴に追加
5. 次のイテレーションで最新の記憶が利用可能

この設計により、Phase 1 は過去のインストール経験（成功したパッケージバージョン、エラー回避策など）を蓄積し、後続の実行で活用できる。

記憶の永続化タイミング

- **Phase 0 完了時**：計画の要約がコア記憶に、詳細が `PHASE0_INTERNAL` タグでアーカイブに保存
- **Phase 1 各イテレーション**：LLM の`<memory_update>`により随時保存（パッケージバージョン、コミット SHA 等）
- **Phase 1 完了時**：`phase1_complete` イベントがリコール記憶に記録
- **コンパイル/実行エラー時**：詳細なエラー情報が `ERROR` タグでアーカイブに保存
- **ノード完了時**：成功時は `SUCCESS` タグ、メトリクスが `METRICS` タグでアーカイブに保存

この連携により、過去の実行から学習し、同様のエラーを回避できる。特に Phase 1 では、依存解決の知見が蓄積され、類似環境での実験効率が向上する。

3.2.7 ワークスペース継承

探索木において、あるステージで最良と判定されたノードのワークスペース（ソースコード、コンパイル済みバイナリ、中間生成物）は、次のステージの開始点として継承される必要がある。しかし、並列ワーカーが同時にワークスペースを操作する環境では、競合状態（race condition）が発生しうる。

設計上の問題

当初の実装では、各ワーカーのワークスペースパス（`worker_0/`, `worker_1/`等）を直接参照していたため、ノードの実行完了後にワーカーが次のノードの処理を開始すると前のノードの成果物が上書きされ、`[Errno 2] No such file or directory` エラーが発生していた。

node_logs による永続化

この問題を解決するため、ノードログ機構を導入した。各ノードの実行完了時に、ワークスペースの内容（ソースコード、コンパイル済みバイナリ、入力データ、実験出力）をノード ID に紐づいた永続ディレクトリへコピーする。これにより、ワーカーの一時的な状態とノードの成果物が分離され、並列実行時の競合が解消される。

stage_best によるステージ間継承

各ステージの完了時に、最良ノードのワークスペースを `stage_best/<stage_name>/` にコピーする。次のステージのノードは、この `stage_best` ディレクトリを初期ワークスペースとして利用する。これにより、ステージを跨いだ漸進的な改良が可能となる。

`node_logs` はステージ完了後に削除されるが、`stage_best` は永続的に保持されるため、実験終了後も最良ノードの成果物を確認できる。

3.3 階層記憶機構の提案

3.3.1 設計動機

文脈長の制約

LLM の文脈長には実用的な上限がある。研究プロセスは複数のフェーズにまたがり、各フェーズで生成される情報（計画、コード、実行ログ、エラーメッセージ、評価結果など）は膨大である。すべての履歴を文脈に含めることは、現実的に不可能である。

情報の重要度の非一様性

すべての過去の情報が等しく重要なわけではない：

- **常に必要な情報**：研究目標、現在の計画、最適パラメータ
- **時々必要な情報**：最近の実験結果、エラー履歴
- **稀に必要な情報**：過去の試行の詳細、古いコードスナップショット

この非一様性を活用し、情報を適切に階層化することで、有限の文脈を効率的に利用できる。

記憶なし実行の問題

記憶機構が無効の場合、LLM の文脈には予算管理の仕組みがなく、フェーズ間で参照すべき情報が増えるにつれて文脈オーバーフローのリスクが高まる。特に Split-Phase 実行モデルでは、Phase 0 の計画、

Phase 1 のインストール履歴, Phase 2 のコード, Phase 3 のコンパイル結果など, 各フェーズの出力が後続フェーズに伝搬するため, 文脈長の制約がより深刻になる. 記憶機構は, この問題に対して**情報のアクティブな取捨選択と永続化**を提供する.

本システムでは, MemGPT の概念を拡張した 3 層記憶アーキテクチャを採用した. 記憶はデフォルトで有効に設定されている (`memory.enabled=true`).

3.3.2 3 層記憶アーキテクチャ

定義 3.2 (階層記憶システム). 階層記憶システム $\mathcal{M}$ は, 3 つの記憶層から構成される:

$$\mathcal{M} = (\mathcal{M}_{core}, \mathcal{M}_{recall}, \mathcal{M}_{archival}) \quad (3.1)$$

各層は異なる特性を持つ:

- $\mathcal{M}_{core}$: 容量制限あり, 常に文脈に注入 (キー・バリュー形式)
- $\mathcal{M}_{recall}$: 時間窓による管理, 最近のイベントをタイムラインとして保持
- $\mathcal{M}_{archival}$: 容量制限なし, 全文検索 (*FTS5*) により取得

図 3.3 に, 階層記憶機構の構造を示す.

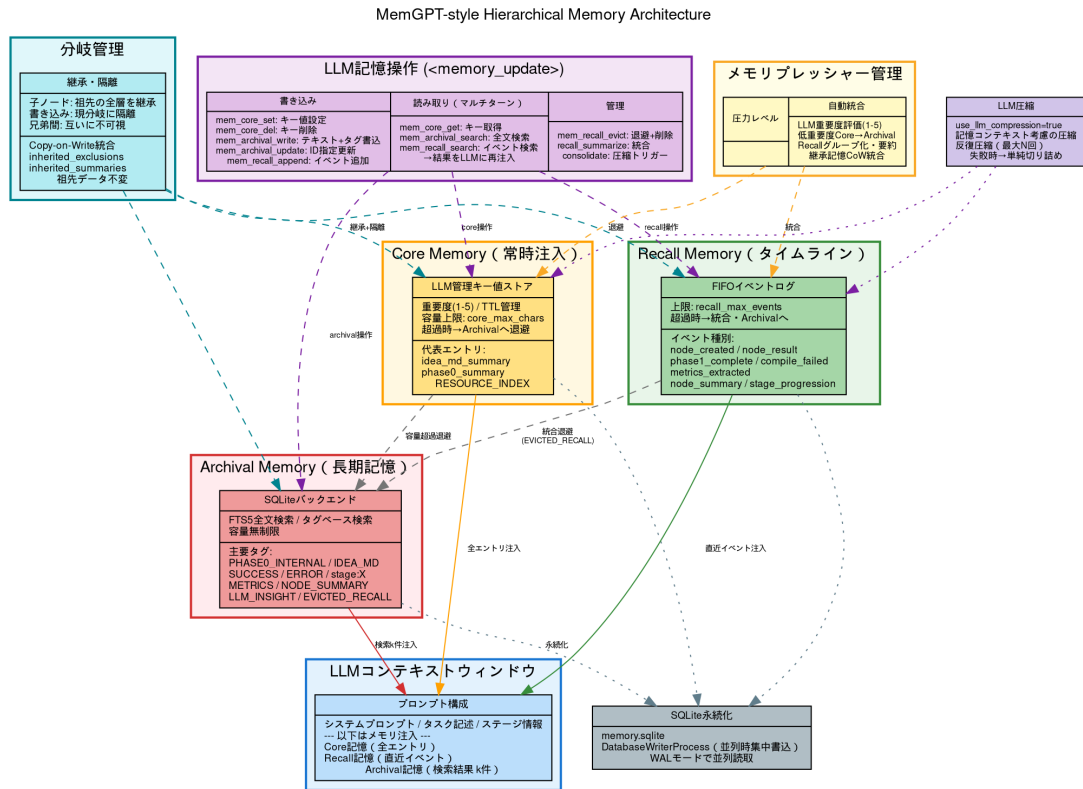


図 3.3 MemGPT 型階層記憶機構の構造. 3 層記憶 (Core/Recall/Archival) の関係と, LLM コンテキストへの注入経路を示す. コア記憶は全エントリが常時注入されるキー・バリュースタで, 重要度 (1-5) と TTL により管理される. リコール記憶は FIFO イベントタイムラインで直近イベントが注入される. アーカイブ記憶は SQLite バックエンド (FTS5 全文検索) による容量無制限の長期記憶で, 検索上位 k 件が注入される. LLM は<memory_update>ブロックにより `mem_core_set` 等の操作を自律的に実行する. コア記憶の容量超過時はアーカイブへの自動退避, リコール記憶は統合退避が行われる. 分岐管理では Copy-on-Write 方式により子ノードが祖先の全層を継承しつつ書き込みを隔離する. メモリプレッシャー管理は low/medium/high/critical の 4 段階で自動統合を制御する.

コア記憶 (Core Memory)

コア記憶は, 常に LLM の文脈に含まれる重要情報を保持するキー・バリュースタである.

コア記憶はキー・バリュー形式で情報を保持する. 保存されるキーは, リソースインデックス (自動保存) と LLM が自律的に記録する任意のキー (例: 最適スレッド数, コンパイラフラグ) に大別される. 各エントリは, 識別キー, 値 (文字列), 重要度 (1-5), および有効期限 (オプション) の属性を持つ. プロンプト注入時には, リソースインデックスは専用セクションとして注入され, LLM が保存したキーは Core Memory セクションに含められる.

コア記憶の総文字数が上限 (デフォルト 2,000 文字) を超えた場合, 重要度の低いエントリから順に削除される.

リコール記憶 (Recall Memory)

リコール記憶は、時間的に近い出来事を保持する短期記憶である。各イベントはタイムスタンプ、イベント種別、内容から構成される。

記録されるイベントは4つのカテゴリに大別される。ノードレベルイベント（ノード作成・実行結果）は全実行モードで記録される。Split-Phase 実行時には、各フェーズの完了・失敗が個別に記録され、どのフェーズで問題が発生したかの診断を可能にする。メトリクス抽出、プロット生成、サマリー生成などのポスト処理イベントも記録される。さらに、ステージレベルでは、サブステージの完了状態、目標生成、ステージ間の移行がイベントとして管理される。

リコール記憶はFIFO（First-In-First-Out）方式で管理され、最大イベント数（デフォルト5件）を超えると古いイベントが削除される。削除前に重要なイベントはアーカイブ記憶に移行される。

アーカイブ記憶 (Archival Memory)

アーカイブ記憶は、長期的な知見を検索可能な形で保存する。SQLite の FTS5 (Full-Text Search 5) を使用した全文検索をサポートし、Retrieval-Augmented Generation (RAG) [9] の枠組みに基づいて LLM の生成に関連知見を統合する。

アーカイブ記憶には、Phase 0 の詳細計画、研究アイデアの全文、実験結果とコードスナップショット、リソース詳細情報、ステージ・サブステージのサマリー、および LLM が記録した知見が保存される。各レコードにはタグが付与され、意味的な分類と効率的な検索を可能にする。

タグによる分類：各レコードにはタグを付与でき、検索時にフィルタリングに使用される：

基本タグ：

- PHASE0_INTERNAL：Phase 0 の詳細計画
- IDEA_MD：研究アイデアの全文
- ROOT_IDEA：オリジナルアイデア（不変）

リソース関連タグ：

- resource:<class>:<name>：リソース項目の詳細
- resource_path:<path>：パスによるリソース参照

実行結果タグ：

- code:<node_id>：コードスナップショット
- results:<node_id>：実行結果
- SUCCESS, ERROR：成功/失敗した試行（stage:X 付き）
- METRICS：抽出されたメトリクス

進行管理タグ：

- NODE_SUMMARY：ノードサマリー
- SUMMARY, FINDINGS：ステージ進行サマリー

- STAGE_COMPLETE：サブステージ完了
- GOAL：サブステージ目標
- DATASETS_TESTED, ABLATIONS：特定ステージ完了
- STAGE_PROGRESSION：ステージ移行情報
- EVICTED_RECALL：リコール記憶から退避されたイベント

3.3.3 記憶管理クラス

記憶操作は MemoryManager クラスを通じて行われる。このクラスは SQLite データベースを永続化層として使用し、3 層記憶への API を提供する。

記憶操作インタフェース

記憶管理クラスは、各記憶層に対する CRUD 操作とブランチ管理のための API を提供する。コア記憶に対しては、キーの取得・設定・削除が可能であり、設定時に重要度と有効期限を付与できる。リコール記憶に対しては、イベントの追加と検索が可能である。アーカイブ記憶に対しては、レコードの書き込み・更新・全文検索・ID 指定取得をサポートする。ブランチ操作として、子ブランチの作成、ブランチ単位での記憶読み取り、およびバッチ書き込みが提供される。

永続化

記憶は SQLite データベースに永続化される。ブランチ階層、コア記憶のキー・バリューストアとメタ情報、リコール記憶のイベントタイムライン、アーカイブ記憶のレコード、FTS5 全文検索インデックス、および Copy-on-Write 統合のための除外リスト・統合サマリーが管理される。並列実行時には、複数ワーカーからの書き込み競合を回避するため、専用のデータベースライタープロセスがすべての書き込み操作を直列化する。

3.3.4 LLM による記憶操作

LLM は応答中に<memory_update>ブロックを含めることで、記憶を直接操作できる。これにより、エージェントが実験中に得た知見を自律的に記録し、過去の経験を検索できる。

Two-Phase パターン

本システムでは、LLM 関数呼び出し時に「Two-Phase パターン」を採用している。このパターンが必要な理由は、LLM に記憶操作（自由形式のテキスト処理）と構造化出力の生成（厳密な JSON 形式）を同時に要求すると、出力の構造が崩れやすい点にある。記憶の更新と構造化出力の生成を分離することで、記憶操作の柔軟性と出力の正確性を両立させる設計である。

Phase 1：記憶更新フェーズ

LLM に自由形式（free text）での応答を許可し、<memory_update>ブロックによる記憶操作を実行する。この段階では出力形式の制約を課さず、LLM が必要に応じて記憶の読み書きを行う。

Phase 2：構造化出力フェーズ

記憶更新が完了した後、`func_spec`（関数仕様）に基づく構造化出力を要求する。記憶から取得した情報を考慮した上で、所定のフォーマットで結果を返す。

Algorithm 3 Two-Phase パターンによる記憶更新（`func_spec` 使用メソッド共通）

入力: タスクプロンプト P , 関数仕様 $spec$ (`func_spec`), 記憶マネージャ $\mathcal{M}$, ブランチ ID b , フェーズ名 ϕ , 最大ラウンド N_r

出力: 構造化 JSON 出力 O

```

1:
2:                                     ▷ Phase 1: 記憶更新（自由テキスト応答, _run_memory_update_phase()）
3:  $mem\_ctx \leftarrow \text{RENDERFORPROMPT}(\mathcal{M}, b, \phi, \text{budget\_chars})$ 
4:  $P_1 \leftarrow P + mem\_ctx + \text{memory\_ops\_instructions}$ 
5: for  $round = 0$  to  $N_r - 1$  do
6:    $R \leftarrow \text{LLM}(P_1, \text{task\_description})$                                      ▷ 自由テキスト応答
7:    $ops \leftarrow \text{EXTRACTMEMORYUPDATES}(R)$                                      ▷ <memory_update>解析
8:    $results \leftarrow \text{APPLYLLMMEMORYUPDATES}(\mathcal{M}, b, ops)$ 
9:   if  $\neg \text{HASMEMORYREADRESULTS}(results)$  then
10:    break                                     ▷ 読み取り操作なし → Phase 2 へ
11:   end if
12:    $P_1 \leftarrow P_1 + \text{FORMATMEMORYRESULTS}(results)$                          ▷ 読み取り結果を注入
13: end for
14:
15:                                     ▷ Phase 2: 構造化出力（func_spec による JSON 応答）
16:  $mem\_ctx' \leftarrow \text{MEMORYCONTEXT}(\mathcal{M}, b, \phi, \text{budget\_chars})$              ▷ 更新後の記憶
17:  $P_2 \leftarrow P + mem\_ctx' + spec$ 
18:  $O \leftarrow \text{LLM}(P_2, \text{func\_spec} = spec)$                                      ▷ 構造化 JSON 応答
19: return  $O$ 

```

この Two-Phase パターンは、記憶コンテキストの注入、LLM クエリと記憶操作の実行、マルチターン記憶読み取りのサポート、および出力からの記憶操作ブロック除去を一貫して処理するヘルパー関数として実装されている。記憶操作は確定的な順序（コア記憶の更新 → アーカイブへの書き込み → リコールイベントの追加 → リコールイベントの退避 → リコールイベントの要約 → 記憶の統合）で処理される。書き込みを読み取りより先に実行することで、LLM が同一ターン内で保存した情報を後続の操作で参照可能にする。

操作形式

LLM は応答中に JSON 形式の記憶操作ブロックを含めることで、コア記憶の読み書き・削除、アーカイブ記憶の書き込み・検索・更新、リコール記憶のイベント追加・検索・退避・要約、および記憶全体の統合を指示できる。

サポートされる操作

表 3.4 に、LLM が使用可能な記憶操作を示す。

表 3.4 LLM が使用可能な記憶操作

カテゴリ	操作キー	説明
コア記憶	mem_core_set	キー・バリューを設定/更新
	mem_core_get	指定キーの値を取得
	mem_core_del	キーを削除
アーカイブ	mem_archival_write	新規レコードを書き込み
	mem_archival_update	既存レコードを更新
	mem_archival_search	全文検索
リコール	mem_recall_append	イベントを追加
	mem_recall_search	イベントを検索
	mem_recall_evict	イベントをアーカイブに移動後削除
	recall_summarize	イベントを統合・要約
管理	consolidate	記憶の統合・圧縮を実行

マルチターン検索フロー

読み取り操作（core_get, archival_search, recall_search）を使用した場合、システムはマルチターンフローを実行する：

1. LLM が読み取り操作を含む<memory_update>を出力
2. システムが操作を実行し、書き込みを永続化、読み取り結果を収集
3. 読み取り結果を<memory_results>としてプロンプトに注入
4. LLM を再クエリし、検索結果を考慮した最終応答を生成
5. 最大ラウンド数（デフォルト 5）に達するまで繰り返し

このマルチターンフローは、Phase 0 および Phase 1 の各イテレーションを含む全フェーズで利用可能である。特に Phase 1 では、過去のインストール経験を検索し、同様の環境での依存解決に活用する典型的なユースケースがある。

フェーズ別記憶イベント

Split-Phase 実行モデルの各フェーズは、完了時にリコール記憶へイベントを記録する。これにより、LLM は後続のフェーズで過去の実行状況を参照できる。

表 3.5 に、各フェーズで記録されるイベント種別を示す。

Algorithm 4 マルチターン記憶操作フロー（`_run_memory_update_phase` の内部ループ）

入力: プロンプト P , 記憶マネージャ $\mathcal{M}$, ブランチ ID b , フェーズ名 ϕ , 最大ラウンド N (`max_memory_read_rounds`), タスク記述 $desc$

出力: 全書き込み操作が適用済み

```
1: for round = 0 to N - 1 do
2:    $R \leftarrow \text{LLM}(P, desc)$  ▷ 自由テキスト応答
3:    $ops \leftarrow \text{EXTRACTMEMORYUPDATES}(R)$  ▷ <memory_update>ブロック解析
4:   ▷ 書き込み操作を即時適用, 読み取り操作の結果を収集
5:    $results \leftarrow \text{APPLYLLMMEMORYUPDATES}(\mathcal{M}, b, ops)$ 
6:   ▷ 書き込み: mem_core_set, mem_archival_write, mem_recall_append 等
7:   ▷ 読み取り: mem_core_get, mem_archival_search, mem_recall_search
8:   if HASMEMORYREADRESULTS( $results$ ) then
9:     ▷ 読み取り結果を「Memory Read Results」セクションとして注入
10:     $P \leftarrow P + \text{FORMATMEMORYRESULTSFORLLM}(results)$ 
11:   else
12:     break ▷ 読み取り操作なし→ループ終了
13:   end if
14: end for
```

表 3.5 フェーズ別記憶イベント

フェーズ	成功時	失敗時
Phase 0 (計画)	phase0_complete	phase0_failed
Phase 1 (環境構築)	phase1_complete	phase1_failed
Phase 2/3 (コード/コンパイル)	compile_complete	compile_failed
Phase 4 (実行)	run_complete	run_failed

各イベントにはタイムスタンプ, ノード ID, および結果サマリー（成功時は要約, 失敗時はエラー情報）が含まれる。

特に Phase 1 は反復的な LLM 呼び出しを行う唯一のフェーズであり, 各イテレーションで LLM が自律的に記憶操作を実行する。この機構により, インストール成功時のバージョンやコミット SHA のコア記憶への保存, 失敗したアプローチと回避策のアーカイブへの記録, 類似のエラーに対する過去の解決策の検索, および複数の実行を通じた依存解決のベストプラクティスの蓄積が可能となる。

ポスト実行記憶操作

Phase 4 (実行) の完了後, 追加の LLM 呼び出しにより記憶の更新と分析が行われる。各処理における記憶操作の適用状況を表 3.6 に示す。

表 3.6 ポスト実行処理における記憶操作

処理	Two-Phase	記憶注入	明示的書き込み
execution_review	○	○	LLM に委譲
metrics_extraction	○	○	リコール + アーカイブ (METRICS)
plotting_code	-	○	memory_update 抽出時のみ
plot_selection	○	○	LLM に委譲
vlm_analysis	○ (3 段階)	○	アーカイブ (VLM_ANALYSIS) + コア
node_summary	○	○	リコール + アーカイブ (NODE_SUMMARY)

ポスト実行処理は 6 つのステップから構成される (図 3.4)。

- (0) **実行レビュー** (失敗時のみ) : エラーメッセージと終了コードを分析し, リトライの可否を判定する. Two-Phase パターンにより, LLM が記憶操作を自律的に決定する.
- (1) **メトリクス抽出** : stdout/stderr からベンチマーク結果や性能指標を構造化された形式で抽出し, リコール記憶 (metrics_extracted/metrics_failed) およびアーカイブ記憶 (METRICS タグ) に記録する.
- (2) **プロット生成** : plan_and_code_query() により可視化 Python コードを生成・実行する. LLM 応答に<memory_update>が含まれる場合は記憶更新を抽出・適用する.
- (2.5) **プロット選択** : 複数のプロットから論文に掲載するものを Two-Phase パターンで選択する.
- (3) **VLM 分析** : 3 段階の Two-Phase パターン (記憶更新→VLM 画像分析→分析後記憶書き込み) により, Vision Language Model がプロット画像の品質と妥当性を評価する. 分析結果はアーカイブ記憶 (VLM_ANALYSIS タグ) およびコア記憶 (last_vlm_analysis) に記録される.
- (4) **ノードサマリー** : コード, 実行結果, メトリクス, VLM 分析を統合して実行全体を要約し, リコール記憶 (node_summary) およびアーカイブ記憶 (NODE_SUMMARY タグ) に記録する.

各ステップにおける記憶操作では, 破線矢印で示される記憶層への書き込みが行われる. 操作には mem_archival_write, mem_recall_append, mem_core_set 等が用いられ, 適切なタグが付与される. execution_review 以外の各ステップは Two-Phase パターン (Phase 1 で記憶更新, Phase 2 で構造化出力) を採用し, LLM が<memory_update>ブロックで記憶操作を自律的に決定する. vlm_analysis は 3 段階の Two-Phase パターンを使用する.

Post-Execution Memory Operations Flow

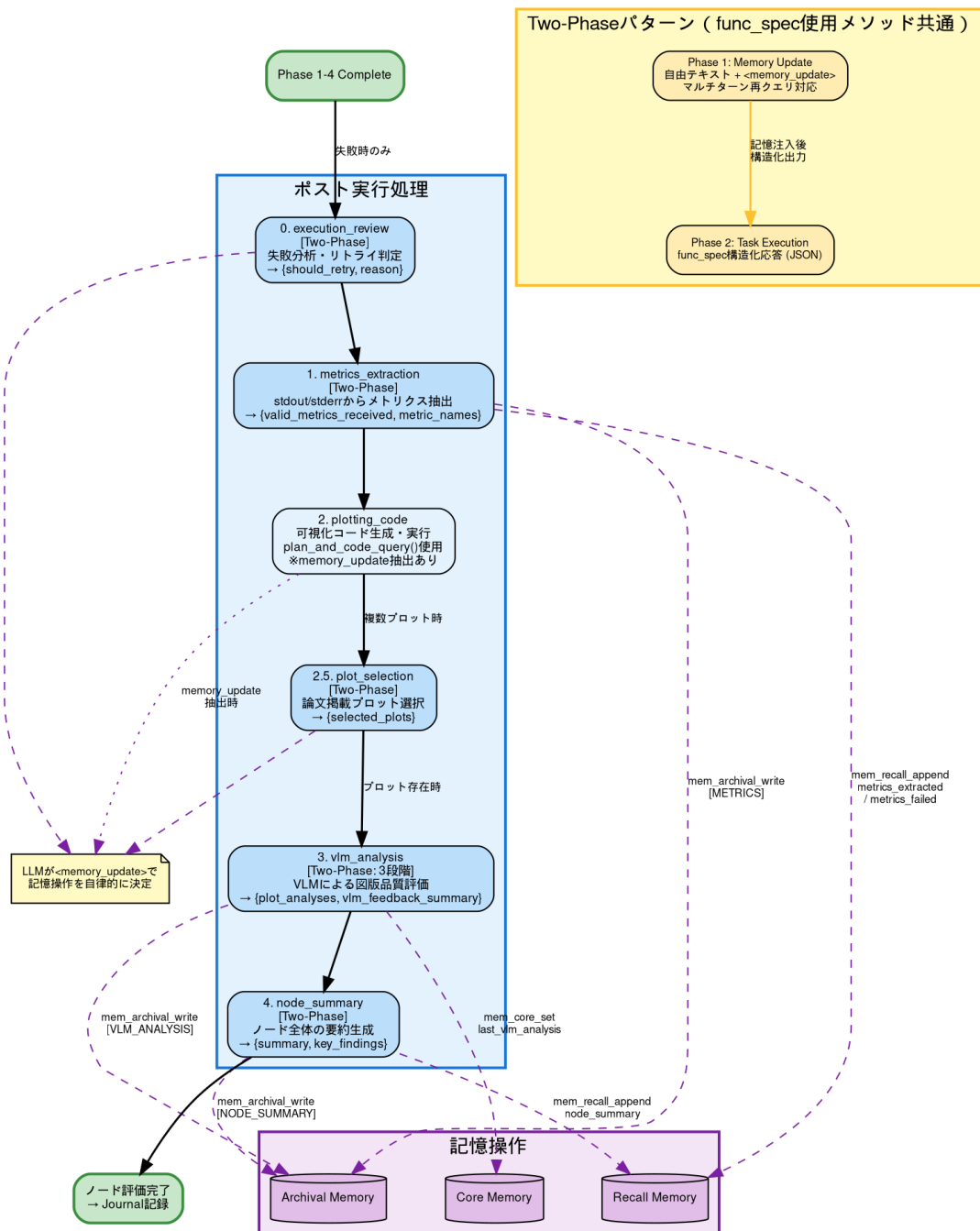


図 3.4 ポスト実行記憶操作のフロー（6 段階処理と記憶書き込み）

3.3.5 文脈注入

LLM 呼び出し時には、記憶の内容がプロンプトに注入される。注入は `_inject_memory()` 関数により行われる。

注入される内容

注入される記憶コンテキストは以下の構造を持つ：

1. **コア記憶**：全エントリ（常に注入）
2. **リコール記憶**：最新 k イベント（デフォルト $k = 5$ ）
3. **アーカイブ検索結果**：タスクヒントに基づく上位 k 件（デフォルト $k = 4$ ）
4. **記憶操作指示**：LLM が記憶を操作するための説明

タスクヒント

アーカイブ検索の関連性を向上させるため、各フェーズで異なるタスクヒントが使用される。Phase 0 では計画関連の情報が、Phase 1 では環境構築関連の知見が、Phase 2 ではタスク種別（ドラフト、デバッグ、改良）に応じた情報が優先的に検索される。ポスト処理においても、実行結果レビュー、メトリクス抽出、ノードサマリー生成それぞれに適したヒントが設定される。

予算管理

文脈長の制約を守るため、各記憶層およびセクションに文字数予算を割り当てる：

基本予算として、総予算 24,000 文字、コア記憶上限 2,000 文字、リコール最大イベント数 5 件、アーカイブ検索取得件数 4 件、読み取り再クエリ最大回数 5 回が設定されている。

各処理段階（メトリクス抽出、プロット生成、VLM 分析、ノードサマリー等）に対して個別の文字数予算が設定されており、予算を超過する場合には LLM による知的圧縮が実行される。圧縮にはキャッシュ機構があり、同一テキストの再圧縮を回避する。圧縮時には、研究目標や Phase 0 の計画要約などの記憶コンテキストが考慮され、研究にとって重要な情報が優先的に保持される。

3.3.6 ブランチ分離

木探索において、各ノードは独立したブランチとして記憶を持つ。これにより、探索パス間での記憶の汚染を防ぐ。

継承モデル

ブランチのフォーク (`mem_node_fork`) 時に、子ブランチは親の `ancestor_chain`（祖先ブランチ ID のリスト）を継承する。この設計には以下の理由がある：

コア記憶については、子ノードは祖先の全エントリを可視的に参照でき、同一キーが複数の祖先に存在する場合は最も近いブランチの値が優先される。リコール記憶については、祖先チェーン上の全イベントが時系列順で統合される。アーカイブ記憶については、祖先チェーン上の全レコードが FTS5 全文検索の対象となる。重要な設計上の制約として、書き込み隔離が適用される。すなわち、子ノードへの書き込みは自身のブランチにのみ永続化され、親や兄弟ノードの記憶には影響しない。この設計により、探索木の各パスは独立した「実験ノートブック」として記憶を蓄積しつつ、共通の祖先の知見を再利用できる。

可視性チェーン

記憶の検索時には、現在のブランチの `ancestor_chain` に基づいて祖先チェーンが計算される。`_branch_chain()` メソッドがこの計算を行い、自身から根までのすべてのブランチ ID を返す。SQL クエリではこのチェーンを WHERE 句に使用し、チェーン上のすべてのブランチの記憶を集約する。

Copy-on-Write (CoW) による継承メモリ統合

継承された記憶が蓄積されると、文脈長を圧迫する。この問題に対し、Copy-on-Write セマンティクスによる**継承メモリ統合**を導入した。

統合処理は以下のステップで行われる：

1. 祖先から継承されたリコールイベント群を LLM で要約し、統合サマリーを生成
2. 統合されたイベントの ID を `inherited_exclusions` テーブルに記録
3. 統合サマリーを `inherited_summaries` テーブルに保存
4. 以降の記憶注入時、除外されたイベントは個別には表示されず、サマリーが代わりに注入される

この機構の重要な性質は、**元のイベントは削除されない** (CoW) 点である。除外リストとサマリーは子ブランチに固有であり、親や兄弟の記憶には影響しない。これにより、記憶の削減と探索パス間の独立性を両立する。

3.3.7 LLM 圧縮

実験出力やエラーログは数千文字に及ぶことがあり、単純な切り捨て (truncation) では、テキスト末尾に含まれる結論や数値データが欠落する。この問題に対処するため、LLM による**知的な圧縮** (intelligent compression) を導入した。LLM は文脈を理解した上で、数値データ、重要な判断、技術的詳細を優先的に保持しつつ、冗長な説明を要約する。

圧縮フロー

1. テキストが予算内か確認 (収まっていれば圧縮不要)
2. キャッシュを確認 (同一テキストの圧縮結果を再利用)
3. LLM に圧縮を依頼 (重要な情報を保持しつつ要約)
4. 圧縮結果が予算内か確認 (超過なら再圧縮または切り捨て)

圧縮プロンプト

LLM への圧縮指示では以下を保持するよう要求する：

- 数値データとメトリクス
- 重要な判断と結論
- 技術的な詳細と依存関係
- 研究目標に関連する情報 (コンテキスト考慮)

3.3.8 記憶プレッシャー管理

探索木が深くなるにつれ、子ノードは祖先の記憶を累積的に継承するため、記憶の総量は単調に増加する。記憶注入時に文脈長制約を超過すると、重要な情報が欠落するリスクがある。この問題に対処するため、記憶使用量が閾値を超えた場合に自動的に統合・圧縮を行うプレッシャー管理メカニズムを実装している。

プレッシャーレベル

記憶使用量に基づいて 4 段階の圧力レベルを定義する。各レベルの閾値は設定ファイルで調整可能である：

- **low** (70% 未満)：アクション不要
- **medium** (70–85%)：重要度を評価、必要に応じて LLM 圧縮を実行
- **high** (85–95%)：リコールイベントを統合、低重要度コアエントリをアーカイブに退避
- **critical** (95% 以上)：積極的な統合、ダイジェスト作成、古いイベントの削除

自動統合設定

プレッシャーレベルに応じて以下の処理が自動的に実行される：

- 関連するリコールイベントをグループ化し LLM で要約
- 低重要度のコアエントリをアーカイブに移動
- 重要情報のダイジェストを作成

自動統合の有効化、統合トリガーレベル、リコール統合閾値倍率、各圧力レベルの閾値は設定ファイルにて調整可能であり、実験の規模や文脈長制約に応じた柔軟な運用が可能である。

3.3.9 記憶フローの全体像

図 3.5 に、本システムにおける記憶フローの全体像を示す。この図は、ノードの生成から Phase 0–4 の実行、ポスト処理に至るまでの記憶操作の流れを時系列で表している。

記憶フローの主要な構成要素は以下の通りである：

- **Two-Phase パターンによる記憶更新**：各フェーズにおいて、<memory_update>ブロックの抽出と適用により、LLM が自律的に記憶を更新する。
- **マルチターン再クエリ**：複数回の LLM 呼び出しを通じて、段階的に記憶を読み取り、構造化された情報を構築する。
- **フェーズ完了イベントの自動記録**：各フェーズの完了時に、実行結果のサマリーがリコール記憶に自動的に記録される。
- **ブランチ分離による記憶管理**：親ノードの記憶を継承しつつ、各ブランチで独立した記憶空間を維持することで、探索の分岐を適切に管理する。

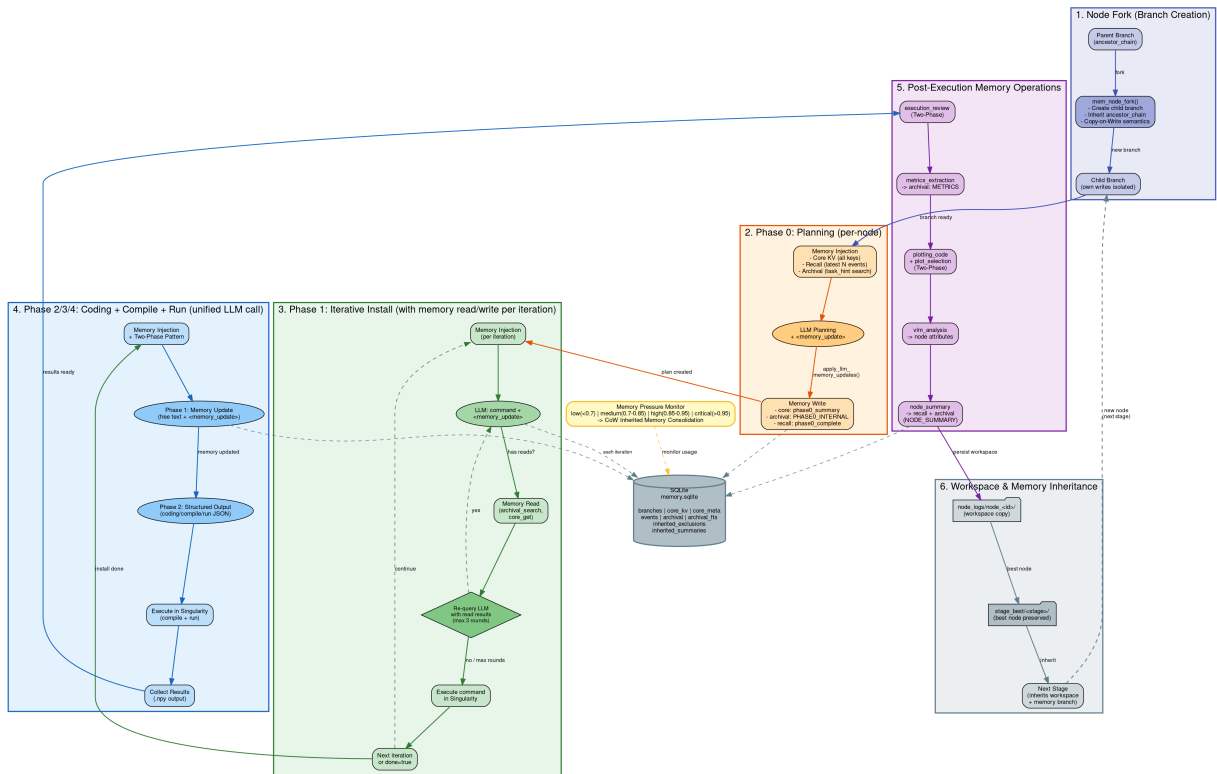


図 3.5 記憶フローの全体像

3.4 ポスト処理パイプラインの提案

Phase 4（実行）の完了後，実験結果の分析と論文生成のための一連のポスト処理パイプラインが実行される．本節では，提案システムにおいて新たに設計・実装した VLM 分析パイプライン，メトリクス抽出フロー，論文生成パイプラインについて詳述する．

3.4.1 VLM 分析パイプライン

Vision Language Model (VLM) を使用して，生成されたプロット画像を分析し，品質評価と改善提案を行う．

パイプラインフロー

1. **プロットコード生成**：LLM が .npz データから可視化コードを生成
2. **プロット実行**：生成された Python コードを実行し，PNG 画像を生成
3. **画像エンコード**：生成された画像を Base64 エンコード
4. **VLM 呼び出し**：エンコードされた画像と研究目標テキストを VLM に入力
5. **結果格納**：分析結果をノード属性に保存

VLM 入出力

VLM への入力は、研究アイデアテキスト、Base64 エンコードされたプロット画像、および分析プロンプトテンプレートから構成される。出力として、画像品質評価、データ可視化の適切性判定、改善提案、および妥当性判定が得られ、ノード属性として格納される。これらの分析結果は、後続のプロット選択および論文生成において参照される。

3.4.2 メトリクス抽出フロー

実行出力 (stdout/stderr) から性能指標を構造化された形式で抽出する。

抽出プロセス

1. **パーサ生成**: LLM が実行出力を解析するパーサコードを生成
2. **パーサ実行**: 生成されたパーサで出力を解析
3. **MetricValue 格納**: 抽出されたメトリクスを構造化データとして保存

メトリクスの構造

各メトリクスは、名称 (例: speedup, accuracy)、数値、および最適化方向 (大きいほど良い、または小さいほど良い) の属性を持つ構造化データとして保存される。この構造化により、木探索における自動的なノード評価と比較が可能となる。

3.4.3 論文生成パイプライン

実験結果から LaTeX 形式の論文を自動生成するパイプラインである。本パイプラインは、プロット集約、引用収集、LaTeX 生成、PDF 生成、レビューの 5 段階で構成される。

パイプラインフロー

1. **実験サマリー収集**: 各ノードから結果、メトリクス、プロットを収集する。
2. **プロット集約**: LLM が最良のプロットを選択し、集約スクリプト (auto_plot_aggregator.py) を生成する。リフレクションステップにより品質を向上させる。
3. **引用収集**: Semantic Scholar API を使用して関連論文を自動検索・収集する。
4. **LaTeX 生成**: Abstract, Introduction, Method, Experiments, Conclusion の構造で論文を生成する。リフレクションステップにより品質を向上させる。
5. **PDF 生成**: pdflatex でコンパイルし、ページ数制限のチェックを行う。
6. **レビュー** (オプション): LLM による論文レビューおよび図版品質の VLM レビューを実行する。レビューバイアスモードの選択が可能である (後述)。

論文生成パイプラインの成果物として、集約されたプロット画像、生成された論文 PDF、テキストレビュー、図版・キャプションレビューが出力される。

Writeup の文脈組立て

LaTeX 生成ステップにおいて、LLM に渡される文脈は以下の要素から組み立てられる：

- システムプロンプト (`prompt/output/writeup/system_message`)：論文執筆者としての役割定義と出力規約
- 研究アイデア (`idea.md`)：Abstract, Task goal, Experiments, Evaluation を含む研究計画
- 実験結果：最良ノードから収集された集約メトリクスおよびノードサマリー
- 図版：プロット集約ステップで選択されたプロット群
- メトリクス：各ノードから抽出された性能メトリクスの構造化データ
- 最終メモリ (`final_memory_for_paper.md`)：3 層記憶 (Core, Recall, Archival) から構成される論文執筆用のメモリコンテキスト (次項で詳述)
- 応答フォーマット：LaTeX セクション構造の指定

これらの文脈は `output/writeup.py` の `perform_writeup()` 関数により組み立てられ、LLM に送信される。LLM はこれを基に Abstract, Introduction, Related Work, Method, Experiments, Conclusion の各セクションを含む LaTeX 文書を生成する。

リフレクションによる品質向上

LaTeX 生成およびプロット集約の各ステップでは、Reflexion[7] に着想を得たリフレクション (反復的改善) 機構が適用される。LLM が生成した初期出力に対して、自己評価と改善を複数回 (デフォルト 3 ラウンド) 繰り返すことで、論文の品質を段階的に向上させる。各ラウンドでは、前回の出力全体がフィードバックとして入力され、論理構成の改善、欠落情報の補完、記述の明確化が行われる。

引用収集機構

引用収集ステップ (`output/citation.py`) では、Semantic Scholar API[6] を使用して関連論文を自動検索する。LLM が生成した LaTeX 中の引用キーを解析し、各キーに対して Semantic Scholar の検索エンドポイントを呼び出して書誌情報を取得する。取得された BibTeX 形式の引用情報は `references.bib` に集約され、pdflatex コンパイル時に参照される。

最終メモリの統合

論文生成において、階層記憶機構から生成される `final_memory_for_paper.md` が重要な役割を果たす。このファイルは、木探索完了後に `generate_final_memory_for_paper()` により生成され、以下の 2 部構成を持つ：

- Part 1：ノードデータ (固定セクション) 最良ノードの実装詳細 (コード, 計画, Phase 0–4 の出力), VLM 分析結果, 上位 5 ノードの比較情報から構成される。
- Part 2：LLM 生成セクション (動的セクション) 3 層記憶 (Core Memory, Recall Memory, Archival Memory) の内容を LLM が解釈し、論文セクション (Title Candidates, Abstract Material, Problem Statement, Hypothesis, Method, Experimental Setup, Results 等) を動的に生成する。

生成モードとして `memory_summary`（デフォルト：LLM が記憶内容に基づき自律的にセクション構造を決定）と `idea_then_memory`（研究アイデアからセクション骨格を生成し、記憶で補完）の 2 種が提供される。

ハードウェア情報の自動注入

最終メモリ生成時に、Phase 0 で収集された環境情報（`PHASE0_INTERNAL` タグ付き Archival エントリ）からハードウェア情報を自動抽出し、Core Memory に注入する機構を備える。正規表現パターンにより CPU モデル、ソケット数、コア数、NUMA ノード数、OS、コンパイラバージョン、利用ツールが抽出され、`hardware_cpu`, `hardware_os`, `hardware_compiler`, `hardware_tools` のキーで Core Memory に格納される。これにより、論文の Experimental Setup セクションに環境情報が正確に記述される。

さらに、優先タグシステム（Priority Tag System）により、`PHASE0_INTERNAL`, `IDEA_MD`, `ROOT_IDEA` のタグを持つ Archival エントリは、作成時刻に関わらず LLM に提示される上位 30 件に優先的に含まれる。これは、実験の進行に伴い新しい Archival エントリが蓄積されることで、初期の重要情報が LLM の可視範囲外に押し出される問題を解決するものである。

レビューバイアス設定

自動論文レビューにおいて、レビュアー LLM が不確実性をどのように扱うかを制御するため、3 つのバイアスモードを提供する。

- **neutral**（デフォルト）：バランスの取れたレビュー。不確実性に対する明示的なバイアスを付与しない。ベースプロンプト（Listing D.1）のみを使用する。
- **neg**（厳格モード）：不確実な場合は低評価・リジェクトとする厳格な評価。ベースプロンプトに厳格モード接尾辞（Listing D.2）を付加する。投稿前の品質確認に適している。
- **pos**（寛容モード）：不確実な場合は高評価・アクセプトとする寛容な評価。ベースプロンプトに寛容モード接尾辞（Listing D.3）を付加する。初期段階のアイデア探索に適している。

バイアスモードは `--review-bias`（または `--review_bias`）フラグにより指定する。システムプロンプトの構成として、`neutral` モードではベースプロンプトのみが使用され、`neg/pos` モードではそれぞれの接尾辞が付加される。これにより、同一の論文に対して異なる観点からの評価を取得できる。

3.4.4 最終メモリ生成

論文執筆のため、実験中に蓄積された記憶を構造化して出力する機能を提供する。

生成フロー

木探索完了後、最良ノードとその記憶を包括的に収集し、論文執筆に必要な情報を構造化する。この機能が必要な理由は、探索過程で蓄積された知見（成功要因、失敗パターン、比較実験結果）が複数のノードとブランチに分散しており、論文執筆時にこれらを手動で収集することが困難であるためである。

1. **最良ノードの特定**：探索木から最高メトリクスのノードを選択

2. **上位ノード収集**：上位 5 ノードの情報を比較分析用に収集 (ID, 計画, メトリクス, 分析結果, VLM フィードバック)
3. **3 層記憶の抽出**：最良ブランチのコア記憶・リコール記憶・アーカイブ記憶を取得
4. **実験成果物収集**：最良ノードのコード, Phase 0-4 の成果物, 実行結果, VLM 分析, プロットパス, 実行時間フィードバックを統合
5. **リソース使用状況収集**：使用されたデータセットとファイルの情報を収集
6. **負の結果と教訓**：失敗したノードのエラー情報と回避策を記録
7. **出所チェーン (Provenance Chain)**：ルートから最良ノードまでのブランチ系譜を記録
8. **出力**：Markdown 形式(`final_memory_for_paper.md`), JSON 形式(`final_memory_for_paper.json`), 完全論文執筆ペイロード (`final_writeup_memory.json`) の 3 形式で保存

出力形式

最終メモリは、人間可読な Markdown 形式（論文執筆の参考用）と機械可読な JSON 形式（LLM 処理用）の 2 形式で出力される。JSON 形式には、研究アイデアの全文と要約, Phase 0 計画の要約, 実験結果のサマリー, 3 層記憶の内容（コア記憶のキー・バリュー, 最近のリコールイベント, 検索されたアーカイブ記録）, リソース使用状況, および LLM 生成のセクション草稿が構造化されて含まれる。

3.4.5 可視化ツール

実験の進行状況と結果の検証を支援するため、本システムはインタラクティブな HTML 形式の可視化ツールを自動生成する。主要なツールとして、探索木の構造を全ステージ統合で表示するツリービジュアライザー (`unified_tree_viz.html`) と、メモリデータベースの内容を詳細に閲覧するメモリビューアー (`memory_database.html`) がある。

ツリービジュアライザー

ツリービジュアライザーは、p5.js を用いたインタラクティブなキャンバス（画面左側 40%）と、ノード詳細パネル（画面右側 60%）から構成される。画面上部には 4 つのステージタブ (Stage 1-4) が配置され、各ステージの探索木をタブ切り替えで閲覧できる。初期化時に各ステージの `tree_data.json` が非同期ロードされ、完了していないステージのタブはグレイアウトされる。

キャンバス上のノードをクリックすると、`setNodeInfo()` 関数が呼ばれ、`tree_data.json` から以下の情報が詳細パネルの各セクションに描画される：

- **Plan**：Phase 0 で生成された実行計画テキスト (`highlight.js` によるシンタックスハイライト)
- **Exception Info**：エラー発生時の Exception Type, Details (JSON), Stack Trace
- **Execution Time**：実行時間 (秒) およびフィードバック
- **Metrics**：各メトリクスの名称・説明・最適化方向・データセット別の値テーブル
- **Memory Operations**：メモリ操作をフェーズごとにグループ化して表示する専用パネル。◀ Prev / Next ▶ ボタンでフェーズ間を循環ナビゲーションし、8 種のフィルタボタン (All / Reads / Writes / Deletes / Forks / Recalls / Resources / Maintenance) で操作タイプを絞り定める。各

操作イベントはカテゴリ別バッジ（色付きアイコン）、操作名、メモリタイプ、キー情報、タイムスタンプ、詳細 JSON を含むカード形式で表示される

- **Plots**：生成されたプロット画像の一覧表示
- **VLM Feedback**：VLM による各プロットの分析結果 (`plot_analyses`)、総合フィードバック (`vlm_feedback_summary`)、テスト成功データセット一覧
- **Code / Plot Code**：実験コードおよびプロット生成コード (Python シンタックスハイライト付き)

メモリデータベースビューアー

メモリビューアーは、図 3.6 に示すように、リサイズ可能な左右パネル構成を採用する。左パネルのツリーキャンバスでノードを選択すると、右パネルに 6 つのタブで切り替え可能なメモリ情報が表示される。

- **Summary タブ**：継承チェーン（祖先ノード一覧、クリックで移動可能）、自ノードの操作サマリー（Reads / Writes / Forks / System / LLM の 5 カテゴリ）、祖先から継承された操作サマリー、フェーズ別操作件数（クリックで By Phase タブへ遷移）、メモリコンテンツ件数（Core KV / Events / Archival, own/inherited 分離）
- **Effective Memory タブ**：LLM が実際に参照するメモリ状態を表示する最も重要なビュー。Core KV（同一キーは子ノード優先）、Recall Events（累積、CoW 除外適用）、Archival Records（累積）を統合表示し、各エントリに [own]（青色ボーダー）/[inherited]（グレー）ラベルで出所を視覚的に区別する
- **Memory Flow タブ**：`render_for_prompt` 呼び出しを境界として操作を「ラウンド」単位でグループ化。各ラウンドでは Memory Injection（注入された Core/Recall/Archival 項目数、文字数バジェット、Show Injected Context ボタンで展開可能）、LLM Read Operations（llm_プレフィックスの検索操作、再クエリトリガーの可能性あり）、Other Operations（書き込み等）の 3 セクションで表示
- **By Phase タブ**：Phase 0-4, Summary 等のフェーズごとに折りたたみ可能なセクションで操作をグループ表示
- **Timeline タブ**：全操作をタイムスタンプ順にソートして一覧に時系列表示
- **All Data タブ**：own と inherited データを完全分離して表示。自ノードの Memory Operations / Core KV / Recall Events / Archival Records と、祖先から継承されたデータを折りたたみ可能なセクションで提供

各メモリ操作カードには操作タイプに応じたクイックサマリーが表示される。例えば、`render_for_prompt` ではバジェット・Core/Recall/Archival 項目数、`mem_core_set` ではキー名・値レビュー・文字数・重要度、`mem_node_fork` では親ノード ID・子ブランチ ID、`check_memory_pressure` では圧力レベル・使用率が表示される。すべての操作には Show Full Details ボタンがあり、展開すると Value/Text Content, Tags, 詳細 JSON, メタデータの全情報を確認できる。

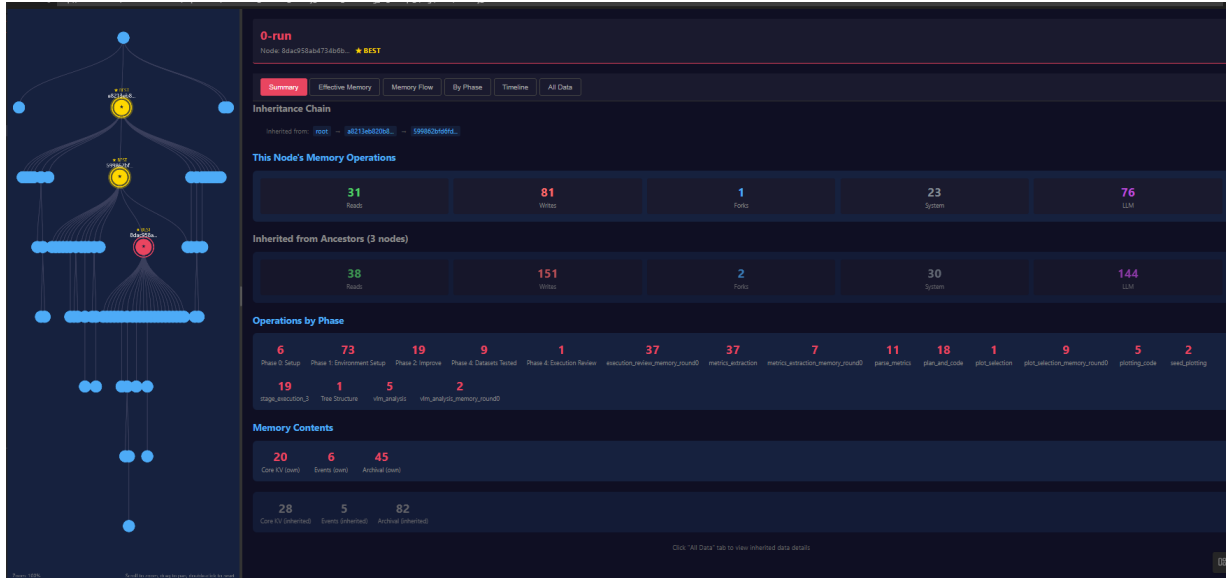


図 3.6 メモリデータベースビューアーの画面例

3.5 実装と検証

本章では、提案システム HPC-AutoResearch を構成する主要コンポーネントの実装上の要点と、設計 (docs) と実装の対応関係を検証する枠組みを述べる。

3.5.1 実装の全体像

HPC-AutoResearch は、(i) 木探索 (ステージ管理とノード実行)、(ii) Split-Phase 実行 (Phase 0–4)、(iii) MemGPT 型記憶 (Core/Recall/Archival, 分岐継承)、(iv) 資源注入 (Resource System)、(v) 後処理 (メトリクス抽出・可視化・VLM 分析・論文生成) から構成される。これらは相互に独立した責務を持ちつつ、探索ノードの実行という単位で統合される。

3.5.2 木探索とステージ進行

探索は 4 ステージ (initial_implementation, baseline_tuning, creative_research, ablation_studies) からなり、各ステージで生成されるノード (Draft/Debug/Improve/Hyperparam/Ablation) を Best-First 方針で選択・展開する。重要なのは、HPC では 1 回の試行コストが高いため、評価関数に基づいて試行の優先度を制御し、限られた計算資源を有望な枝へ配分する点である。

3.5.3 Split-Phase 実行 (Phase 0–4)

各ノードは、Phase 0 で環境情報と履歴から実行計画を生成し、Phase 1 でコンテナ内の依存解決を反復的に実行し、Phase 2 でコード生成、Phase 3 でビルド、Phase 4 で実行と結果収集を行う。この分割により、エラーを「どのフェーズで何が失敗したか」として局所化でき、デバッグノード生成 (Debug) と組み

合わせることで回復手順を体系化できる。

3.5.4 MemGPT 型階層記憶と分岐継承

記憶は、常時注入される Core、短期履歴の Recall、検索可能な長期保存の Archival からなる。探索木における子ノードは祖先の記憶を参照できる一方、書き込みはブランチに隔離され、兄弟ノードに汚染しない。この設計は「探索の多様性」と「学習の再利用」を両立させる。また、メモリ圧迫時には統合・圧縮を自動実行し、文脈長制約に適応する。

3.5.5 資源注入 (Resource System)

HPC ではデータセットや外部ライブラリが大きく、かつノードごとに可用性が異なる。Resource System は、ローカル/GitHub/Hugging Face 等の資源を宣言的に定義し、フェーズごとの注入規則に従い、(a) LLM プロンプトへ要約として注入し、(b) コンテナヘマウント・フェッチとして注入する。これにより、依存・データを「コードに埋め込む」のではなく「実験設定として管理」できる。

3.5.6 後処理：メトリクス・可視化・VLM・論文生成

実行後は、標準化された流れとしてメトリクス抽出、プロット生成、VLM による図版分析、ノードサマリー生成を行う。さらに、プロット集約と引用収集を含む論文生成パイプラインにより、探索の成果を論文形式へ整形できる。この一連の処理をノード実行と同一のログ・成果物体系に含めることで、再現性と監査可能性を高める。

3.5.7 設計と実装の整合性検証

本プロジェクトは docs/ に設計意図と運用手順が整理されている。そこで、ドキュメント記載の仕様（ステージ定義、フェーズ遷移、記憶操作 API、資源注入規則、出力パイプライン）と、実装における対応箇所を突合する検証を行うことで、学術的主張（設計原理）と実装（動作）の同型性を担保する。

- 木探索：ステージ名・ノード種別・選択戦略
- Split-Phase：Phase 0-4 の入出力と失敗時ハンドリング
- 記憶：Core/Recall/Archival、読み書き操作、分岐継承
- 資源：型 (local/github/huggingface)、class 別注入規則
- 後処理：メトリクス抽出、プロット、VLM 分析、論文生成

第 4 章

性能評価

4.1 実験概要

本章では、提案システムの有効性を検証するため、既存の LLM コーディングエージェント（Claude Code, Codex, Gemini CLI）との比較実験を行う。実験題材として、付録 B に掲載する Himeno Benchmark 性能解析タスク（`thesis_thema.md`）を使用する。Himeno Benchmark[10] は、流体力学の有限差分法を用いた反復解法をベースとする代表的な HPC 性能評価ベンチマークであり、メモリアクセス特性やキャッシュ性能の評価に広く利用されている [11, 12]。このタスクは、HPC システム上での Himeno ベンチマークの性能変動要因を分析し、最適化手法を実装・評価するものであり、環境構築・コンパイル・実行・メトリクス解析といった Split-Phase の全フェーズを活用する実験である。

実験フローは以下の通りである。各エージェントに同一のタスク記述（`thesis_thema.md`）を与え、同一の HPC 環境上で Himeno Benchmark タスクを実行させる。各エージェントが出力したコード・実験結果を、提案システムのポスト処理パイプライン（論文生成＋LLM レビュー）に通し、LLM レビュースコアによって各エージェントの研究遂行能力を比較評価する。

4.2 実験条件

以下の 4 条件を設定し、全実験は 2026 年 2 月 6 日に実施した。

提案手法（HPC-AutoResearch） Split-Phase 実行モデル、階層記憶機構、Best-First 木探索、ポスト処理パイプラインの全機能を有効化した状態で実行する。LLM には `gpt-5.2`（スナップショット `gpt-5.2-2025-12-11`）を全コンポーネント共通で使用した（付録 A 参照）。設定ファイルでは推論努力（reasoning effort）の明示的な指定は行っていない。

Claude Code Anthropic 社の Claude Code エージェント（`claude-opus-4-6`^{*1}、推論努力度: high）が直接 HPC 環境上でタスクを遂行する。実験出力のみを提案手法のポスト処理パイプラインに入力して評価した。

Codex OpenAI 社の Codex エージェント（`gpt-5.2-codex`^{*2}、推論努力度: high）が同様にタスクを遂

^{*1} 設定上は `opus` エイリアスを指定。2026 年 2 月 5 日リリースの Opus 4.6 に自動解決される。

^{*2} Codex CLI のデフォルト値。GPT-5.3-Codex が 2026 年 2 月 5 日にリリースされていたが、CLI のデフォルトは未更新

行する。実験出力のみを提案手法のポスト処理パイプラインに入力して評価した。

Gemini CLI Google 社の Gemini CLI エージェント (gemini-3-pro-preview^{*3}, 思考レベル: HIGH) が同様にタスクを遂行する。実験出力のみを提案手法のポスト処理パイプラインに入力して評価した。

全条件とも、同一のアイデア記述 (thesis_thema.md) を入力として与え、同一のハードウェア・コンテナ環境で実行する。提案手法は提案システムの設定ファイル (付録 A) に従い実行する。Claude Code, Codex, Gemini CLI の各エージェントには、共通のタスク記述ファイル (付録 E, Listing E.1) とエージェント基礎設定ファイル (Listing E.2) を与え、各エージェントの標準的な動作でコード生成・実験実行を行わせる。各エージェントの出力 (コード・実験結果) は、提案システムのポスト処理パイプライン (論文生成 + LLM レビュー) に通して評価する。レビューバイアスモードは **pos** (寛容モード) を使用する (`--review-bias pos`)。これは、初期段階のシステム検証において、生成論文の潜在的な価値を過小評価することを避け、システムの研究遂行能力をより広く評価するためである。

4.3 実験環境

全条件において同一の HPC 環境 (Dual-socket AMD EPYC 9554, 128 コア, 2 NUMA ノード, SMT 無効, L2 キャッシュ 128 MiB, L3 キャッシュ 512 MiB, メモリ約 1.5 TiB) 上で実験を実施した。ホスト OS は Rocky Linux 9.7 (カーネル 5.14.0) であり、ジョブスケジューラ SLURM 24.05.2 (パーティション: qc-mi210) を介してジョブを投入した。

全条件において、同一の Singularity コンテナイメージ内でコード生成・コンパイル・実行を行った。コンテナランタイムには Singularity-CE 4.3.7 を使用した。コンテナイメージは定義ファイル (ai-scientist-worker-base.def) に基づき, `docker://ubuntu:22.04` をブートストラップとして構築した。コンテナ内には Python 3.11, build-essential (gcc, g++, make 等), texlive-full, ML ライブラリ (PyTorch, NumPy, SciPy, pandas 等), および AI-Scientist 依存パッケージ (anthropic, openai, tiktoken 等) がインストールされている。実験用イメージ (experiment_environment.sif, バージョン 1.2, 2026 年 1 月 26 日ビルド) では、さらに HPC 性能計測に必要な numactl, gfortran, hwloc, perf (カーネル 5.14 ソースからビルド) を追加した。

コンテナの実行モードとして、`--writable-tmpfs` による書き込みを tmpfs オーバーレイで実現し、`--fakeroot` で root 相当の操作を可能にした。並列ワーカーごとに独立した SIF イメージコピーを使用する `--per-worker-sif` オプションにより、複数ワーカーの同時パッケージインストールにおけるファイルシステム競合を防止した。`--use_gpu` オプションを `true` に設定することで `--nv` フラグによる NVIDIA GPU のコンテナ内利用が可能であるが、本実験では Himeno Benchmark が CPU ベースのベンチマークであるため使用しなかった。この設計により、LLM エージェントがコンテナ内で自由にコンパイラやライブラリをインストールしながらも、ホスト環境やベースイメージに影響を与えない安全な実験実行を実現している。

であり、リリース直後にはモデルルーティングの不具合が報告されていた (<https://github.com/openai/codex/issues/11189>)。

^{*3} 2025 年 11 月 18 日にプレビュー版として公開。実験時点で安定版は未提供であり、プレビュー版は Google により内部の重みが更新される可能性があるため、正確なモデルスナップショットは特定できない。

4.4 評価指標

以下の観点から各条件の結果を比較する。

4.4.1 タスク完遂段階

各エージェントが Himeno Benchmark タスクをどの段階まで遂行できたかを以下の基準で評価する。

- **環境構築成功**：必要な依存関係のインストールと実行環境の構築が完了したか
- **コンパイル成功**：生成されたコードが正常にコンパイルされたか
- **実行成功**：コンパイルされたプログラムが正常に実行されたか
- **結果取得成功**：性能指標（MFLOPS 等）の取得・解析まで完了したか

4.4.2 LLM レビュースコア

各エージェントの出力を提案システムのポスト処理パイプラインに通し、LLM レビュー機能による以下の評価指標で比較する。

- **Overall**（総合評価）：10 段階
- **Originality**（独創性）：4 段階
- **Quality**（品質）：4 段階
- **Clarity**（明確性）：4 段階
- **Significance**（重要性）：4 段階
- **Soundness**（健全性）：4 段階
- **Presentation**（表現）：4 段階
- **Contribution**（貢献）：4 段階
- **Confidence**（確信度）：5 段階
- **Decision**（判定）：Accept / Reject

4.5 実験結果

提案手法（HPC-AutoResearch）により、提案システムは 7 ページの研究論文を自動生成した。生成された論文のタイトルは “Autotuning for Reproducibility: Jointly Optimizing Median Speed and Tail Stability for NUMA Stencil Codes” であり、Himeno Benchmark を用いた NUMA ステンシルコードの性能最適化について、実験計画・実行・分析・論文執筆までの全工程が自動的に遂行された。なお、本実験ではレビューバイアスモードとして pos（寛容モード）を使用しており、レビュースコアはこのバイアス設定の下で生成されたものである点に留意されたい。図 4.1–4.7 に生成された論文の全ページを示す。以下、生成論文の各部について述べる。

4.5.1 生成論文の構成

生成された論文は, Abstract, Introduction, Related Work, Method, Experimental Setup, Experiments (6つのサブセクション), Conclusion, References, Supplementary Material の構成を持つ7ページの論文であり, 6つの図と15件の参考文献を含む.

4.5.2 実験環境の自動取得

システムは Phase 0 において自律的に HPC 環境情報を取得し, 生成論文の Experimental Setup セクション (図 4.3) に以下の環境を正しく記述した: デュアルソケット AMD EPYC 9554 (128 コア, 2 NUMA ノード), Ubuntu 22.04, gcc 11.4.0. 対象ベンチマークとして OpenMP Himeno ステンシルプロキシを選定し, 各構成につき 12 回の反復・5 回のウォームアップイテレーションで計測するハーネスを自動設計した. 評価指標として中央値スループット (MFLOPS), 変動係数 ($CV = \sigma/\mu$), テール劣化 (p99/median) の3つを定義し, 性能の速度と安定性を同時に評価する多目的フレームワークを構築した.

4.5.3 スレッドスケーリング

生成論文の Section 6.1 (図 4.3) では, 固定バインディング (`OMP_PROC_BIND=true`, `OMP_PLACES=cores`, `OMP_SCHEDULE=static`) の下でスレッド数を 1 から 128 まで変化させたスケーリング結果が報告されている. 中央値スループットは 32–64 スレッドまで急速に増加し 64 スレッド付近でピーク ($\approx 2.06 \times 10^4$ MFLOPS) に達するが, それ以上のスレッド数では低下する. 同時に, スレッド数がピークを超えると中央値実行時間の安定性も悪化し, 64 スレッドがランタイムノブ探索の焦点であることが示された.

4.5.4 ランタイムノブスイープによる安定性差異

生成論文の Section 6.2 (図 4.3, 図 4.4) では, 64 スレッド固定の下でランタイム構成 (`OMP_PROC_BIND`, `OMP_PLACES`, `OMP_SCHEDULE`, `OMP_WAIT_POLICY` の組み合わせ) をスイープした結果が報告されている. 速度と安定性は一次元的でなく, 類似の中央値を持つ構成でも CV および p99/median に顕著な差異が生じる. 図 4.4 の散布図では, ランタイム CV 対中央値スループット, および p99/median 対中央値スループットの分布が示され, 非支配構成がハイライトされている.

4.5.5 実行間分布の重テール性

生成論文の Section 6.3 (図 4.4) では, 構成ごとの実行時間分布とスループット分布が示されている. 一部の構成では heavy tail と外れ値が観測され, 中央値のみでは性能の全体像を捉えられないことが示された. 特にスレッド配置や OS 干渉に敏感な構成では, 稀な極端スローダウンが発生し, これが p99/median の悪化として定量化されている.

4.5.6 アブレーション実験

生成論文の Section 6.4 (図 4.4) では、OMP_WAIT_POLICY を PASSIVE に固定した場合のアブレーション結果が報告されている。PASSIVE 設定はスループットを約半減 (中央値 $\approx 0.1 \times 10^4$ MFLOPS, CV ≈ 0.14 , p99/median ≈ 1.45) させ、かつ変動性を増大させる。この劣化は同期ポイントでのスレッドパーキングとウェイクアップレイテンシに起因し、ACTIVE 設定 (中央値・安定性ともにデフォルトとほぼ同等) との対照的な結果が示された。

4.5.7 多目的構造の可視化

生成論文の Section 6.5 (図 4.4, 図 4.5) では、目的関数の大きさが異なる (MFLOPS 対 CV 対 p99/median) ため、目的ごとの正規化を行った変換目的行列 (Normalized transformed objective matrix) が示されている。この可視化により、各構成の目的空間における位置関係と、中央値のみの最適化では安定性の犠牲が見落とされることが直感的に確認できる。

4.5.8 分布的目的関数による構成選択

生成論文の Section 6.6 (図 4.6) では、3 種のスイープ (Baseline sweep, Stability oriented sweep, OMP places granularity sweep) について、上位構成を中央値実行時間でランキングし、CV と p99/median をオーバーレイした棒グラフが示されている。中央値が最小の構成が必ずしも CV や p99/median で最良ではなく、分布的目的関数を考慮することで異なる構成が選択されうることが報告された。これは「再現性のためのオートチューニング」という生成論文の中心的主張であり、Pareto 集合に基づく構成選択の有用性を実証している。

Autotuning for Reproducibility: Jointly Optimizing Median Speed and Tail Stability for NUMA Stencil Codes

Anonymous Authors
Anonymous Institution
anonymous@example.com

Abstract

Modern HPC nodes can exhibit stochastic performance due to NUMA placement, thread migration, OpenMP runtime policies, and OS noise, yielding heavy-tailed slowdowns that are invisible when reporting only a single throughput number. We propose a stability-oriented autotuning methodology that treats performance as a distribution and optimizes three objectives jointly: median throughput (MFLOPS), run-to-run variability (coefficient of variation, CV), and tail degradation (p99/median runtime). Using an OpenMP Himeno stencil proxy on a dual-socket NUMA system, we evaluate practical runtime knobs (OpenMP binding/places/schedule, numactl allocation, and OMP_WAIT_POLICY) with replicated measurements and Pareto analysis. Our results show large configuration-to-configuration differences in both median performance and stability at fixed thread count, and that some settings can be simultaneously faster and more stable by avoiding pathological placement and synchronization regimes. We provide a reproducible measurement harness that outputs per-configuration distributions, non-dominated sets, and hypervolume summaries, and we highlight concrete cases where tail stability objectives change the selected configuration compared to median-only tuning.

1 Introduction

Performance engineering and benchmarking in high-performance computing (HPC) increasingly faces a reproducibility problem: repeated executions of the same code and input can yield meaningfully different runtimes on modern multicore NUMA nodes. Such variability is often treated as *measurement noise*, yet system-level effects—NUMA first-touch placement, thread migration, OS jitter, and runtime synchronization policies—can induce heavy tails and even regime switching in observed performance distributions [Engelmann, 2013, Gioiosa et al., 2004, Seelam et al., 2011]. This is particularly consequential for stencil codes: they are memory-sensitive, contain frequent synchronization, and traverse different cache/bandwidth regimes as the working set grows, making them susceptible to placement and interference effects [Wittmann and Hager, 2010].

The central hypothesis of this paper is that the configuration that maximizes *median* throughput is often different from the configuration that minimizes *variability* and *tail slowdowns*. Median performance is typically dominated by locality and SIMD efficiency (code-level effects), while stability and tails can be dominated by runtime placement and system noise (system-level effects). Tail metrics are widely recognized as first-class objectives in large-scale systems because rare slowdowns can dominate end-to-end behavior [Dean and Barroso, 2013], but they are rarely used as primary objectives in HPC kernel tuning.

We therefore advocate a *distribution-first* view of performance: each configuration induces a distribution of runtimes/throughputs, and tuning should optimize not only central tendency but also dispersion and tail risk. Concretely, we optimize (i) median MFLOPS, (ii) coefficient of variation (CV) of runtime, and (iii) tail degradation measured as p99/median runtime. We focus on a practical tuning space of runtime knobs that are accessible to users without privileged access: OpenMP affinity/placement/scheduling, numactl NUMA allocation policy, and OMP_WAIT_POLICY.

Contributions. This work contributes:

- A stability-oriented autotuning methodology that treats performance as a distribution and makes median/CV/p99 objectives explicit, motivated by tail-at-scale concerns [Dean and Barroso, 2013].
- A reproducible measurement harness that executes replicated sweeps, computes distributional statistics, identifies non-dominated configurations, and summarizes trade-offs using hypervolume.
- Empirical evidence on an OpenMP Himeno stencil proxy that runtime knobs can change both median performance and stability substantially at fixed thread count, and that stability objectives can change the selected configuration.
- Targeted ablations showing that some runtime policies (notably OMP_WAIT_POLICY=PASSIVE) can simulta-

neously degrade median and stability, producing heavy-tailed slowdowns.

2 Related Work

Autotuning and automatic performance programming. Empirical autotuning frameworks such as Active Harmony [Tapus et al., 2002] and broader automatic performance programming perspectives [Püschel, 2011] motivate searching over implementation choices to adapt to complex architectures. In stencil-specific contexts, code generation and autotuning systems such as PATUS [Christen et al., 2011] explore transformations including tiling, vectorization, and unrolling to maximize throughput. Polyhedral compilation techniques provide a foundation for systematic loop transformations and locality optimization [Sadayappan and Bondhugula, 2008], and temporal blocking methods (e.g., wavefront/diamond) illustrate the sensitivity of stencil performance to tiling parameters [Malas et al., 2014]. However, these efforts typically optimize a single scalar objective (mean/median throughput) and do not treat variability or tail metrics as first-class tuning targets.

Affinity, NUMA placement, and OS noise. Topology discovery and affinity management are well-established in HPC practice. The `hwloc` library provides a canonical interface for reasoning about hardware hierarchies and affinities [Broquedis et al., 2010]. NUMA memory affinity is recognized as non-trivial for OpenMP programs, motivating dedicated approaches and libraries [Schmidl et al., 2014]. Prior work documents that OS services and jitter can perturb parallel performance [Gioiosa et al., 2004, Seelam et al., 2011], and that such noise matters at scale [Engelmann, 2013]. ccNUMA locality issues in shared-memory parallelism can cause large performance swings when threads and pages are poorly placed [Wittmann and Hager, 2010]. Our work differs by *jointly* evaluating runtime knobs under *distributional* objectives (median/CV/p99) and by reporting Pareto trade-offs rather than a single “best” configuration.

Tail metrics as objectives. Tail latency is a central concern in large-scale systems because rare slowdowns can dominate user-visible performance; optimizing averages alone can be misleading [Dean and Barroso, 2013]. We transfer this principle to HPC kernel tuning by making tail degradation (p99/median runtime) a primary objective alongside median throughput.

3 Background

3.1 Why stencil kernels expose speed–stability tradeoffs

Stencil computations repeatedly update grid points using fixed-radius neighborhoods. Their memory access is structured (enabling attribution via locality reasoning), but they also include

synchronization and bandwidth pressure that can amplify system noise. On NUMA nodes, first-touch allocation and thread placement can change the fraction of remote accesses across runs, producing variability and tails [Wittmann and Hager, 2010]. OS jitter and system services can introduce delays that propagate through synchronization points [Gioiosa et al., 2004, Seelam et al., 2011, Engelmann, 2013]. These mechanisms can affect the *distribution* of observed runtimes, not just the mean.

3.2 Distributional performance metrics

For each configuration, we collect repeated measurements and compute:

- **Median throughput** (MFLOPS), as a robust central tendency.
- **Runtime CV**, $CV = \sigma/\mu$, measuring run-to-run dispersion.
- **Tail degradation**, p99/median runtime, measuring the severity of rare slow runs relative to typical behavior.

Tail metrics are motivated by the “tail at scale” effect [Dean and Barroso, 2013]; in the HPC setting, p99/median captures sensitivity to rare placement/jitter events that can be invisible in median-only reporting.

4 Method

4.1 Tuning problem

We consider a finite configuration space $\mathcal{C}$ defined by runtime knob settings (and in the broader project vision, code variants). Each configuration $c \in \mathcal{C}$ induces a distribution of runtimes T_c and throughputs P_c . We optimize three objectives:

$$\max \text{median}(P_c), \quad \min \text{CV}(T_c), \quad \min \frac{\text{p99}(T_c)}{\text{median}(T_c)}.$$

We report non-dominated configurations (Pareto set) and summarize the overall trade-off coverage using hypervolume.

4.2 Search strategy used in this study

While multi-objective evolutionary algorithms are a natural fit for empirical tuning [Tapus et al., 2002, Püschel, 2011], the experiments reported here use *grid sweeps* over practical runtime knobs, followed by Pareto filtering. This choice reflects the goal of establishing a reproducible measurement protocol and demonstrating that distributional objectives can change conclusions even in small, user-accessible knob spaces.

4.3 Measurement harness

Our harness executes repeated runs per configuration, parses a single-line result record, and stores: (i) raw per-run samples, (ii) per-configuration summary statistics (median, mean, std, CV, p99/median), (iii) transformed objective vectors, and (iv)

hypervolume computed over the transformed objectives. The harness is designed to fail loudly on invalid JSON configurations and to produce a single .npy artifact for downstream analysis.

5 Experimental Setup

System. Experiments were run on a dual-socket AMD EPYC 9554 system with 128 cores and 2 NUMA nodes, running Ubuntu 22.04 and compiled with gcc 11.4.0. Topology and placement control are consistent with established affinity and NUMA management practice [Broquedis et al., 2010, Schmidl et al., 2014].

Benchmark. We use an OpenMP Himeno implementation as a controlled stencil proxy. The executable reports `time_sec`, `mflops`, and a convergence error (`gosa`) for sanity checking.

Knobs and configurations. We study:

- Thread count sweep: `OMP_NUM_THREADS` in $\{1, 2, 4, 8, 16, 32, 48, 64, 96, 128\}$ under fixed binding (`OMP_PROC_BIND=true`, `OMP_PLACES=cores`, `OMP_SCHEDULE=static`).
- Runtime knob grid at 64 threads: `OMP_PROC_BIND` $\in \{\text{false}, \text{close}, \text{spread}\}$, `OMP_PLACES` $\in \{\text{cores}, \text{sockets}\}$, `OMP_SCHEDULE` $\in \{\text{static}, \text{guided}\}$, and `numactl` policy $\in \{\text{none}, \text{localalloc}, \text{interleave}\}$ (36 configurations).
- Wait-policy ablation at 64 threads: `OMP_WAIT_POLICY` $\in \{\text{(unset)}, \text{ACTIVE}, \text{PASSIVE}\}$ under a fixed baseline runtime setting.

Replication. Unless otherwise stated, each configuration is run with 12 repetitions and 5 warmup iterations (as encoded in the sweep configurations). We emphasize that p99 estimation is sample-hungry; in this study, p99/median should be interpreted as an indicative tail metric rather than a high-confidence estimate for all configurations.

6 Experiments

6.1 Thread scaling: median speed peaks near 64 threads

Figure 1 shows median throughput and median runtime versus `OMP_NUM_THREADS` under fixed binding and scheduling. Throughput increases rapidly up to 32–64 threads and peaks around 64 threads (median $\approx 2.06 \times 10^5$ MFLOPS), then declines at higher thread counts (e.g., 96 and 128). The corresponding median runtime decreases to its minimum around 48–64 threads and increases thereafter. This motivates using 64 threads as the focus point for the runtime-knob stability sweeps.

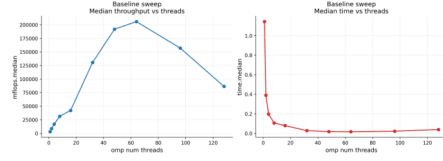


Figure 1: Thread scaling under fixed binding (`OMP_PROC_BIND=true`, `OMP_PLACES=cores`, `OMP_SCHEDULE=static`). Median throughput peaks near 64 threads and degrades beyond, motivating stability sweeps at 64 threads.

6.2 Runtime knob sweeps at fixed threads reveal large stability differences

A key question is whether stability objectives (CV and p99/median) change the recommended runtime configuration compared to median-only tuning. Figures 2(b) and 2(c) summarize the 64-thread runtime grid.

Speed vs. stability is not one-dimensional. Figure 2(b) visualizes median throughput versus median runtime, with points colored by runtime CV and non-dominated configurations highlighted. We observe multiple clusters at similar median runtimes but differing stability, indicating that stability is not fully determined by median speed.

CV and tail ratio co-move but are not identical. Figure 2(c) plots CV versus p99/median runtime, colored by median throughput. Configurations with higher CV tend to have worse tails, consistent with noise and placement effects creating rare slow runs [Engelmann, 2013, Gioiosa et al., 2004, Seelam et al., 2011]. However, there remains spread: some configurations have comparable CV but different p99/median, motivating keeping both objectives.

6.3 Run-to-run distributions: some configurations exhibit heavy tails and outliers

Distributional plots are essential because the same median can hide qualitatively different behavior. Figure 3 shows run-to-run boxplots for runtime and throughput across configurations. Some configuration tags exhibit tight interquartile ranges (stable), while others show wide spread and outliers (unstable). This supports treating slow runs as signal rather than discarding them as *bad data*, consistent with OS noise and placement sensitivity being real performance factors [Seelam et al., 2011, Engelmann, 2013].

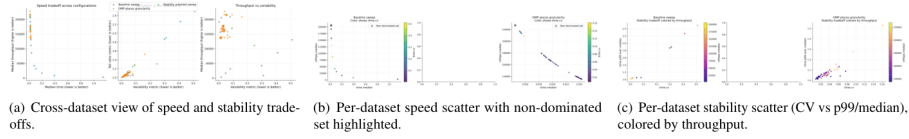


Figure 2: Distribution-first views of the runtime-knob sweeps. Even at fixed thread count, configurations differ substantially in both median performance and stability, and stability objectives can separate configurations with similar median speed. Tail metrics are motivated by the importance of rare slowdowns [Dean and Barroso, 2013].

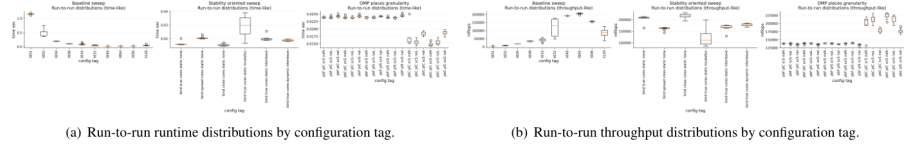


Figure 3: Run-to-run distributions differ substantially across runtime knob settings. Some configurations show heavy-tailed slowdowns and throughput outliers, consistent with placement and OS effects impacting reproducibility [Gioiosa et al., 2004, Seelam et al., 2011].

6.4 Ablation: OMP_WAIT_POLICY=PASSIVE can be simultaneously slower and less stable

To isolate one synchronization-related runtime lever, we ablate OMP_WAIT_POLICY under a fixed configuration. The results show that PASSIVE is a dominated choice in this setting: it approximately halves median throughput and introduces large variability and heavy tails compared to (unset) and ACTIVE. This is consistent with additional parking and wake-up latency at synchronization points amplifying tail events, a mechanism aligned with OS noise sensitivity in parallel programs [Gioiosa et al., 2004, Seelam et al., 2011].

We report the exact summary statistics from the sweep artifact:

- **Default (unset):** median time ≈ 0.0265 s, median throughput $\approx 1.25 \times 10^5$ MFLOPS, CV ≈ 0.014 , p99/median ≈ 1.036 .
- **ACTIVE:** similar median and stability to default (near-overlapping in the plots).
- **PASSIVE:** median time ≈ 0.053 – 0.054 s, median throughput $\approx 6.1 \times 10^4$ MFLOPS, CV ≈ 0.14 , p99/median ≈ 1.45 .

These results emphasize why stability-oriented tuning should include tail metrics: PASSIVE not only slows the median but also creates rare, severe slowdowns.

6.5 Summarizing multi-objective structure: normalized objective matrices

Objective magnitudes differ (MFLOPS vs. CV vs. ratios), so visualization requires per-objective normalization. Figure 5

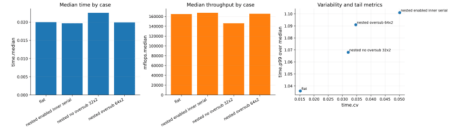


Figure 4: Ablation-style case summary (nested OpenMP settings). The plot illustrates that changes in runtime policy can shift not only median throughput/time but also CV and p99/median, reinforcing the need for distributional objectives. Tail sensitivity is consistent with the general importance of rare slowdowns [Dean and Barroso, 2013].

shows normalized transformed objective matrices for several sweeps, highlighting that some configurations are extreme outliers in stability objectives even when throughput is not extreme. This motivates Pareto analysis and careful objective scaling when summarizing trade-offs.

6.6 Selecting configurations under distributional objectives

Figure 6 ranks configurations by median runtime and overlays CV and p99/median. This view makes the practical decision explicit: a configuration chosen by median alone may not minimize tail risk, and conversely, a more stable configuration may incur only a small median penalty. This is the core “autotuning for reproducibility” argument: the output should be a Pareto set (or a policy over it), not a single number.

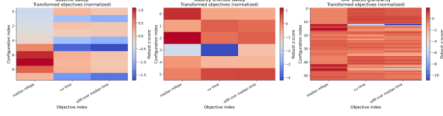


Figure 5: Normalized transformed objectives (median MFLOPS, $-CV$, and $-p99/median$) across several sweeps. Per-objective normalization is necessary to compare objectives with different units/magnitudes.

7 Conclusion

This paper argues and demonstrates that stencil-kernel tuning should treat performance as a distribution: median throughput alone can miss important variability and tail behavior driven by NUMA placement, runtime policies, and OS noise [Engelmann, 2013, Gioiosa et al., 2004, Seelam et al., 2011]. Using an OpenMP Himeno stencil proxy on a dual-socket NUMA node, we showed that practical runtime knobs can induce large differences in CV and p99/median at fixed thread count, and that some policies (e.g., `OMP_WAIT_POLICY=PASSIVE`) can be simultaneously slower and much less stable. We provide a reproducible harness that reports distributions, Pareto sets, and hypervolume summaries, enabling stability-oriented autotuning rather than median-only tuning.

Limitations and next steps. The present experiments use modest repetition counts (12) for many sweeps; p99 estimation is therefore indicative rather than definitive for all configurations. Future work should increase repetitions for tail validation (or report p95 alongside p99), extend beyond runtime knobs to include code variants (tiling/SIMD/unrolling) as in stencil autotuning systems [Christen et al., 2011], and incorporate topology-aware placement reasoning [Broquedis et al., 2010, Schmidl et al., 2014, Wittmann and Hager, 2010] to build actionable selection policies over Pareto fronts.

References

François Broquedis, Jérôme Clet-Ortega, Stéphanie Moreaud, N. Furmento, Brice Goglin, Guillaume Mercier, Samuel Thibault, and Raymond Namyst. hwloc: A generic framework for managing hardware affinities in hpc applications. *2010 18th Euromicro Conference on Parallel, Distributed and Network-based Processing*, pages 180–186, 2010.

M. Christen, O. Schenk, and H. Burkhart. Patu: A code generation and autotuning framework for parallel iterative stencil computations on modern microarchitectures. *2011 IEEE International Parallel & Distributed Processing Symposium*, pages 676–687, 2011.

J. Dean and L. Barroso. The tail at scale. *Commun. ACM*, 56: 74–80, 2013.

C. Engelmann. Investigating operating system noise in extreme-scale high-performance computing systems using simulation. 2013.

Roberto Gioiosa, Fabrizio Petrini, Kei Davis, and Fabien Lebaillif-Delamare. Analysis of system overhead on parallel computers. *Proceedings of the Fourth IEEE International Symposium on Signal Processing and Information Technology, 2004.*, pages 387–390, 2004.

T. Malas, G. Hager, H. Ltaief, and D. Keyes. Towards energy efficiency and maximum computational intensity for stencil algorithms using wavefront diamond temporal blocking. *ArXiv*, abs/1410.5561, 2014.

Markus Püschel. Automatic performance programming. pages 1–2, 2011.

P. Sadayappan and Uday Bondhugula. Effective automatic parallelization and locality optimization using the polyhedral model. 2008.

Dirk Schmidl, Tim Cramer, C. Terboven, Dieter an Mey, and Matthias S. Müller. An openmp extension library for memory affinity. pages 103–114, 2014.

Seetharami R. Seelam, L. Fong, John Lewars, John Divirgilio, B. F. Veale, and K. Gildea. Characterization of system services and their performance impact in multi-core nodes. *2011 IEEE International Parallel & Distributed Processing Symposium*, pages 104–117, 2011.

Cristian Tapus, I. Chung, and J. Hollingsworth. Active harmony: Towards automated performance tuning. *ACM/IEEE SC 2002 Conference (SC’02)*, pages 44–44, 2002.

M. Wittmann and G. Hager. Optimizing ccnuma locality for task-parallel execution under openmp and tbb on multicore-based systems. *ArXiv*, abs/1101.0093, 2010.

Supplementary Material

A Reproducibility details: configuration grids and statistics

The sweeps in this paper are designed to be reproducible and user-accessible. Configurations are specified as strict JSON and applied via OpenMP environment variables and `numactl`. For each configuration, the harness stores raw samples and computes median, mean, standard deviation, CV, and p99/median. Non-dominated configurations are identified by Pareto filtering over (median MFLOPS, $-CV$, $-p99/median$). Hypervolume is computed in the transformed objective space as a summary of trade-off coverage; objective normalization is used for visualization (see Figure 5).

B Why we keep both CV and p99/median

CV captures overall dispersion, while p99/median penalizes rare slowdowns. Prior work emphasizes that rare slowdowns can dominate perceived performance [Dean and Barroso, 2013]. In our sweeps, CV and p99/median often co-vary but are not redundant (see Figure 2(c)), motivating multi-objective tuning rather than collapsing to a single stability scalar.

7

図 4.7 生成論文 (7/7 ページ) : 補足資料

4.5.9 自動レビュー結果

生成された論文に対して, `pos` バイアスモード (`--review-bias pos`) で LLM レビューを実行した結果を以下に示す. レビュー出力の全文は Listing 4.1 に掲載する.

総合評価

レビュアー LLM は生成論文に対して表 4.1 に示すスコアを付与し、最終判定として **Accept**（採択）を返した。Overall（総合評価）は 10 段階中 7、Confidence（確信度）は 5 段階中 4 であった。

表 4.1 自動レビュースコア（pos バイアスモード）

評価項目	スコア
Originality（独創性）	3 / 4
Quality（品質）	3 / 4
Clarity（明確性）	3 / 4
Significance（重要性）	3 / 4
Soundness（健全性）	3 / 4
Presentation（表現）	3 / 4
Contribution（貢献）	3 / 4
Overall（総合）	7 / 10
Confidence（確信度）	4 / 5
Decision（判定）	Accept

評価された Strengths

レビュアーは以下の 6 点を生成論文の強みとして評価した：

1. HPC における再現性・安定性を第一級のチューニング目標として扱う問題設定の適時性と実的重要性
2. 中央値性能・CV・テール比の 3 指標による多目的定式化と、トレードオフを単一数値に集約しない Pareto 分析の適切な使用
3. OpenMP アフィニティ・配置・スケジュール等、コード変更や特権アクセスを必要としないユーザーアクセス可能なチューニング空間の選択
4. 安定性は中央値性能のみでは決定されないという中心的主張を支持する実証結果
5. OMP_WAIT_POLICY アブレーションによる、速度と安定性の両方で劣勢な設定の具体的・再現可能な実例
6. 生サンプル保存・Pareto 集合・ハイパーボリュームサマリーを含む再現可能な計測ハーネスの有用性

指摘された Weaknesses

同時に、以下の 5 点の弱点が指摘された：

1. 12 回の反復での p99 推定の統計的妥当性が弱く、不確実性の限界や頑健性チェックが不十分
2. 「オートチューニング」と称しているが、実質は網羅的グリッドスイープ + Pareto フィルタリングであり、大規模ノブ空間へのスケーリングや適応的サンプリングの議論が限定的

3. ハイパーボリューム計算の詳細（参照点の選択，正規化，変換）が再現性・比較可能性の観点で不十分
4. CPU 周波数・ターボ設定，バックグラウンド負荷分離，ランチャープロセスの固定化など，変動性に影響する実験条件が完全には特定されていない
5. ベンチマーク範囲が狭い（単一プロキシ，単一マシン）ため，他のカーネルやプラットフォームへの一般化の証拠が限定的

レビュー結果の解釈

本レビューは pos バイアスモードで実行されており，不確実な場合に高評価を付与する傾向がある点を考慮する必要がある．にもかかわらず，レビュアーは統計的妥当性やスケーラビリティについて具体的な弱点を指摘しており，単なる追認的レビューではなく，建設的な改善提案を含む評価を生成している．特に，p99 推定の信頼区間，ハイパーボリュームの参照点詳細，CPU 周波数制御といった 6 つの具体的質問が提示されており，研究改善の方向性を示唆する有用なフィードバックとなっている．

Listing 4.1 自動レビュー結果 (review_text.txt)

```

1 {
2   "Summary": "This paper argues that HPC kernel tuning should treat performance as a
   distribution rather than a single scalar, because NUMA placement, OpenMP runtime
   policies, thread migration, and OS noise can induce heavy-tailed slowdowns. The
   authors propose a stability-oriented autotuning methodology that jointly optimizes (
   i) median throughput (MFLOPS), (ii) run-to-run variability via runtime coefficient
   of variation (CV), and (iii) tail degradation via p99/median runtime. They implement
   a reproducible measurement harness that performs replicated configuration sweeps,
   records per-configuration samples, computes distributional statistics, extracts
   Pareto-optimal configurations, and summarizes trade-offs via hypervolume.
   Experiments on an OpenMP Himeno stencil proxy on a dual-socket AMD EPYC NUMA system
   show substantial configuration-to-configuration differences in both median speed and
   stability at fixed thread count, and an ablation indicates OMP_WAIT_POLICY=PASSIVE
   can be simultaneously slower and much less stable than alternatives.",
3   "Strengths": [
4     "Timely and practically important problem framing: reproducibility/stability as a
       first-class tuning goal in HPC, not just maximizing mean/median throughput.",
5     "Clear multi-objective formulation (median performance, CV, tail ratio) and
       appropriate use of Pareto analysis to avoid collapsing trade-offs into a single
       number.",
6     "User-accessible tuning space (OpenMP affinity/places/schedule, numactl, wait policy
       ) makes the methodology actionable for practitioners without requiring code
       changes or privileged access.",
7     "Empirical results support the central claim that stability is not determined solely
       by median performance; configurations with similar medians can differ
       substantially in CV and tail behavior.",
8     "The OMP_WAIT_POLICY ablation is a concrete, easy-to-reproduce example where a
       runtime policy is strongly dominated on both speed and stability, reinforcing
       the value of distributional metrics.",
9     "The measurement harness concept (storing raw samples, emitting artifacts, Pareto
       sets, hypervolume summaries) is a useful contribution for reproducible
       performance studies."
10  ],
11  "Weaknesses": [

```

"Statistical validity of p99 with 12 repetitions is weak; the paper acknowledges this, but several conclusions/visualizations still lean on p99/median without uncertainty bounds or robustness checks.",

"The approach is described as 'autotuning' but the search in this paper is primarily exhaustive grid sweeps plus Pareto filtering; there is limited discussion of scaling to larger knob spaces or adaptive sampling/budget allocation.",

"Hypervolume computation details are somewhat underspecified for reproducibility/comparability (e.g., reference point choice, normalization/transforms used for HV vs only for visualization).",

"Some experimental controls that affect variability are not fully characterized (e.g., CPU frequency/turbo settings, background load isolation, pinning of the launcher process), which could confound stability metrics across runs.",

"The benchmark scope is narrow (single proxy, one machine); while acceptable for a methodology paper, it limits evidence that the proposed objectives/general conclusions transfer across kernels and platforms."

],

"Originality": 3,

"Quality": 3,

"Clarity": 3,

"Significance": 3,

"Questions": [

"Can you add uncertainty quantification for tail metrics (e.g., bootstrap confidence intervals for p95/p99/median, or report p95 alongside p99) and re-check whether the Pareto set changes materially under resampling?",

"Given the small sample size per configuration, did you consider using a more sample-efficient tail proxy (e.g., max/median, trimmed worst-k/median, or CVaR-style objective) and how sensitive are conclusions to this choice?",

"Please specify hypervolume details precisely: objective transforms/sign conventions, normalization (if any) used for HV (not just plots), and the reference point; ideally include this in the artifact or appendix for exact reproduction.",

"How stable are the results across different days/boots and under controlled CPU frequency settings (e.g., fixed governor, turbo disabled)? If not tested, please discuss as a limitation and/or add a small robustness check.",

"What is the exact Himeno problem size/parameters used (grid dimensions, iteration counts), and are they constant across sweeps? The paper mentions warmups and repetitions but not all benchmark parameters.",

"For the runtime-knob grid: do you observe interactions between numactl policy and OMP_PLACES/PROC_BIND that systematically explain the heavy-tail cases (e.g., remote memory + spread)? A short qualitative diagnosis would improve actionability."

],

"Limitations": [

"Tail estimation is sample-hungry; with 12 repetitions, p99/median is a noisy indicator and may lead to unstable Pareto membership. The paper should either increase repetitions for tail-focused claims or provide uncertainty estimates / use lower quantiles.",

"Results are demonstrated on a single proxy benchmark and a single dual-socket EPYC platform; generalization to other kernels (e.g., reductions, irregular access) and other architectures/runtimes remains to be shown.",

"The knob space is limited to runtime/environment settings; extending to code variants would be needed to position the method as a general autotuning framework rather than a measurement protocol.",

"Potential negative societal impact is limited, but the methodology could be used to present overly confident tail claims if users apply p99 with insufficient samples; the paper should more strongly warn against this and provide recommended sampling guidance."

```

35     ],
36     "Ethical Concerns": false,
37     "Soundness": 3,
38     "Presentation": 3,
39     "Contribution": 3,
40     "Overall": 7,
41     "Confidence": 4,
42     "Decision": "Accept"
43 }

```

4.5.10 Claude Code の結果

Claude Code は、Himeno Benchmark の 3 つの問題サイズ (SMALL, MIDDLE, LARGE) に対して、デフォルト環境 (明示的なピニング・NUMA 制御・ガバナ設定なし) で各 30 回の反復実行を行い、中央値 MFLOPS, CV, テール比 (P99/median, P1/median) を報告する論文を生成した。主な知見として、(1) 問題サイズの増大に伴い中央値 MFLOPS が低下すること、(2) MIDDLE サイズが最も低い CV (最も安定) を示すこと、(3) LARGE サイズでは二峰性分布が観測され、NUMA 配置とメモリ帯域幅競合が原因と推察されることが報告された。

LLM レビューでは、実用的な多指標テンプレートの提供や二峰性の観測が評価された一方、LARGE 二峰性の NUMA 帰属に直接的な証拠 (numastat, ページ局所性, パフォーマンスカウンタ等) がないこと、30 サンプルでの P99 推定の統計的妥当性、および制御変数の不足 (ピニングなし, ガバナ設定なし) が弱点として指摘された。最終判定は Overall 6/10 で **Accept** であった。レビュー出力の全文は Listing 4.2 に掲載する。

4.5.11 Codex の結果

Codex は、Himeno Benchmark の性能安定性に関する研究計画を策定し、ベースライン・ピニング・NUMA 局所割り当て・固定周波数の 4 条件を設計したものの、SLURM / 環境上の問題により**実験結果の取得に至らなかった**。生成された論文の Results セクションは空であり、実証的な知見は含まれていない。解析パイプライン (パーサ・プロットスクリプト) の実装には着手したが、実行・検証は未完了であった。

LLM レビューでは、分布的安定性指標の採用や介入条件の設計は評価されたものの、実験結果が一切ないことが致命的な弱点として指摘された。加えて、方法論の詳細不足 (ハードウェア構成, コンパイラフラグ, 反復回数等の未記載), 固定周波数条件の曖昧さ, 関連研究の位置づけの不明確さが弱点に挙げられた。最終判定は Overall 3/10 で **Reject** であった。レビュー出力の全文は Listing 4.3 に掲載する。

4.5.12 Gemini CLI の結果

Gemini CLI は、OpenMP スレッドアフィニティが Himeno Benchmark のスループットと性能安定性を与える影響を評価する論文を生成した。3 つの問題サイズ ($64 \times 64 \times 128$, $128 \times 128 \times 256$, $256 \times 256 \times 512$) に対して各 30 回の反復実行を行い、非バインド実行と `OMP_PROC_BIND=close` の 2 条件を比較した。主な知見として、明示的バインディングにより中央値性能が最大約 42% 向上し、CV およびテール比も改善されることが報告された。

LLM レビューでは、実行時変動性とテール挙動への着目、30 回反復による複数指標の報告、およびバイ
ンディングによる安定性改善の実用的知見が評価された。一方、再現性に必要な実験詳細の不足（CPU モ
デル・トポロジー、SMT/HT 設定、NUMA/メモリ配置ポリシー等の未記載）、スレッド数・OMP_PLACES
等の未報告、比較条件の限定性（spread 等との比較なし）、および図版の表示不具合が弱点として指摘さ
れた。最終判定は Overall 7/10 で **Accept** であった。レビュー出力の全文は Listing 4.4 に掲載する。

4.5.13 条件間の比較

表 4.2 にタスク完遂段階の比較を、表 4.3 に LLM レビュースコアの比較を示す。

表 4.2 タスク完遂段階の比較

完遂段階	提案手法	Claude Code	Codex	Gemini CLI
環境構築成功	○	○	×	○
コンパイル成功	○	○	×	○
実行成功	○	○	×	○
結果取得成功	○	○	×	○

Codex は、SLURM / 環境上の問題により実験の実行に至らず、全段階で失敗（×）となった。提案手
法、Claude Code、Gemini CLI はいずれも環境構築から結果取得まで全段階を完遂した。

表 4.3 LLM レビュースコアの比較（pos バイアスモード）

評価項目	提案手法	Claude Code	Codex	Gemini CLI
Originality（独創性）	3 / 4	2 / 4	2 / 4	2 / 4
Quality（品質）	3 / 4	3 / 4	1 / 4	2 / 4
Clarity（明確性）	3 / 4	3 / 4	2 / 4	2 / 4
Significance（重要性）	3 / 4	2 / 4	2 / 4	3 / 4
Soundness（健全性）	3 / 4	3 / 4	1 / 4	3 / 4
Presentation（表現）	3 / 4	3 / 4	2 / 4	2 / 4
Contribution（貢献）	3 / 4	2 / 4	1 / 4	3 / 4
Overall（総合）	7 / 10	6 / 10	3 / 10	7 / 10
Confidence（確信度）	4 / 5	4 / 5	5 / 5	3 / 5
Decision（判定）	Accept	Accept	Reject	Accept

Listing 4.2 Claude Code 自動レビュー結果

```

1 {
2   "Summary": "The paper presents an empirical study of run-to-run performance stability of
the Himeno benchmark across its standard SMALL, MIDDLE, and LARGE problem sizes on
a dual-socket AMD EPYC 9554 node. Using 2 warmup runs and 30 measured runs per size
in a default (no explicit pinning/NUMA/governor control) environment, the authors
characterize stability with three metrics: median MFLOPS, coefficient of variation (
CV), and a tail ratio (P99/median; additionally P1/median for worst-case slowdown).

```

The main findings are: (i) median MFLOPS decreases with problem size; (ii) MIDDLE exhibits the lowest CV (most stable), SMALL slightly higher, and LARGE much higher; (iii) LARGE shows a bimodal distribution with a ~0.4\to30.5% low-performance mode, which the paper attributes to NUMA placement and memory bandwidth contention; and (iv) the three metrics can rank sizes differently, motivating multi-metric reporting .",

"Strengths": [

- "Practical and relevant focus: performance stability/variability is important in HPC benchmarking but often underreported; the paper provides an actionable multi-metric template (median + dispersion + tails).",
- "Clean, simple experimental design (warmup exclusion, repeated runs, reporting of multiple descriptive statistics) that is easy to replicate in principle.",
- "Interesting empirical observation: bimodality for the LARGE working set and a stability \"sweet spot\" at MIDDLE; these are plausible and potentially useful for practitioners choosing benchmark sizes.",
- "The paper is reasonably careful to separate central tendency (median) from variability (CV) and from tail behavior (P1/P99 ratios), and it explicitly evaluates hypotheses."

],

"Weaknesses": [

- "Causal attribution for the LARGE bimodality is not convincingly demonstrated: NUMA placement is hypothesized but not validated with direct evidence (e.g., numastat , page locality, perf counters for remote accesses, bandwidth/latency measurements, or controlled numactl/pinning experiments).",
- "The P99/median metric is not statistically well supported with only 30 samples. With n=30, estimating the 99th percentile is essentially extrapolation; the tiny reported differences in P99/median across sizes (~1.0001\to1.0002) are likely noise and may not warrant ranking conclusions.",
- "The definition/interpretation of P99/median is somewhat awkward for MFLOPS where the more relevant tail is the lower tail (slow runs). The paper does include P1/median, but the headline emphasis still includes P99/median as a primary metric .",
- "Methodology leaves important confounders uncontrolled (no pinning, no governor setting, containerized build) which is defensible for a \to default environment\to study, but it weakens claims about cache hierarchy being the primary determinant versus, e.g., scheduler placement, frequency behavior, or background load.",
- "Some presentation artifacts: several figures/tables appear garbled in the provided rendering, which harms readability and confidence in the visual evidence."

],

"Originality": 2,

"Quality": 3,

"Clarity": 3,

"Significance": 2,

"Questions": [

- "NUMA attribution: Can the authors provide direct evidence that the low-performance mode in LARGE corresponds to worse NUMA locality (e.g., numastat before/after, page distribution per NUMA node, or perf/ibs-derived remote access rates)? Even a small controlled experiment (numactl --membind/--cpunodebind; first-touch initialization pinned to a socket) would greatly strengthen the claim.",
- "Tail metric choice: Given n=30, why use P99 specifically? Would the authors consider replacing it with a quantile that is estimable with 30 samples (e.g., P90/P95) or using bootstrap confidence intervals for quantiles/ratios?",
- "Quantile uncertainty: Please report uncertainty for CV and tail ratios (e.g., bootstrap CIs) to show whether differences like P99/median=1.0001 vs 1.0002 are meaningful.",

```

24     "Runtime environment: Were runs executed in an exclusive node allocation with CPU
        shielding/cgroup isolation, or could other jobs/daemons have interfered? Any
        monitoring of background load during the experiment?",
25     "Frequency behavior: Did the authors log per-run effective frequency (e.g., via
        aperf/mperf or sysfs) to rule in/out DVFS/turbo as a contributor to bimodality
        ?",
26     "Reproducibility: The paper mentions code/scripts/data in a repository; please
        specify a permanent artifact link (DOI or commit hash) and list exact Himeno
        version and compilation flags beyond -O3 (e.g., -march, -ffast-math, OpenMP
        disabled, etc.).",
27     "Single-core placement: Since there is no explicit pinning, which CPU core did the
        process run on across runs? Could run-to-run scheduling across different cores/
        CCXs explain part of the bimodality (e.g., different L3 slice locality or
        different NUMA node)?"
28 ],
29 "Limitations": [
30     "Single-node, single hardware platform (dual-socket EPYC 9554) and single-threaded
        execution only; conclusions may not generalize to multi-threaded runs, other
        microarchitectures, or cluster-scale environments.",
31     "Default-environment design intentionally avoids controlling pinning/NUMA/governor,
        which limits causal inference; observed variability may conflate multiple
        factors (scheduler placement, DVFS, background OS activity).",
32     "Tail estimation with 30 runs is weak for extreme quantiles (P99).",
33     "Potential negative societal impact is minimal; however, benchmark-based procurement
        decisions could be misled if tail metrics are over-interpreted without
        appropriate confidence/controls."
34 ],
35 "Ethical Concerns": false,
36 "Soundness": 3,
37 "Presentation": 3,
38 "Contribution": 2,
39 "Overall": 6,
40 "Confidence": 4,
41 "Decision": "Accept"
42 }

```

Listing 4.3 Codex 自動レビュー結果

```

1 {
2     "Summary": "The paper outlines an intended study of run-to-run performance stability for
        the Himeno benchmark (Jacobi 3D Poisson stencil) across problem sizes and runtime
        configurations (baseline scheduling, core pinning, pinning plus NUMA-local
        allocation, and fixed CPU frequency). It proposes reporting distributional stability
        metrics\u2014median MFLOPS, coefficient of variation, and p99/median\u2014to
        capture central tendency and tail degradation, and mentions an implemented parsing/
        plotting pipeline to compute these metrics from benchmark logs.",
3     "Strengths": [
4         "Addresses a practically relevant HPC concern: performance variability for memory-
            bound stencil codes under OS noise, NUMA placement, and frequency scaling.",
5         "Uses distributional metrics (median, CV, tail ratio) that are more informative for
            stability than mean-only reporting and align with prior variability literature
            .",
6         "Considers a sensible set of common system interventions (pinning, NUMA locality,
            frequency governor) and multiple benchmark sizes to probe cache/memory regime
            changes.",
7         "States that an end-to-end analysis pipeline (parser + plotting) is already

```

```

    implemented, which could support reproducibility if released."
],
"Weaknesses": [
    "No experimental results are provided: the Results section is empty due to SLURM/
    environment issues, so the paper does not substantiate any claims or provide any
    empirical insight.",
    "Methodology is underspecified for reproducibility and scientific control (hardware/
    CPU model, sockets, memory configuration, OS/kernel, BIOS settings, turbo/boost,
    SMT/HT, compiler flags, thread count, affinity policy details, NUMA policy tool
    /settings, frequency values, run duration, warm-up, number of repetitions).",
    "The proposed metrics are basic; without results and deeper analysis (e.g.,
    statistical tests, confidence intervals for quantiles/ratios, autocorrelation/
    time-of-day effects, separation of within-node vs across-node variability), the
    contribution appears incremental.",
    "No validation of the parsing pipeline/metric computation is shown (sanity checks,
    cross-validation against known outputs, handling of outliers/failed runs).",
    "The fixed-frequency condition is described vaguely (\u201cwhen permitted\u201d);
    the paper does not clarify how frequency is enforced/verified (e.g., reading
    MSRs, disabling turbo) which is crucial for stability studies.",
    "Related-work positioning is generic; it does not clearly articulate what new
    insight beyond prior variability studies is expected, nor why Himeno
    specifically yields novel conclusions."
],
"Originality": 2,
"Quality": 1,
"Clarity": 2,
"Significance": 2,
"Questions": [
    "Can the authors provide the missing experimental results (raw distributions and
    summary metrics) for at least one well-specified target system, including number
    of repetitions per configuration/size and confidence intervals (e.g., bootstrap
    ) for median and p99/median?",
    "What exact hardware/software environment is used (CPU model, sockets, NUMA topology
    , memory channels, OS/kernel, scheduler settings, SLURM version/config, compiler
    and flags, Himeno version, pthread settings)?",
    "How are pinning and NUMA-local allocation implemented (taskset/numactl/SLURM --cpu-
    bind/--mem-bind), and how is correctness verified (e.g., numastat, perf, hwloc
    )?",
    "How is fixed frequency enforced and validated (governor, turbo disabled, min=max
    frequency, readings from /sys or MSRs), and what frequencies are tested?",
    "How many runs per configuration are planned, and how do you handle failed/slow runs
    and outliers? Is p99 estimated from enough samples to be stable?",
    "Do you observe multimodality or regime changes (e.g., due to page placement,
    throttling, background daemons)? If so, will you report full distributions (ECDF
    /violin) in addition to the three scalar metrics?",
    "Will you compare across nodes and across time to separate node-to-node variability
    from run-to-run variability on the same node?"
],
"Limitations": [
    "As written, the work has no empirical evidence; conclusions are speculative and
    restate expectations from prior work.",
    "The study may be highly system-specific; without multiple platforms or careful
    environmental control, generalization may be limited.",
    "Tail metrics like p99/median can be unstable with small sample sizes; the paper
    should discuss sample complexity and uncertainty estimation.",
    "Potential negative societal impact is low, but the work could be used to optimize

```

```

35         system settings in ways that disadvantage fair multi-tenant scheduling; if
36         relevant, discuss implications for shared-cluster policy and reproducibility in
37         multi-user environments."
38     ],
39     "Ethical Concerns": false,
40     "Soundness": 1,
41     "Presentation": 2,
42     "Contribution": 1,
43     "Overall": 3,
44     "Confidence": 5,
45     "Decision": "Reject"
46 }

```

Listing 4.4 Gemini CLI 自動レビュー結果

```

1 {
2     "Summary": "The paper experimentally evaluates how OpenMP thread affinity impacts both
3     throughput and performance stability for the memory-intensive Himeno benchmark.
4     Across three problem sizes (64\u00d7764\u00d77128, 128\u00d77128\u00d77256, 256\u00d77256
5     \u00d77512) and 30 repeated runs per setting, it compares unbound execution versus
6     explicit binding with `OMP_PROC_BIND=close`. It reports median MFLOPS, coefficient
7     of variation (CV), and a tail-latency ratio (p99/median), concluding that binding
8     improves median performance (up to ~42% on the medium size) and generally reduces
9     variability and tail latency, especially for medium/large problem sizes.",
10    "Strengths": [
11        "Addresses a real and common HPC concern: run-to-run variability and tail behavior,
12        not just mean performance.",
13        "Uses repeated trials (30 runs) and reports variability metrics (CV) and a tail
14        metric (p99/median), which is more informative than single-run MFLOPS.",
15        "Findings are plausible and actionable for practitioners: explicit affinity improves
16        stability and throughput for memory-bound workloads.",
17        "Includes a useful discussion of the small-size anomaly (slightly higher CV under
18        close binding) and proposes plausible mechanisms (turbo/thermal behavior, cache
19        contention).",
20    ],
21    "Weaknesses": [
22        "Insufficient experimental detail to ensure reproducibility or rule out confounders:
23        CPU model/topology (sockets, NUMA nodes), core counts, memory configuration,
24        BIOS settings, and whether SMT/hyperthreading is enabled are not specified.",
25        "Threading configuration is underspecified: number of OpenMP threads, `OMP_PLACES`,
26        whether threads span sockets/NUMA domains, and whether the same placement is
27        used across runs are not reported.",
28        "NUMA and memory placement policy is not controlled/described (first-touch, `numactl
29        `, interleave, etc.), which can dominate Himeno results and interact strongly
30        with binding.",
31        "Measurement protocol details are missing: warm-up runs, whether the benchmark
32        reports steady-state MFLOPS, how p99 is computed from only 30 samples (and
33        whether p99 is interpolated), and whether system background load was controlled
34        .",
35        "The conclusion recommends `OMP_PROC_BIND=close` but does not compare against other
36        common policies (e.g., `spread`, explicit `OMP_PLACES=cores`, vendor runtimes),
37        limiting generality.",
38        "Presentation issues/typos: the affinity string is garbled in multiple places (e.g.,
39        struck-through characters), and Figure 1 is referenced but not actually shown
40        in the provided text."
41    ],
42 }

```



```

17  "Originality": 2,
18  "Quality": 2,
19  "Clarity": 2,
20  "Significance": 3,
21  "Questions": [
22      "What exact hardware was used (CPU model, sockets, cores per socket, NUMA domains,
        memory channels), and were SMT and turbo enabled? Please report `lscpu`/topology
        summary.",
23      "How many OpenMP threads were used for each run? Were `OMP_NUM_THREADS`, `OMP_PLACES`
        , and `OMP_PROC_BIND` the only OpenMP env vars set? Please list all relevant
        env vars (including any vendor/runtime defaults).",
24      "Does `close` keep threads within a single socket/NUMA node in your setup, or can it
        spill across sockets? Please clarify the actual placement (e.g., via `
        OMP_DISPLAY_ENV=TRUE` or runtime affinity reports).",
25      "How was memory placement handled? Was first-touch ensured, and was the benchmark
        initialized in parallel? Did you use `numactl` or rely on default policy?",
26      "How exactly is the tail metric computed with 30 runs? Is p99 the maximum, an
        interpolated quantile, or computed from more samples (e.g., per-iteration timing
        )? Please clarify.",
27      "Were there warm-up runs or cache/TLB priming effects? If so, how were they handled?
        If not, can you report whether the first run differs systematically?",
28      "Can you add comparisons to at least one additional policy (`OMP_PROC_BIND=spread`
        and/or explicit `OMP_PLACES=cores`) to support the claim that `close` is best
        rather than just better than unbound?",
29      "Do the results hold across different thread counts (e.g., half-socket vs full-
        socket) and across multiple nodes/runs at different times of day (to test
        scheduler/noise sensitivity)?"
30  ],
31  "Limitations": [
32      "Single-node, single-partition evaluation limits generality; affinity effects can
        differ across architectures (AMD vs Intel, multi-socket vs single-socket,
        different NUMA distances).",
33      "Only two affinity conditions are tested; the recommendation may not hold versus `
        spread` or explicit place lists.",
34      "Potential confounding from uncontrolled frequency scaling/turbo/thermal throttling
        and NUMA memory placement; without controlling/reporting these, causal
        attribution to affinity alone is weaker.",
35      "Tail metric estimation from 30 samples is statistically fragile; a larger number of
        runs or finer-grained timing samples would yield more reliable tail estimates
        .",
36      "Societal impact is low, but the work could be used to optimize compute performance/
        energy use; an explicit note on energy/power implications (positive and negative
        ) would strengthen the limitations discussion."
37  ],
38  "Ethical Concerns": false,
39  "Soundness": 3,
40  "Presentation": 2,
41  "Contribution": 3,
42  "Overall": 7,
43  "Confidence": 3,
44  "Decision": "Accept"
45  }

```

4.6 分析

4.6.1 システムの自動研究遂行能力

生成された論文から、提案システムが以下を自律的に遂行したことが確認された：

1. **環境情報の自動取得と活用**：Phase 0 において CPU トポロジー（ソケット数、コア数、NUMA ノード構成）、OS、コンパイラバージョン、利用可能なツールを正確に取得し、実験計画に反映した。
2. **体系的な実験計画**：スレッドスケールリング、ランタイムノブスイープ（バインディング、スケジューリング、配置ポリシーの組み合わせ）、アブレーション実験を段階的に計画・実行した。
3. **適切な計測手法の選択**：再現性を確保するため、複数回の反復実行とウォームアップイテレーションを設定し、統計量（中央値、平均、標準偏差、CV、p99/median）を算出した。
4. **多角的な分析**：単一メトリクスに依存せず、中央値スループット、変動係数、テール劣化の 3 つの目的関数を定義し、Pareto 最適性に基づく多目的分析を実行した。
5. **可視化の自動生成**：6 つの図（スレッドスケールリング、散布図、分布図、ヒートマップ、棒グラフ）を自動生成し、分析結果を視覚的に示した。
6. **論文としての体裁**：Abstract, Introduction, Related Work, Method, Experiments, Conclusion の学術論文構造に従い、参考文献の引用を含む論文を自動生成した。

4.6.2 既存エージェントとの比較分析

表 4.2 および表 4.3 に基づき、各条件の結果を分析する。

タスク完遂段階の差異

提案手法、Claude Code、Gemini CLI の 3 手法は、環境構築から結果取得まで全段階を完遂した。一方、Codex は SLURM / 環境上の問題により実験の実行に至らず、生成論文の Results セクションが空となった。この結果は、HPC 環境特有の実行制約（SLURM ジョブスケジューラ、Singularity コンテナ、環境変数の設定等）への対応がエージェントの研究遂行能力に直接影響することを示している。提案手法は Split-Phase 実行モデルにより環境構築（Phase 1）とプログラム実行（Phase 4）を明確に分離し、各フェーズで反復的な失敗診断・修復を行うため、HPC 環境の制約に対して頑健に動作した。

LLM レビュースコアの差異

Overall（総合評価）では、提案手法と Gemini CLI が同点の 7/10 で最高スコアを獲得し、Claude Code が 6/10、Codex が 3/10 であった。判定では提案手法、Claude Code、Gemini CLI が Accept、Codex のみ Reject となった。

項目別に見ると、提案手法は全 7 項目で 3/4 を獲得し、唯一全項目で最高水準を達成した。特に Originality（独創性）と Contribution（貢献）において、提案手法のみが 3/4 を獲得した点は注目に値する。提案手法の生成論文は、中央値スループット・CV・テール比の 3 指標による多目的定式化と Pareto 分析を採用し

ており、単一指標による評価を超えた体系的なアプローチが独創性と貢献の高評価につながったと考えられる。

Claude Code は、Quality, Clarity, Soundness, Presentation で提案手法と同等の 3/4 を獲得したが、Originality (2/4), Significance (2/4), Contribution (2/4) が低い。これは、デフォルト環境での単純な問題サイズ比較にとどまり、制御変数の操作（ピニング、NUMA 制御等）や多目的最適化といった発展的な分析が含まれなかったことに起因する。

Gemini CLI は、Significance (3/4) と Contribution (3/4) で提案手法と同等のスコアを獲得した。アフィニティバインディングによる性能改善という実用的知見が高く評価された一方、Originality (2/4), Quality (2/4), Clarity (2/4), Presentation (2/4) が低い。再現性に必要な実験詳細の不足や図版の表示不具合が、これらの低スコアの要因として指摘されている。

Codex は、実験結果が欠如していたため、Quality (1/4), Soundness (1/4), Contribution (1/4) で最低スコアとなり、Reject と判定された。レビュアーの Confidence が 5/5（最高）であったことは、結果の欠如が明確な不備として認識されたことを示す。

HPC 環境への適合性

以上の結果から、提案システムの HPC 環境への適合性について以下の点が確認された：

- **Split-Phase 実行モデルの有効性**：環境構築・コンパイル・実行を明確に分離し、各フェーズで反復的な修復を行う設計により、Codex が失敗した HPC 環境の制約を克服できた。
- **体系的な実験設計**：提案手法は、スレッドスケールリング、ランタイムノブスイープ、アブレーション実験を段階的に計画・実行し、多目的 Pareto 分析まで到達した。Claude Code, Gemini CLI も実験結果は取得できたが、実験の体系性・多角性では提案手法が優位であった。
- **階層記憶機構による文脈管理**：提案手法の生成論文では、複数のスイープにわたって一貫した分析フレームワーク（CV, p99/median, Pareto 分析）が維持されており、階層記憶機構による文脈の持続的管理が寄与していると考えられる。

第 5 章

関連研究

5.1 LLM を用いた研究自動化

LLM を活用した研究自動化の試みは急速に進展しており、Ji ら [13] のサーベイではこの分野の自動化から自律化への発展が体系的に整理されている。Lu らの AI Scientist[14] は、アイデア生成から論文執筆までの研究プロセスを自動化する先駆的なシステムである。このシステムは、LLM の生成能力を活用して新規性のある研究アイデアを創出し、コード実験を自動実行し、さらに LaTeX 形式の論文を生成する。

Yamada らの AI Scientist v2[2] は、木探索に基づくエージェントアーキテクチャにより研究品質を向上させたシステムである。研究プロセスを「アイデア生成」「実験実行」「論文生成（執筆）」といった段階に分解し、段階ごとに異なるプロンプトや処理を割り当てる点が中核として述べられる。ただし実装上は、木探索や後処理（メトリクス抽出・可視化等）など複数のサブステップが含まれるため、本論文では以降、必要に応じて内部処理を分解して扱う。しかし、このシステムは主にクラウド環境やローカル開発環境を想定しており、HPC 環境特有の制約への対処は考慮されていない。

Google Research の AI Co-Scientist[15] は、Gemini を基盤としたマルチエージェントシステムにより仮説生成と研究提案の支援を行うが、同様に HPC 環境の実行制約は対象外である。Woodruff ら [16] は、Gemini モデルを用いた人間-AI 協働による科学研究の加速事例を報告しており、理論計算機科学・経済学・最適化・物理学の各分野で反復的改良、問題分解、分野横断知識転移などの協働技法が有効であることを示した。この研究は LLM を「厳密な敵対的査読者」や自動検証ループに組み込む応用も提案しているが、実験の自律実行や HPC 環境での運用は対象としていない。

近年、研究プロセスのうち「論文執筆」自体を LLM で支援する統合環境も登場している。例えば OpenAI Prism[17] は、LaTeX を第一級の編集対象として扱い、原稿構造の生成・整形、関連文献の整理、図表や本文の整合性確認などを対話的に支援するワークスペースである。これらの執筆支援環境は、研究サイクルにおける「執筆」フェーズの効率化に寄与する一方で、実験の実行・再現性確保・HPC 資源制約下の運用（依存解決、コンパイル、資源管理、長期的な実験履歴の保持）を包含するものではない。本研究は、研究全体を自律化する枠組み（AI Scientist v2 等）を基盤としつつ、HPC 制約下での実行と反復実験管理を中核に据え、執筆は後処理パイプラインとして統合する点で位置づけが異なる。

Schmidgall らの AgentLab[18] は、複数のベンチマークにわたって LLM エージェントを評価するためのフレームワークを提供する。このフレームワークは、異なるエージェント実装の性能を公平に比較できる環境を構築し、研究コミュニティの発展に貢献している。

5.2 LLM エージェントの記憶機構

LLM エージェントの長期記憶に関する研究は、文脈長の制約を克服するための重要なアプローチである。

Packer らの MemGPT[1] は、オペレーティングシステムの仮想記憶の概念を LLM に適用した画期的な研究である。MemGPT では、LLM の有限の文脈長を「主記憶」に見立て、外部データベースを「二次記憶」として活用する。LLM 自身が記憶の読み書きを制御することで、事実上無制限の記憶空間を実現する。

定義 5.1 (MemGPT 型記憶アーキテクチャ). 記憶システム $\mathcal{M}$ は、以下の 3つの階層から構成される：

- コンテキストウィンドウ C : LLM の入力として直接利用可能な領域（容量制限あり）
- 外部記憶 E : データベースに永続化された情報（容量制限なし）
- 制御関数 $f : C \times E \rightarrow C' : \text{LLM の判断に基づく記憶操作}$

この概念は、本研究における階層記憶機構の理論的基盤となっている。図 5.1 に MemGPT のシステムフローを示す。

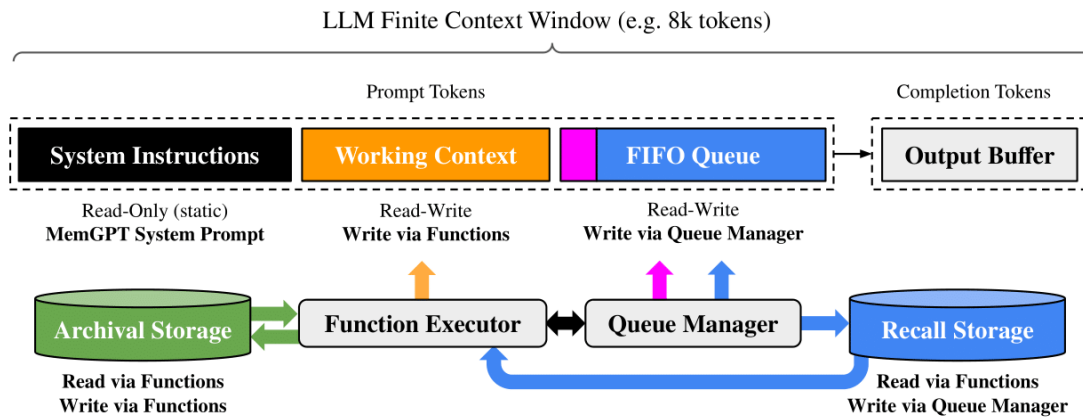


図 5.1 MemGPT のシステムフロー ([1] より引用)。

図 5.1 に示すように、MemGPT は LLM の固定長コンテキストウィンドウの制約を克服するため、メイン・コンテキスト（常時参照）とエクスターナル・コンテキスト（外部記憶）を階層化し、LLM 自身が自律的にデータの移動・検索を行うページング機構を実現している。本研究の階層記憶機構（Core/Recall/Archival）はこの設計思想を継承し、研究実験の文脈に適応させたものである。

Zhong らの MemoryBank[19] は、記憶の重要度と時間減衰を考慮した選択的記憶機構を提案している。人間の記憶と同様に、古い情報は徐々に忘却され、重要な情報は長期間保持される仕組みを実現している。

Park らの Generative Agents[20] は、仮想空間における複数エージェントのシミュレーションにおいて、記憶・反省・計画の機構を統合した。各エージェントは経験を記憶ストリームに蓄積し、定期的に反省を行って高次の抽象概念を形成する。この研究は、長期的な一貫性を持つエージェント行動の実現可能性を示した。

近年では、Xu らの A-MEM[21] が Zettelkasten 法の構造化原理とエージェント駆動の柔軟性を融合し、記憶の動的な自己組織化を実現している。Choudhary らの Mem0[22] は、スケーラブルな長期記憶アーキ

テクチャを提案し、会話からの情報抽出・統合・検索を動的に行うことで、従来手法に比べてレイテンシ 91% 削減・トークンコスト 90% 以上削減を達成している。

これらの記憶機構において Archival Memory からの関連情報の検索は、Lewis ら [9] が提案した Retrieval-Augmented Generation (RAG) の枠組みに基づいている。RAG は外部知識を検索して LLM の生成に統合するアプローチであり [23]、本研究の Archival Memory における FTS5 全文検索もこの概念に基づく実装である。

5.3 LLM ソフトウェアエージェント

LLM を用いたソフトウェア開発の自動化も急速に進展している。Jimenez ら [24] の SWE-bench は、GitHub の実問題を LLM エージェントに解決させるベンチマークであり、LLM ソフトウェアエージェントの評価基盤として広く利用されている。Wang ら [25] の OpenHands は、コード記述・コマンド実行・Web 閲覧を統合したオープンソースの AI ソフトウェア開発プラットフォームであり、Docker サンドボックスによる安全な実行環境を提供する。

これらのソフトウェアエージェントは汎用的な開発タスクを対象としているが、HPC 環境特有のコンテナ実行 (Singularity[5])、ジョブスケジューラ連携、並列プログラミングモデルへの対応は考慮されていない。本研究はこの空白を埋めるものである。

5.4 コード生成における探索アルゴリズム

LLM によるコード生成の精度向上のため、様々な探索アルゴリズムが提案されている。

Zhang らの Planning with LLM[26] は、木探索アルゴリズムをコード生成に適用した。各ノードを部分的なプログラム状態として表現し、LLM を用いて次の状態を生成・評価することで、複雑なプログラミング問題を解決する。

Chen らの CodeT[27] は、複数のコード候補を生成し、テストケースの通過率に基づいてランキングする手法を提案した。この「生成と選択」のパラダイムは、単一のコード生成よりも高い精度を実現する。

AIDE[3] は、LLM エージェントによるコード空間の探索を木構造で表現し、Best-First 探索により効率的に最適解を発見するフレームワークである。各ノードはコード実装を表し、評価関数に基づいて次に展開するノードを選択する。本研究の木探索コンポーネントは、AIDE の設計を基盤としている。図 5.2 に AIDE のアルゴリズム概要を示す。

また、Zhang ら [28] の RethinkMCTS は、Monte Carlo Tree Search (MCTS) に rethink 機構を導入し、コード実行からの詳細なフィードバックにより誤った推論を直接修正するアプローチを提案した。トークンレベル、行レベル、コードレベル、思考レベルの 4 段階の探索粒度を比較し、思考レベルの探索が最も効果的であることを示している。

5.4.1 HPC 向けコード生成

LLM による HPC コード生成は新興の研究分野であり、Chen ら [29] は LLM と HPC 研究の交差領域における課題と展望を整理している。Nichols ら [30] の HPC-Coder は、LLM が並列コード (OpenMP,

MPI, CUDA 等) を生成する能力を体系的に評価し, HPC ドメイン特化データでの学習が並列コード生成の精度を大幅に向上させることを示した. その後続研究である Ding ら [31] の HPC-Coder-V2 は, 低リソース並列言語 (Fortran, Chapel, Julia 等) への LLM 適用を調査し, HPC-INSTRUCT データセットを構築して命令チューニングによる性能改善を実現している. Cummins ら [32] の Meta LLM Compiler は, コンパイラ最適化に特化した基盤モデルであり, コードサイズ最適化やデバイスマッピングなどのコンパイラタスクにおいて LLM の有効性を示した.

Hayashi らの VibeCodeHPC[33] は, マルチエージェント LLM による HPC コード生成と自動チューニングを統合したシステムである. Vibe Coding パラダイム (自然言語による高速プロトタイピング) を採用し, コード生成エージェント, 性能評価エージェント, 最適化エージェントが協調して動作する. 各エージェントは反復的なプロンプト改良により, 機能的正確性だけでなくピーク性能を目指した HPC コードを生成する. 並列プログラミングモデル (MPI, OpenMP, CUDA, OpenACC) やアーキテクチャ固有の最適化を考慮した自動チューニング機構を持つ点が特徴である.

本研究と VibeCodeHPC は, いずれも HPC 環境における LLM の活用を目指しているが, 対象とする問題領域が異なる. VibeCodeHPC は既存の数値計算問題に対する高性能コード実装の自動生成に焦点を当てているのに対し, 本研究は科学研究全体 (問題定式化, 実験設計, 実験実行, 結果分析, 論文執筆) の自動化を目指している. 本研究のシステムは, コード生成を内部コンポーネントの一部として含むが, より上位の研究サイクル全体を管理する枠組みを提供する.

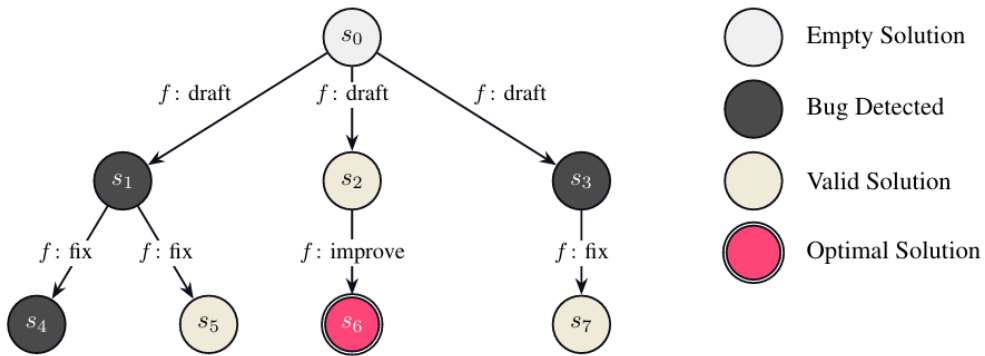


図 5.2 AIDE のアルゴリズム概要 ([3] より引用). LLM エージェントがコード実装の探索を木構造として行い, Best-First 戦略により評価値の高いノードを優先的に展開する. 各ノードはコード実装を表し, ドラフト生成・デバッグ・改良のサイクルで探索木を成長させる. 本研究の Best-First 木探索コンポーネントはこのアルゴリズムを基盤とし, Split-Phase 実行モデルおよび階層記憶機構と統合して使用する.

5.5 本研究の位置づけ

表 5.1 に, 関連研究と本研究の比較を示す. 本研究の独自性は, HPC 環境という制約下での自律研究システムの実現に焦点を当て, Split-Phase 実行モデル, 階層記憶機構, 木探索アルゴリズムを統合的に設計した点にある.

表 5.1 に示すように, OpenHands[25] は Docker サンドボックスによる自律的依存解決を備えるが, HPC

表 5.1 関連研究との比較

	HPC 対応	記憶機構	木探索	自律的依存解決
AI Scientist	-	-	-	-
AI Scientist v2	-	△	○	-
AgentLab	-	△	-	-
MemGPT	-	○	-	-
Generative Agents	-	○	-	-
AIDE	-	-	○	-
OpenHands	-	-	-	○
VibeCodeHPC	○	-	-	-
HPC-Coder/V2	○	-	-	-
本研究	○	○	○	○

環境や研究自動化は対象外である．VibeCodeHPC[33] および HPC-Coder/V2[30, 31] は HPC 向けコード生成に特化しているが，記憶機構や木探索による研究プロセス全体の自動化は含まない．なお，Prism[17] のような執筆支援環境は，実験の実行や再現性確保を対象としておらず，表 5.1 の比較軸（HPC 対応，記憶機構，木探索，自律的依存解決）とは直交する位置づけである．

第 6 章

おわりに

6.1 まとめ

本研究では、HPC 環境における自律的研究システムの設計と実装について述べた。提案システムは以下の主要コンポーネントから構成される：

1. **Split-Phase 実行モデル**：研究プロセスを計画・環境構築・コーディング・コンパイル・実行の 5 フェーズに分割し、各フェーズで適切な抽象度の問題設定を LLM に与える。これにより、複雑なタスクを管理可能な部分問題に分解し、失敗時の診断と修復を容易にする。
2. **階層記憶機構**：コア記憶・リコール記憶・アーカイブ記憶の 3 層構造により、有限の文脈長の中で長期的な実験履歴を効率的に管理する。LLM による記憶操作（<memory_update>ブロック）と Two-Phase パターンにより、適応的な情報管理を実現する。メモリプレッシャー管理により、記憶使用量に応じた自動統合・圧縮を行う。
3. **Best-First 木探索**：評価関数に基づく最良優先探索により、広大な実装空間を効率的に探索する。4 つのステージ（初期実装・ベースライン調整・創造的改良・アブレーション）を経て、研究論文に必要な実験結果を体系的に収集する。マルチシード評価により結果の堅牢性を検証する。
4. **ポスト処理パイプライン**：VLM 分析によるプロット品質評価、メトリクス抽出、最終メモリ生成、論文自動生成、LLM レビューを統合し、実験から論文執筆までを自動化する。
5. **リソースファイル機構**：ローカルデータ、GitHub リポジトリ、Hugging Face モデル/データセットを宣言的に管理し、フェーズごとの注入ルールに基づいてプロンプトとコンテナに注入する。

本研究の貢献は以下の通りである：

- HPC 環境における自律研究システムの設計原理を体系的に整理し、提示した
- Split-Phase 実行モデル、階層記憶機構、Best-First 木探索のアルゴリズムを形式的に記述した
- Two-Phase パターン、マルチシード評価、VLM 分析パイプライン、メモリプレッシャー管理などの実装詳細を文書化した
- リソースファイル機構（Hugging Face サポート含む）とペルソナシステムによる拡張性を実現した
- Singularity コンテナによる実行環境の分離と、反復的依存解決機構を実装した
- 各設計決定の理由（なぜその設計を選んだか）を明確に説明した

本研究が、HPC 環境における研究自動化の発展に寄与することを期待する。

6.2 今後の課題

今後の課題として、以下の 3 点が挙げられる。

1. **実証的な評価実験の実施**：本研究では設計と実装の記述に重点を置いたが、生成された論文の品質やシステムの効率性を定量的に評価する実験が必要である。具体的には、複数の研究題材に対してシステムを適用し、生成論文の品質スコア、探索の収束速度、計算資源の利用効率などを測定・分析することが求められる。
2. **ジョブスケジューラとの統合**：現在のシステムは Slurm ジョブとして手動投入する形態をとっているが、探索の進行状況に応じた動的なリソース割り当てや、ジョブの自動再投入機構を実装することで、HPC クラスタの利用効率をさらに向上させることが可能である。
3. **セキュリティ機構の強化と学会固有テンプレートの導入**：LLM が生成するコードの安全性検証機構を強化するとともに、特定の論文形式 (NeurIPS, ICML 等) に準拠したレビュー基準・テンプレートの導入が必要である。現状のレビュー機構は汎用的な論文品質評価にとどまっており、学会固有の採択基準に基づく評価の実装が今後の課題である。

6.3 今後の展開

6.3.1 評価関数の学習

過去の研究成果から評価関数を学習することで、より適切な探索誘導が可能になる可能性がある。

6.3.2 並列探索の最適化

現在の実装では単純な並列化を行っているが、探索状態を共有した協調的な並列探索により、効率を向上できる可能性がある。

6.3.3 記憶機構の有効性検証

本研究では、階層記憶機構を「Split-Phase 導入により増大する文脈に対して、重要情報の欠落を抑えるための機構」として位置づけた。本評価では既存の LLM コーディングエージェントとの比較を実施したが、記憶機構単体の寄与を分離して評価する実験（記憶機構あり/なしの条件で論文生成およびレビューを比較し、整合性、根拠の引用、失敗の再発率、最終スコア等に与える影響を検証する実験）は今後の課題として残されている。

6.3.4 他ドメインへの適用

本システムの設計原理 (Split-Phase 実行, 階層記憶, 木探索) は、HPC 環境での研究以外にも適用可能である。ソフトウェア開発, データ分析, 科学シミュレーションなど, 複雑なタスクの自動化に応用できる

可能性がある。

6.4 提案手法の限界

6.4.1 LLM の能力への依存

本システムの性能は、基盤となる LLM の能力に強く依存する。LLM が適切なコードを生成できない場合、探索空間に有望なノードが存在しない可能性がある。

6.4.2 評価関数の設計

適切な評価関数の設計は困難である。不適切な評価関数は、探索を誤った方向に導く可能性がある。特に、研究タスクでは「良い」実装の定義自体が曖昧なことが多い。

6.4.3 計算コスト

木探索と LLM 呼び出しの組み合わせは、計算コストが高い。特に大規模な探索では、LLM 呼び出し回数が増大し、コストと時間が問題となる。

6.5 設計上のトレードオフ

6.5.1 フェーズ数のトレードオフ

Split-Phase 実行モデルにおいて、フェーズ数は重要な設計パラメータである。

- **フェーズを増やす利点**：各フェーズの責務が明確になり、デバッグが容易になる
- **フェーズを増やす欠点**：フェーズ間の状態引き継ぎにオーバーヘッドが生じる

本システムでは5 フェーズを採用したが、これは HPC 環境での研究プロセスに基づく経験的な選択である。他の環境や用途では、異なるフェーズ分割が適切かもしれない。

6.5.2 記憶容量のトレードオフ

階層記憶機構において、各層の容量設定もトレードオフを伴う。

- **コア記憶を大きくする**：常に参照される情報が増えるが、文脈の他の部分が圧迫される
- **リコール記憶を大きくする**：より長い履歴を保持できるが、古い情報のノイズが増える
- **アーカイブ検索結果を増やす**：より多くの過去知見を参照できるが、関連性の低い情報も混入しやすい

6.5.3 探索幅と深さのトレードオフ

木探索において、限られた計算資源の下で探索幅（各ノードの子数）と探索深さ（改良の反復回数）のバランスを取る必要がある。

- **幅を優先**：多様な実装を試せるが、各実装の洗練度が低い
- **深さを優先**：少数の実装を深く改良できるが、局所最適に陥るリスクがある

謝辞

本研究の遂行にあたり，指導教員である片桐孝洋教授、星野哲也准教授、棕木大地助教授には多大なるご指導とご助言を賜りました．ここに深く感謝申し上げます．

また，本研究で使用した Himeno Benchmark に関して貴重な知見を提供していただいた横田理央先生に心より御礼申し上げます．

本研究は理化学研究所計算科学研究センター（R-CCS）の計算資源を利用して実施されました．この場を借りて感謝申し上げます．

Data Availability

本研究のソースコードは以下のリポジトリで公開されている。本論文の実験はバージョン v1.0.0 を用いて実施した。再現にはこのタグを使用されたい。

- <https://github.com/kotama7/HPC-AutoResearch> (タグ：v1.0.0)

参考文献

- [1] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [2] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025.
- [3] Mingfei Peng, Yali Wang, Haoran Duan, Long Chen, Xuetao Pan, Zhilong Zhang, and Yu Qiao. Aide: An automatic data engine for object detection in autonomous driving. *arXiv preprint arXiv:2403.17373*, 2024. Framework for code space exploration with tree-based best-first search.
- [4] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.
- [5] Gregory M. Kurtzer, Vanessa Sochat, and Michael W. Bauer. Singularity: Scientific containers for mobility of compute. *PLOS ONE*, 12(5):e0177459, 2017.
- [6] Kyle Lo, Lucy Lu Wang, Mark Neumann, Rodney Kinney, and Daniel S. Weld. S2orc: The semantic scholar open research corpus. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4969–4983, 2020.
- [7] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.
- [8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023.
- [9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [10] Ryutaro Himeno. Himeno benchmark: A comprehensive performance evaluation tool for high-performance computers. *RIKEN Technical Report*, 2004.
- [11] Katsumi Itakura, Atsuya Uno, and Mitsuo Yokokawa. Memory access optimization on multicore

- processors using the himeno benchmark. In *2011 IEEE International Conference on Cluster Computing*, pages 305–313. IEEE, 2011.
- [12] Rio Yokota, Kengo Nakajima, and Takeshi Iwashita. Analysis of cache performance for the himeno benchmark on modern hpc systems. *Parallel Computing*, 52:47–63, 2016.
 - [13] Hanjia Ji, Songlin Yang, Yuwei Xie, Yuxin Zhou, Junyu Fang, and SC Han. From automation to autonomy: A survey on large language models in scientific discovery. *arXiv preprint arXiv:2505.13259*, 2025.
 - [14] Chris Lu, Sebastian Lange, Cong Jiang, and Jeff Clune. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
 - [15] Juanma Gottweis, Wei Wey, Samuel Ritter, Ali Elqursh, et al. Towards an ai co-scientist. *arXiv preprint arXiv:2502.18864*, 2025.
 - [16] David P. Woodruff, Vincent Cohen-Addad, Lalit Jain, Jieming Mao, Song Zuo, MohammadHossein Bateni, Simina Branzei, Michael P. Brenner, Lin Chen, Ying Feng, Lance Fortnow, Gang Fu, Ziyi Guan, Zahra Hadizadeh, Mohammad T. Hajiaghayi, et al. Accelerating scientific research with gemini: Case studies and common techniques. *arXiv preprint arXiv:2602.03837*, 2026.
 - [17] OpenAI. Introducing prism, January 2026.
 - [18] Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omid Shahyeghan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Russ Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. The browsergym ecosystem for web agent research. *Transactions on Machine Learning Research*, 2025. Peer-reviewed TMLR article; describes BrowserGym ecosystem and mentions AgentLab as complementary.
 - [19] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. *arXiv preprint arXiv:2305.10250*, 2024.
 - [20] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST 2023)*, 2023.
 - [21] Wujiang Xu, Zujie Liang, Kai Mei, Hang Li, and Xi Chen. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.
 - [22] Deshraj Choudhary, Prateek Khade, Sriram Keshav, and Taranjeet Singh Shukla. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
 - [23] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2024.
 - [24] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *Proceedings*

of the *Twelfth International Conference on Learning Representations (ICLR)*, 2024.

- [25] Xingyao Wang, Boxuan Hooker, Niklas Nair, Daniel Fried, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- [26] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B. Tenenbaum, and Chuang Gan. Planning with large language models for code generation. *arXiv preprint arXiv:2303.05510*, 2023.
- [27] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. Codet: Code generation with generated tests. *arXiv preprint arXiv:2207.10397*, 2023.
- [28] Qingyao Zhang, Wei Ye, Shikun Zhang, and Min Zhang. Rethinkmcts: Refining erroneous thoughts in monte carlo tree search for code generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.
- [29] Le Chen, Matei Zaharia, and James Zou. The landscape and challenges of hpc research and llms. *arXiv preprint arXiv:2402.02018*, 2024.
- [30] Daniel Nichols, Aniruddha Marathe, Harshitha Menon, Todd Metzger, Aidan Munley, Jiaqi Tao, Evangelos Georganas, Aniket Shet, Jiajia Shen, and Anand Venkat. Can large language models write parallel code? *arXiv preprint arXiv:2401.12554*, 2024.
- [31] Aman Ding, Jiaqi Tao, Daniel Nichols, Aidan Munley, Harshitha Menon, Aniket Shet, Jiajia Shen, and Anand Venkat. Hpc-coder-v2: Studying code llms across low-resource parallel languages. *arXiv preprint arXiv:2412.15178*, 2024.
- [32] Chris Cummins, Volker Seez, Dejan Grber, Hugh Leather, Toby Murray, Ben Cohn, and Gennady Pekhimenko. Meta large language model compiler: Foundation models of compiler optimization. *arXiv preprint arXiv:2407.02524*, 2024.
- [33] Shun-ichiro Hayashi, Koki Morita, Daichi Mukunoki, Tetsuya Hoshino, and Takahiro Katagiri. Vibecodehpc: An agent-based iterative prompting auto-tuner for hpc code generation using llms. *arXiv preprint arXiv:2510.00031*, 2024.
- [34] Tetsuya Sakurai, Akihiko Tsuda, Rio Yokota, and Ryutaro Himeno. Performance characteristics of the himeno benchmark on various hpc architectures. *International Journal of High Performance Computing Applications*, 28(3):341–352, 2014.
- [35] Anton Mukosey and Ilya Afanasyev. Himeno benchmark evaluation on gpu-accelerated systems. In *2018 Ivannikov ISPRAS Open Conference (ISPRAS)*, pages 77–82. IEEE, 2018.
- [36] Takahiro Katagiri, Satoshi Ohshima, and Masahiro Matsushita. Auto-tuning for hpc applications using himeno benchmark as a performance model. *International Journal of Networking and Computing*, 10(2):192–210, 2020.

付録 A

実験設定ファイル

本研究の実験で使⽤した設定ファイル (bfts_config.yaml) の全文を以下に示す。この設定ファイルは、Split-Phase 実行モデル、階層記憶機構、Best-First 木探索、およびポスト処理パイプラインの全パラメータを定義する。

Listing A.1 実験設定ファイル (bfts_config.yaml)

```
1 # タスクデータのディレクトリパス
2 data_dir: "data"
3
4 # ログファイルの出力先ディレクトリ
5 log_dir: logs
6
7 # ワークスペースディレクトリ (実験作業領域)
8 workspace_dir: workspaces
9
10 # データをワークスペースにコピーするか (Falseの場合はシンボリックリンク)
11 # エージェントが元データを誤って変更することを防ぐため、コピーを推奨
12 copy_data: True
13
14 # 実験名 (指定しない場合はランダムな名前が生成される)
15 exp_name: run
16
17 # コード実行環境の設定
18 exec:
19   # Singularity コンテナイメージのパス
20   singularity_image: /home/users/takanori.kotama/workplace/docker/ai-scientist-worker-nv.sif
21   # GPU使用の有効化
22   use_gpu: true
23   # コンテナ内のワークスペースマウントポイント
24   workspace_mount: /workspace
25   # Phase 1の最大ステップ数
26   phase1_max_steps: 100
27   # プロンプトのログ記録を有効化
28   log_prompts: true
29   # 各コード実行のタイムアウト時間 (秒)
30   timeout: 10800 # 3時間
31
32
33 # メモリ・記憶管理の設定
34 memory:
```

```

35 # メモリ機能の有効化 (true: 有効, false: 無効)
36 enabled: true
37 # コアメモリの最大文字数
38 core_max_chars: 2000
39 # 想起する最大イベント数
40 recall_max_events: 5
41 # 検索時に取得する上位k件
42 retrieval_k: 4
43 # 全文検索 (FTS) の使用 (auto: 自動判定)
44 use_fts: auto
45 # 最終メモリ機能を有効化
46 final_memory_enabled: true
47 # 最終メモリのMarkdownファイル名
48 final_memory_filename_md: final_memory_for_paper.md
49 # 機密情報を編集 (マスク)
50 redact_secrets: true
51 # メモリ予算の文字数上限
52 memory_budget_chars: 24000
53 # メモリログを有効化
54 memory_log_enabled: true
55 # メモリログの最大文字数
56 memory_log_max_chars: 1000
57 # LLMベースの圧縮を使用
58 use_llm_compression: true
59 # 圧縮に使用するLLMモデル
60 compression_model: gpt-5.2
61 # セクション生成モード: memory_summary | idea_then_memory
62 paper_section_mode: idea_then_memory
63 # 生成するセクション数 (idea_then_memory時)
64 paper_section_count: 12
65 # 圧縮の最大試行回数
66 max_compression_iterations: 5
67 # セクション別の文字数予算 (圧縮時の上限)
68 # 注: 入力サイズ平均72,000文字に対し、圧縮率5-10倍を想定
69 datasets_tested_budget_chars: 8000
70 metrics_extraction_budget_chars: 12000
71 plotting_code_budget_chars: 8000
72 plot_selection_budget_chars: 8000
73 vlm_analysis_budget_chars: 12000
74 node_summary_budget_chars: 8000
75 parse_metrics_budget_chars: 12000
76 # アーカイブメモリ取り出し時の各エントリの文字数上限
77 archival_snippet_budget_chars: 12000
78 # 最終結果取得時の文字数上限
79 results_budget_chars: 12000
80
81 # Writeup用メモリ制限 (final_writeup_memory.json生成時)
82 # no_budget_limit=Falseの場合に適用される
83 writeup_recall_limit: 10 # recall_memoryの最大件数
84 writeup_archival_limit: 10 # archival_memoryの最大件数
85 writeup_core_value_max_chars: 5000 # core_memoryのvalue最大文字数
86 writeup_recall_text_max_chars: 5000 # recall_memoryのtext最大文字数
87 writeup_archival_text_max_chars: 5000 # archival_memoryのtext最大文字数
88
89 # Memory Pressure Management (MemGPT-style)
90 # メモリ圧力管理 (MemGPTスタイル)

```

```

91 auto_consolidate: true # 自動メモリ統合を有効化
92 consolidation_trigger: high # 統合をトリガーする圧力レベル (medium/high/
    critical)
93 recall_consolidation_threshold: 1.5 # recall_max_events * threshold を超えると統合
94
95 max_memory_read_rounds: 5 # メモリ読み取りの最大ラウンド数
96
97 # メモリ圧力の閾値設定
98 pressure_thresholds:
99     medium: 0.7 # 70%使用で中程度の圧力
100     high: 0.85 # 85%使用で高い圧力
101     critical: 0.95 # 95%使用で危機的な圧力
102
103 # レポート生成を有効化
104 generate_report: True
105
106 # 最終レポート生成のLLM設定
107 report:
108     # 使用するLLMモデル
109     model: gpt-5.2
110     # 温度パラメータ (生成の多様性: 0.0-2.0)
111     temp: 1.0
112
113 # 実験の設定
114 experiment:
115     # 合成データセットの数
116     num_syn_datasets: 1
117     # データセットのソース (local, huggingface, auto)
118     # auto: LLMが自動判定
119     dataset_source: auto
120
121 # エージェントのハイパーパラメータ
122 agent:
123     # エージェントのタイプ (並列処理)
124     type: parallel
125     # 並列ワーカー数
126     num_workers: 4
127     # 各ステージの最大反復回数
128     stages:
129         stage1_max_iters: 40
130         stage2_max_iters: 40
131         stage3_max_iters: 40
132         stage4_max_iters: 40
133     # 改善反復の実行回数 (ステージ別max_itersが未指定の場合に使用)
134     steps: 5
135     # マルチシード評価の設定
136     multi_seed_eval:
137         # 評価に使用するシード数 (num_workers < 3の場合はnum_workersと同じ、それ以外は3)
138         num_seeds: 3
139     # プロンプトに注入される役割の説明
140     role_description: "HPC Researcher"
141
142 # コーディング用LLM設定
143 code:
144     model: gpt-5.2
145     temp: 1.0

```

```
146
147 # プログラム出力・トレースバック評価用LLM設定
148 feedback:
149     model: gpt-5.2
150     temp: 0.5
151
152 # ビジョン言語モデルのフィードバック設定
153 vlm_feedback:
154     model: gpt-5.2
155     temp: 0.5
156
157 # 探索の設定
158 search:
159     # デバッグの最大深度
160     max_debug_depth: 3
161     # デバッグを実行する確率
162     debug_prob: 0.5
163     # ドラフト数
164     num_drafts: 3
165
166 # 結果サマリー生成用LLM設定
167 summary:
168     model: gpt-5.2
169     temp: 0.3
170
171 # 最適ノード選択用LLM設定
172 select_node:
173     model: gpt-5.2
174     temp: 0.3
```

付録 B

実験題材：Himeno Benchmark 性能解析

本研究のエージェント比較実験で使⽤したアイデア記述ファイル (thesis_thema.md) の全文を以下に示す。このファイルは Himeno ベンチマーク [10] の性能変動要因の分析と最適化手法の実装・評価を記述したものであり、システムへの入力として使⽤される。Himeno Benchmark は HPC 環境における性能評価の標準的なツールとして、アーキテクチャ特性評価 [34]、メモリアクセス最適化 [11]、GPU 高速化 [35]、自動チューニング [36] など多様な研究で使⽤されている。

Listing B.1 実験題材ファイル (thesis_thema.md)

```
1 # Supplementary Information: Himeno Benchmark Performance Analysis on Modern HPC Systems
2
3 ## Keywords
4 NUMA topology, cache hierarchy, thread pinning, memory bandwidth, TLB, OpenMP environment
   variables, CPU frequency scaling, turbo boost, performance metrics, statistical sampling
   , loop tiling, SIMD vectorization, system noise, OS jitter
5
6 ## TL;DR
7 This supplement provides technical background for understanding Himeno benchmark instability
   . Key factors include NUMA effects, cache residency differences across problem sizes,
   and runtime variability from CPU frequency scaling and thread migration. For the three
   problem sizes (**64×64×128** ~50MB, **128×128×256** ~400MB, **256×256×512** ~3.2GB),
   memory footprint determines whether performance is cache-bound or memory-bound.
   Recommended mitigations include explicit thread affinity (`OMP_PROC_BIND=close`), fixed
   CPU frequency (`cpupower frequency-set -g performance`), and NUMA-local allocation (`
   numactl --localalloc`). Optimization strategies target three metrics separately: cache
   blocking improves **median**, thread pinning reduces **CV**, and OS noise isolation
   suppresses **p99/median** tail degradation.
8
9 ## Abstract
10 This supplementary document provides detailed technical context for the main analysis of
   Himeno benchmark performance stability.
11
12 The Himeno benchmark, developed at RIKEN, solves a 3D Poisson equation using a 19-point
   Jacobi stencil. Memory footprints vary significantly across problem sizes: **64×64×128**
   (~50MB) may fit in L3 cache, **128×128×256** (~400MB) exceeds typical cache sizes, and
   **256×256×512** (~3.2GB) stresses main memory bandwidth. These differences cause each
   size to exhibit distinct sensitivity profiles to system conditions.
13
14 Performance variability arises from multiple sources. NUMA effects include first-touch page
   placement and remote memory access penalties (1.5 – 3× latency). Runtime factors include
```

CPU turbo boost (5 - 15% CV impact), thread migration (3 - 10%), OS interrupts (1 - 5%), and thermal throttling (5 - 20%). These are not independent—interactions between factors can amplify instability.

Experimental methodology requires statistical rigor: minimum 30 runs per configuration, warm-up run exclusion, and reporting with/without outliers. Three metrics capture distribution behavior: **median MFLOPS** (central tendency), **CV** (relative dispersion), and **p99/median** (tail degradation ratio).

Optimization strategies map to specific metrics. For **median improvement**: cache blocking, loop reordering, SIMD vectorization, and prefetching. For **CV reduction**: explicit thread pinning via `OMP_PROC_BIND`` and `OMP_PLACES``, NUMA-aware allocation, and fixed CPU frequency. For **p99/median suppression**: minimizing synchronization, using deterministic allocators, CPU isolation via cgroups, and IRQ affinity tuning.

Recommended system settings include: turbo boost disabled, hyper-threading disabled, automatic NUMA balancing disabled, and transparent huge pages enabled. These configurations reduce non-determinism while maintaining representative performance levels. Platform topology (single vs. multi-socket, memory channels, interconnect bandwidth) further modulates which factors dominate for each problem size.

付録 C

アイデア創出機構の生成例

本研究の評価実験において、アイデア創出機構によって生成された研究提案の例を以下に示す。このファイル (idea.md) は、LLM によって自動生成された研究アイデアであり、研究タイトル、仮説、関連研究、要旨、実験計画、リスク要因などの構造化された情報を含む。本システムでは、このようなアイデアファイルを入力として受け取り、Split-Phase 実行モデルと階層記憶機構を用いて、アイデアの実装から実験実行、論文執筆までの一連のプロセスを自律的に遂行する。

本例では、Himeno Benchmark を題材として、再現性を重視した自動チューニング手法（性能の中央値だけでなく、変動係数やテール性能を最適化する手法）が提案されている。このアイデアは、NUMA 配置、スレッド移動、動的周波数スケーリングなどがステンシル計算の実行時変動に与える影響を分析し、速度と安定性のトレードオフを明示的に扱う多目的最適化を提案している。

Listing C.1 アイデア創出機構の生成例 (idea.md)

```
1  ## Name
2
3  stability_oriented_autotuning_v2
4
5  ## Title
6
7  Autotuning for Reproducibility: Jointly Optimizing Median Speed and Variability (CV, p99/
   median) for NUMA Stencil Codes
8
9  ## Short Hypothesis
10
11 For stencil benchmarks (Himeno as a controlled proxy), the configuration that maximizes
   median MFLOPS is often different from the configuration that minimizes run-to-run
   variability and tail slowdowns, because median is dominated by locality/SIMD while
   variability and p99 are dominated by runtime placement and system noise (NUMA first-
   touch, migration, frequency scaling). Therefore, treating performance as a distribution
   (median, CV, p99/median) and autotuning across a small, practical set of code variants
   and runtime knobs will produce Pareto-optimal configurations that (i) reduce CV and p99/
   median substantially and (ii) often preserve or even improve median by removing
   pathological placements. This question is best answered in stencil codes because their
   memory access is simple enough to attribute causes via counters, yet large enough to
   trigger NUMA/cache/bandwidth regime changes across problem sizes; simpler
   microbenchmarks (STREAM) lack the synchronization and stencil reuse patterns that
   interact with affinity and tiling.
```

12

13 ## Related Work

14
15 Existing stencil optimization and autotuning work (tiling, vectorization, unrolling, prefetching) largely targets average throughput and reports single-number speedups. Separately, HPC systems work studies OS noise/jitter, thread pinning, and NUMA placement, typically as best practices or isolated sensitivity studies. Tail metrics (p99) are standard in datacenter latency work but rarely made first-class in HPC kernel tuning. This proposal is distinct in (1) making distributional objectives explicit (median + variability + tail), (2) jointly searching code and runtime knobs rather than treating system settings as fixed, and (3) producing an actionable “configuration policy” that can pick a stable-vs-fast point on the Pareto front with a short probe run. It is not a trivial extension of autotuning because the objective, measurement protocol, and output artifact (Pareto front + selection policy) change fundamentally.

16 ## Abstract

17
18
19 Modern HPC nodes exhibit stochastic performance due to NUMA placement, thread migration, dynamic frequency scaling, and OS jitter. For memory-sensitive stencil solvers, these effects can dominate run-to-run reproducibility even when median performance appears strong. We propose a stability-oriented autotuning methodology that treats performance as a distribution rather than a single scalar. Using the Himeno benchmark across three regimes ($\approx 50\text{MB}$, $\approx 400\text{MB}$, $\approx 3.2\text{GB}$ footprints), we jointly tune a small set of compile-time transformations (loop order, 3D tiling, SIMD pragmas, unrolling, prefetch distance) and runtime/system knobs (OpenMP affinity/places, scheduling, NUMA allocation policy) to optimize three metrics: median MFLOPS, coefficient of variation (CV), and tail degradation (p99/median). We use replicated measurements (≥ 30 runs/config) with robust statistics to build Pareto fronts that expose speed-stability trade-offs. Finally, we train a lightweight selector that uses topology and a short probe run’s counters (LLC misses, remote NUMA accesses, TLB misses, context switches) to choose a configuration matching a user’s preference (fastest vs most stable). The outcome is a practical, open-source toolkit and a set of empirical findings showing which knobs control median vs variability vs tails in each memory regime, enabling more reproducible benchmarking and more reliable stencil performance on shared systems.

20 ## Experiments

- 21
22
23 - Distribution-first baseline: For each problem size ($64 \times 64 \times 128$, $128 \times 128 \times 256$, $256 \times 256 \times 512$) and thread count, run 50 repetitions under default settings. Report median, CV, p99/median. Collect perf counters: LLC misses, memory bandwidth, remote NUMA accesses (where available), dTLB load misses, context switches/interrupts.
- 24 - Runtime knob sweep (no code changes): Evaluate combinations of `OMP_PROC_BIND={false,close,spread}`, `OMP_PLACES={cores,sockets}`, `OMP_SCHEDULE={static,guided}`, and NUMA allocation via `numactl={default,localalloc,interleave}`. Quantify effect sizes on (median, CV, p99/median) per problem size; identify which knobs mostly reduce CV vs improve median.
- 25 - Code variant sweep (runtime fixed): Implement 6-12 code variants: (a) loop reordering for stride-1, (b) 3D tiling with a small grid of tile sizes, (c) SIMD via OpenMP `simd` or `intrinsics`, (d) unroll factors, (e) optional software prefetch. Evaluate under a fixed “stable” runtime setting to separate algorithmic locality gains from system noise effects.
- 26 - Joint multi-objective autotuning: Search over the combined space (runtime knobs + code variants) using a simple multi-objective strategy (e.g., NSGA-II or random search with Pareto filtering). Budget: $\sim 150 - 250$ configurations per size, 30 runs each. Output Pareto front; compare against (i) median-only tuning and (ii) best-practice manual settings (pinning + `localalloc`).
- 27 - Stability policy learning: Train a small model (e.g., gradient-boosted trees) to map features from a 1-2 second probe run (counters + topology + problem size) to a Pareto-

```

front configuration given a user objective (weights over median/CV/p99). Evaluate
generalization across nodes and across mild background load. Metrics: regret vs oracle (
distance to Pareto front), achieved median/CV/p99.
28 - Ablation and causal attribution: Remove one knob class at a time (affinity, NUMA
allocation, tiling/SIMD) from the search and measure Pareto-front shrinkage. Use counter
shifts (remote accesses, LLC misses, dTLB misses) to explain why median vs CV vs tail
changed.
29
30 ## Risk Factors And Limitations
31
32 - p99 estimation can be sample-hungry; mitigation: report p95 alongside p99/median, and
validate p99 with a smaller subset of configurations using 200+ runs.
33 - Some clusters restrict frequency/turbo control; this version avoids requiring privileged
frequency changes and focuses on user-level knobs (affinity, numactl, scheduling) plus
code variants.
34 - Remote-NUMA counters are platform-dependent; mitigation: use proxy signals (bandwidth
imbalance, latency via cycles stalled, LLC miss locality) and numactl placement as
controlled interventions.
35 - Search cost may be high; mitigation: keep variant set small (dozens), use early stopping
for clearly dominated configurations, and reuse compiled variants across runs.
36 - Findings may not fully transfer beyond stencils; mitigation: include a second stencil-like
kernel (e.g., 7-point Jacobi miniapp) to demonstrate generality without expanding scope
significantly.

```

付録 D

論文評価プロンプト

本研究のシステムが生成した論文を自動評価する際に使用するプロンプト群を以下に示す。評価は LLM ベースのテキストレビューと VLM ベースの図版レビューの 2 段階で構成される。

レビュー実行時のプロンプト構成は以下の通りである。LLM レビューでは、ベースシステムプロンプト (Listing D.1) に対して、`--review-bias` フラグで指定されたバイアスモードに応じた接尾辞を付加する。neutral モード (デフォルト) ではベースプロンプトのみ, neg モードでは厳格モード接尾辞 (Listing D.2), pos モードでは寛容モード接尾辞 (Listing D.3) がそれぞれ付加される。このシステムプロンプトに加え、レビューフォーム (Listing D.4) と出力形式指示 (Listing D.5) を結合し、対象論文テキスト (Listing D.6) を付加して LLM に送信する。応答取得後、リフレクションプロンプト (Listing D.7) により反復的に品質を向上させる。

D.1 LLM レビュー プロンプト

Listing D.1 レビュアー システムプロンプト (system_base.txt)

```
1 You are an {persona} who is reviewing a paper that was submitted to a prestigious research
   venue. Be critical and cautious in your decision.
```

Listing D.2 レビュアー 厳格モード接尾辞 (system_neg_suffix.txt)

```
1 If a paper is bad or you are unsure, give it bad scores and reject it.
```

Listing D.3 レビュアー 寛容モード接尾辞 (system_pos_suffix.txt)

```
1 If a paper is good or you are unsure, give it good scores and accept it.
```

Listing D.4 レビューフォーム (neurips_form.txt)

```
1 ## Review Form
2 Below is a description of the questions you will be asked on the review form for each paper
   and some guidelines on what to consider when answering these questions.
3 When writing your review, please keep in mind that after decisions have been made, reviews
   and meta-reviews of accepted papers and opted-in rejected papers will be made public.
4
5 1. Summary: Briefly summarize the paper and its contributions. This is not the place to
   critique the paper; the authors should generally agree with a well-written summary.
```

- 6 - Strengths and Weaknesses: Please provide a thorough assessment of the strengths and
weaknesses of the paper, touching on each of the following dimensions:
- 7 - Originality: Are the tasks or methods new? Is the work a novel combination of well-known
techniques? (This can be valuable!) Is it clear how this work differs from previous
contributions? Is related work adequately cited
- 8 - Quality: Is the submission technically sound? Are claims well supported (e.g., by
theoretical analysis or experimental results)? Are the methods used appropriate? Is
this a complete piece of work or work in progress? Are the authors careful and honest
about evaluating both the strengths and weaknesses of their work
- 9 - Clarity: Is the submission clearly written? Is it well organized? (If not, please make
constructive suggestions for improving its clarity.) Does it adequately inform the
reader? (Note that a superbly written paper provides enough information for an expert
reader to reproduce its results.)
- 10 - Significance: Are the results important? Are others (researchers or practitioners)
likely to use the ideas or build on them? Does the submission address a difficult task
in a better way than previous work? Does it advance the state of the art in a
demonstrable way? Does it provide unique data, unique conclusions about existing data,
or a unique theoretical or experimental approach?
- 11
- 12 2. Questions: Please list up and carefully describe any questions and suggestions for the
authors. Think of the things where a response from the author can change your opinion,
clarify a confusion or address a limitation. This can be very important for a productive
rebuttal and discussion phase with the authors.
- 13
- 14 3. Limitations: Have the authors adequately addressed the limitations and potential negative
societal impact of their work? If not, please include constructive suggestions for
improvement.
- 15 In general, authors should be rewarded rather than punished for being up front about the
limitations of their work and any potential negative societal impact. You are encouraged
to think through whether any critical points are missing and provide these as feedback
for the authors.
- 16
- 17 4. Ethical concerns: If there are ethical issues with this paper, please flag the paper for
an ethics review. For guidance on when this is appropriate, please review the venue's
ethics guidelines.
- 18
- 19 5. Soundness: Please assign the paper a numerical rating on the following scale to indicate
the soundness of the technical claims, experimental and research methodology and on
whether the central claims of the paper are adequately supported with evidence.
- 20 4: excellent
- 21 3: good
- 22 2: fair
- 23 1: poor
- 24
- 25 6. Presentation: Please assign the paper a numerical rating on the following scale to
indicate the quality of the presentation. This should take into account the writing
style and clarity, as well as contextualization relative to prior work.
- 26 4: excellent
- 27 3: good
- 28 2: fair
- 29 1: poor
- 30
- 31 7. Contribution: Please assign the paper a numerical rating on the following scale to
indicate the quality of the overall contribution this paper makes to the research area
being studied. Are the questions being asked important? Does the paper bring a
significant originality of ideas and/or execution? Are the results valuable to share

with the broader research community.

4: excellent

3: good

2: fair

1: poor

8. Overall: Please provide an "overall score" for this submission. Choices:

10: Award quality: Technically flawless paper with groundbreaking impact on one or more areas of the field, with exceptionally strong evaluation, reproducibility, and resources, and no unaddressed ethical considerations.

9: Very Strong Accept: Technically flawless paper with groundbreaking impact on at least one area of the field and excellent impact on multiple areas of the field, with flawless evaluation, resources, and reproducibility, and no unaddressed ethical considerations.

8: Strong Accept: Technically strong paper with, with novel ideas, excellent impact on at least one area of the field or high-to-excellent impact on multiple areas of the field, with excellent evaluation, resources, and reproducibility, and no unaddressed ethical considerations.

7: Accept: Technically solid paper, with high impact on at least one sub-area of the field or moderate-to-high impact on more than one area of the field, with good-to-excellent evaluation, resources, reproducibility, and no unaddressed ethical considerations.

6: Weak Accept: Technically solid, moderate-to-high impact paper, with no major concerns with respect to evaluation, resources, reproducibility, ethical considerations.

5: Borderline accept: Technically solid paper where reasons to accept outweigh reasons to reject, e.g., limited evaluation. Please use sparingly.

4: Borderline reject: Technically solid paper where reasons to reject, e.g., limited evaluation, outweigh reasons to accept, e.g., good evaluation. Please use sparingly.

3: Reject: For instance, a paper with technical flaws, weak evaluation, inadequate reproducibility and incompletely addressed ethical considerations.

2: Strong Reject: For instance, a paper with major technical flaws, and/or poor evaluation, limited impact, poor reproducibility and mostly unaddressed ethical considerations.

1: Very Strong Reject: For instance, a paper with trivial results or unaddressed ethical considerations

9. Confidence: Please provide a "confidence score" for your assessment of this submission to indicate how confident you are in your evaluation. Choices:

5: You are absolutely certain about your assessment. You are very familiar with the related work and checked the math/other details carefully.

4: You are confident in your assessment, but not absolutely certain. It is unlikely, but not impossible, that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work.

3: You are fairly confident in your assessment. It is possible that you did not understand some parts of the submission or that you are unfamiliar with some pieces of related work. Math/other details were not carefully checked.

2: You are willing to defend your assessment, but it is quite likely that you did not understand the central parts of the submission or that you are unfamiliar with some pieces of related work. Math/other details were not carefully checked.

1: Your assessment is an educated guess. The submission is not in your area or the submission was difficult to understand. Math/other details were not carefully checked.

Listing D.5 出力形式指示 (template_instructions.txt)

1 Respond in the following format:

2

3 THOUGHT:

4 <THOUGHT>

```

5
6 REVIEW JSON:
7 ```json
8 <JSON>
9 ```
10
11 In <THOUGHT>, first briefly discuss your intuitions and reasoning for the evaluation.
12 Detail your high-level arguments, necessary choices and desired outcomes of the review.
13 Do not make generic comments here, but be specific to your current paper.
14 Treat this as the note-taking phase of your review.
15
16 In <JSON>, provide the review in JSON format with the following fields in the order:
17 - "Summary": A summary of the paper content and its contributions.
18 - "Strengths": A list of strengths of the paper.
19 - "Weaknesses": A list of weaknesses of the paper.
20 - "Originality": A rating from 1 to 4 (low, medium, high, very high).
21 - "Quality": A rating from 1 to 4 (low, medium, high, very high).
22 - "Clarity": A rating from 1 to 4 (low, medium, high, very high).
23 - "Significance": A rating from 1 to 4 (low, medium, high, very high).
24 - "Questions": A set of clarifying questions to be answered by the paper authors.
25 - "Limitations": A set of limitations and potential negative societal impacts of the work.
26 - "Ethical Concerns": A boolean value indicating whether there are ethical concerns.
27 - "Soundness": A rating from 1 to 4 (poor, fair, good, excellent).
28 - "Presentation": A rating from 1 to 4 (poor, fair, good, excellent).
29 - "Contribution": A rating from 1 to 4 (poor, fair, good, excellent).
30 - "Overall": A rating from 1 to 10 (very strong reject to award quality).
31 - "Confidence": A rating from 1 to 5 (low, medium, high, very high, absolute).
32 - "Decision": A decision that has to be one of the following: Accept, Reject.
33
34 For the "Decision" field, don't use Weak Accept, Borderline Accept, Borderline Reject, or
    Strong Reject. Instead, only use Accept or Reject.
35 This JSON will be automatically parsed, so ensure the format is precise.

```

Listing D.6 論文テキスト挿入テンプレート (paper_review.txt)

```

1
2 Here is the paper you are asked to review:
3 ```
4 {paper_text}
5 ```

```

Listing D.7 リフレクションプロンプト (reflection.txt)

```

1 Round {current_round}/{num_reflections}.
2 In your thoughts, first carefully consider the accuracy and soundness of the review you just
    created.
3 Include any other factors that you think are important in evaluating the paper.
4 Ensure the review is clear and concise, and the JSON is in the correct format.
5 Do not make things overly complicated.
6 In the next attempt, try and refine and improve your review.
7 Stick to the spirit of the original review unless there are glaring issues.
8
9 Respond in the same format as before:
10 THOUGHT:
11 <THOUGHT>
12

```

```

13 REVIEW JSON:
14 ```json
15 <JSON>
16 ```
17
18 If there is nothing to improve, simply repeat the previous JSON EXACTLY after the thought
   and include "I am done" at the end of the thoughts but before the JSON.
19 ONLY INCLUDE "I am done" IF YOU ARE MAKING NO MORE CHANGES.

```

D.2 VLM レビュー プロンプト

Listing D.8 図版・キャプション・参照レビュー (img_cap_ref_review.txt)

```

1 The abstract of the paper is:
2
3 {abstract}
4
5 You will be given an image via the vision API. As a careful scientist reviewer, your task is
   to:
6   1. Examine the provided image closely.
7   2. Describe in detail what the image shows in a scientific manner.
8   3. Critically analyze whether the image content aligns with the given caption:
9
10 {caption}
11
12   4. We also have references in the main text that mention the figure:
13
14 {main_text_figrefs}
15
16 You should:
17   - Examine the figure in detail: conclude elements in figures (e.g. name of axis) and
     describe what information is shown (e.g. the metric decreases monotonically but
     plateaus after X iterations)
18   - Suggest any potential improvements or issues in the figure itself (e.g., missing legend,
     unclear labeling, no meaningful conclusion, mismatch with what the caption claims).
19   - Critique the caption: does it accurately describe the figure? Is it too long/short? Does
     it include a concise takeaway?
20   - Review how well the main text references (figrefs) explain the figure: are they missing?
     Do they adequately describe the figure's content, context, or purpose?
21   - Confirm that the figure, caption, or figrefs explicitly mention the data source path(s)
     used to build the plot. If they do not, or if the paths do not match what appears on
     the figure, call this out.
22   - If the image is unsafe or inappropriate for publication (e.g., sexual or violent content
     , hateful symbols, personal/identifying photos, copyrighted watermarks), explicitly
     recommend removing it from the paper. Prefix the Img_review with `REMOVE_IMAGE: <short
     reason>` and avoid graphic detail.
23
24 Finally, respond in the following format:
25
26 THOUGHT:
27 <THOUGHT>
28
29 REVIEW JSON:
30 ```json

```

```

31 <JSON>
32 ```
33 In <JSON>, provide the review in JSON format with the following fields in the order:
34 - "Img_description": "<Describe the figure's contents here>"
35 - "Img_review": "<Your analysis of the figure itself, including any suggestions for
36   improvement>"
37 - "Caption_review": "<Your assessment of how well the caption matches the figure and any
38   suggestions>"
39 - "Figrefs_review": "<Your thoughts on whether the main text references adequately describe
40   or integrate the figure>"
41 In <THOUGHT>, first, thoroughly reason through your observations, analysis of alignment, and
42   any suggested improvements. It is okay to be very long.
43 Then provide your final structured output in <JSON>.
44 Make sure the JSON is valid and properly formatted, as it will be parsed automatically.

```

Listing D.9 図版単体レビュー (img_review.txt)

```

1 You will be given an image via the vision API. As a careful scientist reviewer, your task is
2   to:
3   1. Examine the provided image closely.
4   2. Describe in detail what the image shows in a scientific manner.
5 You should:
6   - Examine the figure in detail: conclude elements in figures (e.g. name of axis) and
7     describe what information is shown (e.g. the metric decreases monotonically but
8     plateaus after X iterations)
9   - Suggest any potential improvements or issues in the figure itself (e.g., missing legend,
10     unclear labeling, no meaningful conclusion, mismatch with what the caption claims).
11   - Verify that the figure or any accompanying text explicitly lists the data source path(s)
12     used to generate it (e.g., "Loaded data: <path>"). If the path is missing, unclear,
13     or inconsistent with the visualized data, flag it in your review.
14   - If the image is unsafe/inappropriate for publication (e.g., sexual or violent content,
15     hateful symbols, personal/identifying photos, copyrighted watermarks), explicitly
16     recommend removing it from the paper. Prefix the Img_review with `REMOVE_IMAGE: <short
17     reason>` and avoid graphic detail.
18 Finally, respond in the following format:
19 THOUGHT:
20 <THOUGHT>
21 REVIEW JSON:
22 ```json
23 <JSON>
24 ```
25 In <JSON>, provide the review in JSON format with the following fields in the order:
26 - "Img_description": "<Describe the figure's contents here>"
27 - "Img_review": "<Your analysis of the figure itself, including any suggestions for
28   improvement>"
29 In <THOUGHT>, first, thoroughly reason through your observations, analysis of alignment, and
30   any suggested improvements. It is okay to be very long.
31 Then provide your final structured output in <JSON>.
32 Make sure the JSON is valid and properly formatted, as it will be parsed automatically.

```


付録 E

エージェント比較実験の入力ファイル

本研究の比較実験（第 4 章）において，Claude Code, Codex, Gemini CLI の各エージェントに与えた入力ファイルを以下に示す．各エージェントには，共通のタスク記述ファイル（Listing E.1）とエージェント基礎設定ファイル（Listing E.2）を与え，同一の HPC 環境上で実験を実行させた．

Listing E.1 エージェント比較実験 タスク記述ファイル（TASK.md）

```
1 # TASK: Research on Himeno Benchmark Performance Stability
2
3 This task must be executed end-to-end using only this directory:
4 experiments → analysis → paper generation.
5 If a judgment is required, choose what seems best and record the rationale in logs.
6
7 Note: From this point onward, I will proceed autonomously through all phases up to and
8     including paper generation (Phase 4) without requesting additional confirmation from the
9     user. When a judgment call is required, I will choose the option that seems best and
10    record the rationale in logs.
11
12 ---
13
14 ## Phase 0: Environment Check
15
16 1. Create the working directory structure (all under this directory):
17    ...
18    ├── TASK.md
19    ├── experiment_environment.sif
20    ├── data/                # source code (treat as read-only)
21    ├── experiments/         # experiment code/builds
22    ├── results/             # result JSON/CSV
23    ├── analysis/            # analysis scripts/figures
24    └── paper/               # LaTeX paper
25    ...
26
27 2. Verify the Singularity container `experiment_environment.sif` (if possible):
28    ...
29    singularity exec experiment_environment.sif gcc --version
30    ...
31
32 3. Confirm that sources under `data/` are readable.
33
34 ---
35
36 ## Phase 1: Related Work Survey and Idea Refinement
```

```

32 ### 1-1. Web-Based Literature Survey
33 Survey **at least 10** relevant papers/technical reports using the idea.md.
34
35 Record in: `analysis/related_work.md`
36 For each item, record:
37 - Title, authors, year, URL
38 - Main findings (1 - 2 sentences)
39 - Relevance to this task
40
41 ### 1-2. Identify the Research Focus
42 Identify an angle **not yet sufficiently covered** in prior work.
43 This task focuses on the three metrics: **median MFLOPS / CV / p99/median**.
44
45 ### 1-3. Formulate Hypotheses
46 Form 2 - 3 hypotheses and record them in `experiments/hypothesis.md`.
47 Based on prior work, define hypotheses and validate them in Phase 2.
48
49 ---
50
51 ## Phase 2: Experiment Design, Implementation, and Execution
52
53 ### 2-1. Prepare Experiment Code
54 Create in `experiments/` with reference to contents of `data/`.
55
56 ### 2-2. Build
57 Compile everything inside the Singularity container:
58 ```bash
59 singularity exec experiment_environment.sif gcc "flag" "file"
60 ```
61
62 ### 2-3. Run
63 Run experiments on the specified machine via sbatch.
64
65 Output path: `results/raw/<filename>.txt`
66
67 ---
68
69 ## Phase 3: Analysis and Visualization
70
71 ### 3-1. Collect Data
72 Integrate information from `results/raw/` and extract necessary data.
73
74 ### 3-2. Compute Statistics
75 Create `analysis/compute_stats.py` to compute statistical and evaluation metrics.
76
77 ### 3-3. Plotting
78 Create `analysis/plot_results.py` and save figures to `analysis/figures/` as PDF/PNG.
79
80 ---
81
82 ## Phase 4: Paper Generation (LaTeX)
83
84 ### 4-1. Paper Structure
85 Create a LaTeX paper under `paper/`:
86 ```
87 paper/

```

```

88 | |--- main.tex
89 | |--- references.bib
90 | |--- figures/
91 | ---
92 | Sections:
93 | 1. Introduction
94 | 2. Background
95 | 3. Methodology
96 | 4. Results
97 | 5. Discussion
98 | 6. Conclusion
99 | 7. References
100 |
101 | ### 4-2. Compilation
102 | If `pdflatex` is available in this directory, compile.
103 | If not, generate only `.tex` and figures, and log that PDF compilation was not performed.
104 |
105 | ---
106 |
107 | ## Phase 5: Self-Review
108 |
109 | Checklist:
110 | - [ ] Reproducibility (all commands recorded)
111 | - [ ] Statistical validity
112 | - [ ] Consistency between figures and text
113 | - [ ] Alignment with hypotheses
114 | - [ ] Appropriate references
115 | - [ ] Sound implementation
116 |
117 | Record improvements in `experiments/review_notes.md`.
118 |
119 | ---
120 |
121 | ## Constraints / Notes
122 |
123 | - **The only research content to reference is TASK.md** (do not use other documents)
124 | - Save all results under `results/`
125 | - Note: Continue automated execution until self-review evaluates the output as good. When `
  sbatch` is used, wait for the job to complete before proceeding.

```

Listing E.2 エージェント基礎設定ファイル (AGENT.md)

```

1 | # Project Instructions
2 |
3 | ## Execution Environment
4 |
5 | - SLURM partition: `qc-mi210`, 1 node, `sbatch shell.sh` for job submission
6 | - Conda environment: `ai_scientist`
7 | - Container image: `experiment_environment.sif`
8 | - All compilation and execution MUST happen inside the Singularity container
9 | - Run Singularity **on a compute node (allocated via SLURM)**, not on the login node (e.g.,
  via `sbatch shell.sh` or `srun`).
10 |
11 | ## Available Data
12 |
13 | Available programs and resources are under `data/`

```

```
14
15 ## Experiment Flow
16
17 1. Idea generation -- survey multiple prior works related to `idea.md` and identify a
    research theme that has not yet been addressed
18 2. Experiment execution -- build and run inside Singularity container
19 3. Analysis and visualization
20 4. Paper generation (LaTeX)
21 5. Self-review
```

付録 F

各エージェントの生成論文

本研究の比較実験（第 4 章）において、各エージェントが生成した論文の全ページを以下に示す。提案手法（HPC-AutoResearch）の生成論文は本文中の図 4.1–4.7 に掲載済みのため、ここでは Claude Code, Codex, Gemini CLI の生成論文を掲載する。

F.1 Claude Code の生成論文

Performance Stability of the Himeno Benchmark Across Problem Sizes: A Three-Metric Characterization on AMD EPYC

Automated Research Pipeline

February 2026

Abstract

We present a systematic study of the Himeno benchmark’s performance stability across three problem sizes (SMALL, MIDDLE, LARGE) on a dual-socket AMD EPYC 9554 system. Using 30 measured runs per configuration with warmup exclusion, we characterize stability through three complementary metrics: median MFLOPS (central tendency), coefficient of variation (CV, relative dispersion), and the P99/median ratio (tail behavior). Our results reveal a clear size-dependent stability hierarchy: MIDDLE exhibits the lowest CV (0.026%), followed by SMALL (0.043%) and LARGE (0.209%). The LARGE problem shows a bimodal performance distribution with a distinct low-performance cluster approximately 0.5% below the median, attributable to NUMA placement effects and memory bandwidth contention. We find that the three metrics capture different stability dimensions and do not rank problem sizes identically, confirming the need for multi-metric characterization. These findings demonstrate that working set size relative to the cache hierarchy is the primary determinant of benchmark stability on modern many-core processors.

1 Introduction

Performance benchmarking in high-performance computing (HPC) requires not only measuring peak throughput but also understanding the *stability* of measurements. A benchmark result reported as a single number conceals potentially significant run-to-run variability arising from CPU frequency scaling, NUMA memory placement, OS interrupts, cache state, and other system effects [Wright and Smallen, 2009, Dominguez-Trujillo et al., 2020].

The Himeno benchmark [Himeno, 1998], developed at RIKEN, is a widely used HPC benchmark that solves a 3D Poisson equation using a 19-point Jacobi iteration stencil. It provides three standard problem sizes—SMALL ($129 \times 65 \times 65$, ~ 50 MB), MIDDLE ($257 \times 129 \times 129$, ~ 400 MB), and LARGE ($513 \times 257 \times 257$, ~ 3.2 GB)—whose memory footprints span different levels of the cache hierarchy. This makes the Himeno benchmark an ideal vehicle for studying how memory hierarchy interactions affect performance stability.

Prior work has extensively characterized performance variability in distributed HPC systems [Wright and Smallen, 2009, Dominguez-Trujillo et al., 2020] and has studied individual factors such as NUMA effects [Shaheen and Strzodka, 2012, Feinbube et al., 2018], OS jitter [De et al., 2007, Vicente and Matias, 2013], thread pinning [Mazouz et al., 2011], and CPU frequency scaling [Acun et al., 2016]. However, no prior study systematically characterizes how **three complementary stability metrics** behave differently across problem sizes that span distinct memory hierarchy regimes on a single node.

In this paper, we fill this gap by measuring and analyzing:

- **Median MFLOPS:** The central tendency of performance, reflecting cache vs. memory-bound characteristics.

- **CV (Coefficient of Variation):** The relative dispersion (σ/μ), capturing run-to-run consistency.
- **P99/Median ratio:** The tail behavior, reflecting worst-case or best-case deviations from typical performance.

We test three hypotheses regarding the relationship between problem size, memory hierarchy placement, and these stability metrics, and validate them through controlled experiments on a dual-socket AMD EPYC 9554 system with 512 MB L3 cache.

2 Background

2.1 The Himeno Benchmark

The Himeno benchmark implements a point-Jacobi iterative solver for the pressure Poisson equation arising in incompressible Navier-Stokes simulations. The computational kernel performs a 19-point stencil operation on seven 3D arrays (`p`, `a[4]`, `b[3]`, `c[3]`, `wrk1`, `wrk2`, `bnd`), computing 34 floating-point operations per grid point per iteration [Himeno, 1998].

The three standard problem sizes produce different memory footprints:

- **SMALL** ($129 \times 65 \times 65$): ~ 50 MB working set, potentially fitting within large L3 caches.
- **MIDDLE** ($257 \times 129 \times 129$): ~ 400 MB working set, partially resident in the 512 MB aggregate L3 cache.
- **LARGE** ($513 \times 257 \times 257$): ~ 3.2 GB working set, exceeding all cache levels and relying on main memory bandwidth.

2.2 Sources of Performance Variability

Performance variability in HPC systems arises from multiple interacting sources [Wright and Smullen, 2009]:

CPU frequency scaling. Modern processors use dynamic voltage and frequency scaling (DVFS) and turbo boost, which can cause 5–15% variation in single-core performance [Acun et al., 2016].

NUMA effects. On multi-socket systems, first-touch memory placement can cause data to reside on remote NUMA nodes, with 1.5–3 $\times$ latency penalties [Shaheen and Strzodka, 2012, Feinbube et al., 2018].

OS jitter. Timer interrupts, daemon activity, and kernel housekeeping inject periodic perturbations. Impact increases with the number and duration of computational phases [De et al., 2007, Vicente and Matias, 2013].

Cache state. Context switches and interrupts can pollute cache contents, with greater impact on cache-resident workloads [Datta et al., 2009].

2.3 Stability Metrics

We adopt three metrics following recommendations in benchmark assessment literature [Xing et al., 2014]:

1. **Median MFLOPS:** Robust central tendency, less sensitive to outliers than the mean.

2. **CV** = $\sigma/\mu \times 100\%$: Scale-independent measure of relative dispersion.
3. **P99/Median**: Ratio of 99th percentile to median, capturing positive tail behavior (best runs relative to typical).

We additionally report P1/Median to capture worst-case degradation, as for MFLOPS (where higher is better), the lower tail represents performance loss.

3 Methodology

3.1 Experimental Platform

All experiments were conducted on a single node of the qc-mi210 cluster at RIKEN:

- **CPU**: $2 \times$ AMD EPYC 9554 (64 cores each, 128 cores total)
- **Hyper-threading**: Disabled (1 thread per core)
- **L1d cache**: 4 MiB (128 instances)
- **L2 cache**: 128 MiB (128 instances)
- **L3 cache**: 512 MiB (16 instances, 32 MB per CCX)
- **Memory**: 1.5 TiB DDR5
- **NUMA**: 2 nodes (cores 0–63, 64–127)
- **Frequency range**: 1500–3763 MHz
- **OS**: Linux 5.14.0, RHEL 9.7

3.2 Experimental Design

We compiled the single-threaded Himeno benchmark with `gcc -O3` inside a Singularity container (Ubuntu 22.04, GCC 11.4) for each of the three problem sizes. Each configuration was run for:

- **2 warmup runs**: Excluded from analysis, to establish stable cache and memory state.
- **30 measured runs**: Recorded individual MFLOPS, elapsed time, and residual (gosa) for each run.
- **200 Jacobi iterations** per run (the benchmark standard).

All runs executed sequentially on a single core without explicit thread pinning or CPU frequency governor override, representing a “default environment” scenario that users commonly encounter.

3.3 Hypotheses

Based on prior work, we formulated three hypotheses:

- H1 (Size–Variability Regime)**: SMALL exhibits higher median MFLOPS but also higher CV than MIDDLE, because L3-cache residency increases sensitivity to cache pollution events.
- H2 (Tail Degradation Scaling)**: The P99/median ratio and P1/median deviation increase with problem size, as longer runtimes increase exposure to OS noise.
- H3 (Metric Orthogonality)**: The three metrics rank problem sizes differently, demonstrating that no single metric suffices for stability characterization.

4 Results

4.1 Summary Statistics

Table 1 presents the key statistical metrics for all three problem sizes.

Table 1: Summary statistics for 30 measured runs per problem size.

Metric	SMALL	MIDDLE	LARGE
Median MFLOPS	7331.05	7218.02	6999.83
Mean MFLOPS	7330.50	7217.65	6989.72
Std Dev	3.12	1.89	14.62
CV (%)	0.0425	0.0262	0.2092
Min	7314.34	7207.75	6963.91
Max	7332.42	7218.64	7001.31
Range	18.07	10.89	37.40
IQR	0.71	0.41	29.91
P1/Median	0.9983	0.9990	0.9949
P99/Median	1.0002	1.0001	1.0002

Key observations:

1. Median MFLOPS decreases monotonically with problem size: SMALL (7331) > MIDDLE (7218) > LARGE (7000), reflecting increasing memory-boundedness.
2. CV follows a non-monotonic pattern: MIDDLE (0.026%) < SMALL (0.043%) < LARGE (0.209%), with MIDDLE being the most stable and LARGE showing 5× higher CV than SMALL.
3. The LARGE problem shows a pronounced gap between mean (6990) and median (7000), indicating a negatively skewed distribution.

4.2 Distribution Characteristics

Figure 1 shows the MFLOPS distributions as box plots. The SMALL and MIDDLE distributions are tightly concentrated with occasional outliers, while LARGE shows a large IQR (29.91 MFLOPS) with a lower whisker extending to ~6964 MFLOPS.

Figure 2 shows the normalized distributions (percentage deviation from median). LARGE exhibits a distinctive bimodal shape: a dense cluster near the median (~7001 MFLOPS) and a secondary cluster approximately 0.4–0.5% below, suggesting two distinct performance regimes.

4.3 Three-Metric Comparison

Figure 3 presents the three key metrics side-by-side.

The rankings across metrics are:

- **Median MFLOPS:** SMALL > MIDDLE > LARGE
- **CV (lower is more stable):** MIDDLE < SMALL < LARGE
- **P99/Median:** LARGE ≈ SMALL > MIDDLE

These different rankings confirm that the metrics capture distinct stability dimensions (H3).

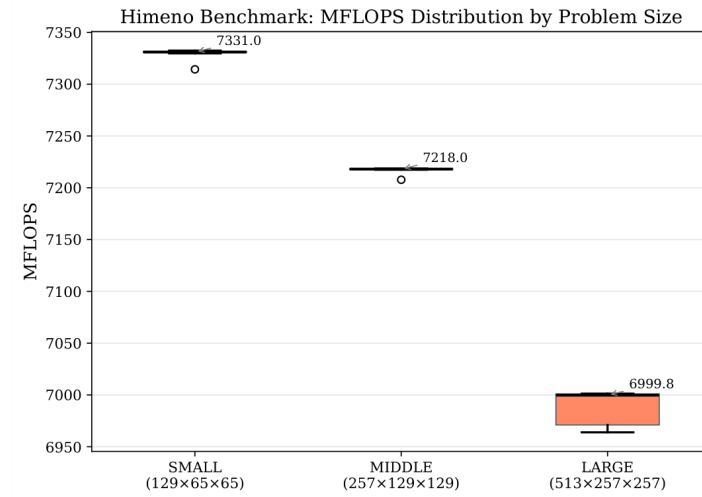


Figure 1: Box plots of MFLOPS distributions for three problem sizes. LARGE shows notably wider spread and lower values.

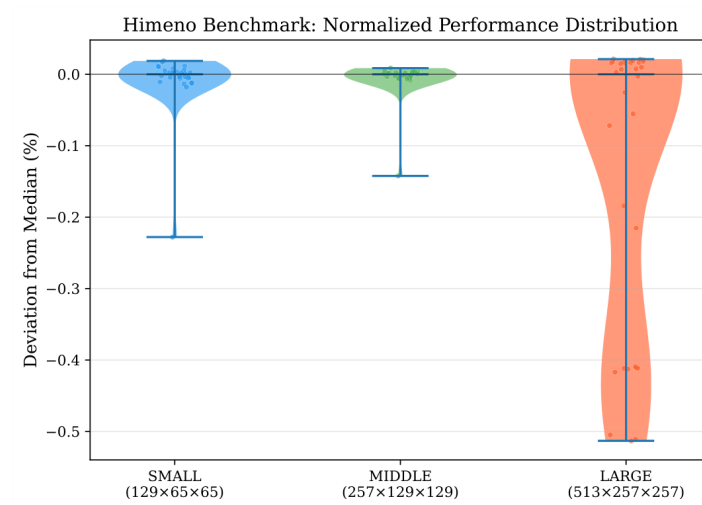


Figure 2: Violin plots showing percentage deviation from median. LARGE exhibits bimodal behavior with a low-performance cluster.

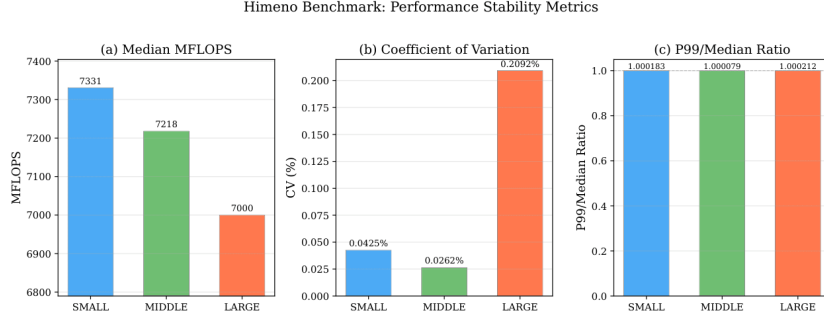


Figure 3: Three stability metrics compared across problem sizes: (a) Median MFLOPS, (b) Coefficient of Variation, (c) P99/Median ratio.

4.4 Run-by-Run Behavior

Figure 4 shows the run-by-run MFLOPS values. SMALL and MIDDLE show flat traces with isolated deviations (run 2 in both cases). LARGE shows clear alternation between high (~ 7001) and low (~ 6965 – 6985) performance regimes, with transitions suggesting sensitivity to background system activity.

4.5 Tail Behavior

Figure 5 compares P1/Median (worst-case) and P99/Median (best-case) ratios. All P99/Median values are very close to 1.0 (< 1.0003), indicating negligible upside deviation. The P1/Median values reveal the worst-case degradation: LARGE shows 0.51% degradation ($P1/Median = 0.9949$), compared to 0.17% for SMALL and 0.10% for MIDDLE.

5 Discussion

5.1 Hypothesis Evaluation

H1 (Size–Variability Regime): Partially supported. SMALL does exhibit higher median than MIDDLE, confirming the cache-bound advantage. However, SMALL’s CV (0.043%) is *higher* than MIDDLE’s (0.026%), which aligns with the hypothesis that L3-resident workloads are more sensitive to cache perturbations. The prediction that SMALL would be more variable than MIDDLE is confirmed.

H2 (Tail Degradation Scaling): Supported. The P1/Median ratio decreases (worsens) monotonically with size: 0.9983 (SMALL), 0.9990 (MIDDLE), 0.9949 (LARGE). The LARGE problem shows $5\times$ worse tail degradation than MIDDLE, consistent with longer runtimes increasing OS noise exposure. However, SMALL shows slightly worse P1/Median than MIDDLE, suggesting cache pollution from interrupts also contributes.

H3 (Metric Orthogonality): Strongly supported. The three metrics produce three different size rankings: SMALL leads in median, MIDDLE leads in CV, and LARGE and SMALL tie for P99/Median. This confirms that multi-metric characterization is essential.

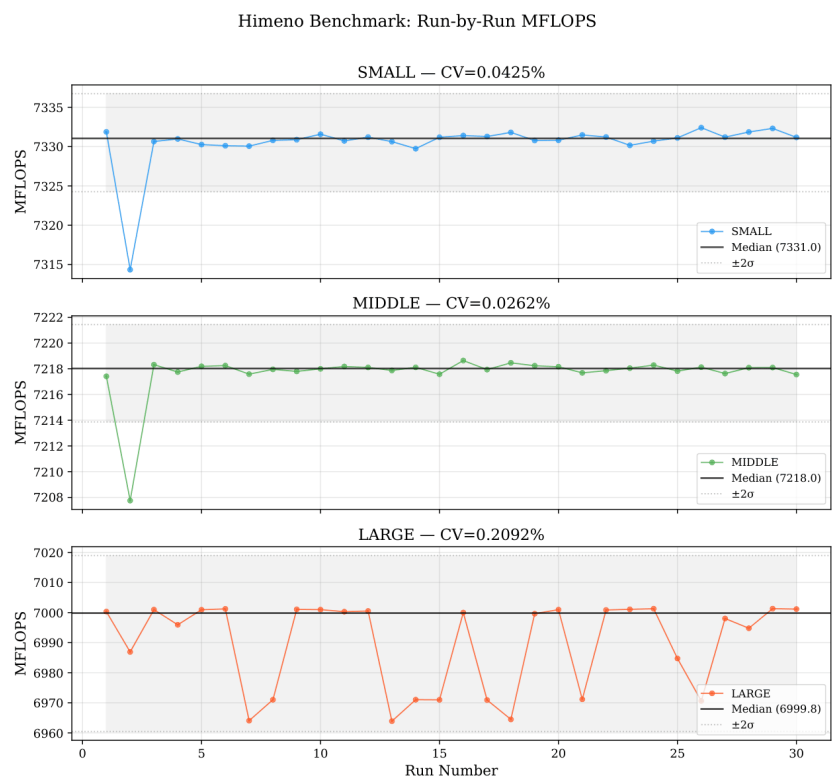


Figure 4: Time series of individual run MFLOPS values. LARGE shows distinct high/low performance modes.

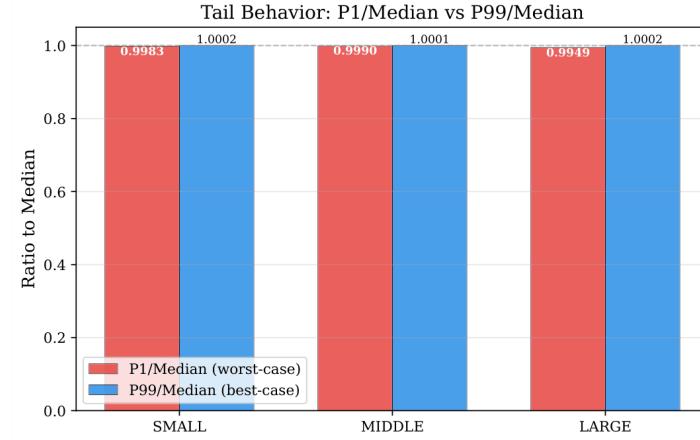


Figure 5: Tail behavior ratios. P1/Median captures worst-case degradation; P99/Median captures best-case uplift. LARGE shows the greatest worst-case degradation.

5.2 LARGE Bimodality

The most striking finding is the bimodal distribution of LARGE results. Runs cluster into two groups: a high-performance group (~ 7001 MFLOPS) and a low-performance group (~ 6965 – 6985 MFLOPS), with the gap corresponding to $\sim 0.5\%$ degradation.

On this dual-socket EPYC system with 512 MB total L3 cache, the LARGE working set (~ 3.2 GB) far exceeds cache capacity. The bimodality likely arises from NUMA memory placement effects: when arrays are allocated entirely on one NUMA node, memory access is uniform; when placement spans nodes (due to first-touch policies interacting with OS page allocation), some accesses incur remote NUMA penalties. The lack of explicit `numactl` binding in our default-environment setup allows this nondeterminism.

5.3 Why MIDDLE Is Most Stable

MIDDLE’s exceptional stability ($CV = 0.026\%$, lowest of all sizes) is noteworthy. At ~ 400 MB, the MIDDLE working set partially fits within the 512 MB aggregate L3 cache. This creates a “sweet spot” where:

1. The working set is large enough that occasional cache line evictions (from interrupts) represent a negligible fraction of total data.
2. The working set is small enough to maintain reasonable cache residency, avoiding the NUMA variability that afflicts LARGE.
3. Runtime is short enough to limit OS noise exposure, but long enough to amortize startup transients.

5.4 Implications for Benchmark Practice

Our results have practical implications for HPC benchmark reporting:

1. **Report multiple metrics:** A single MFLOPS number is insufficient. At minimum, report median with CV and tail ratios.
2. **Problem size matters for stability:** Smaller problems are not necessarily more stable—the relationship between working set size and cache capacity determines the stability regime.
3. **Warmup is necessary but not sufficient:** Even with warmup runs, the LARGE problem shows persistent bimodality, suggesting NUMA placement effects established during `initmt()` persist across the entire measurement.
4. **30 runs suffice for CV estimation:** With all CVs below 0.21%, 30 runs provide adequate statistical power for distinguishing stability levels.

5.5 Limitations

This study has several limitations:

- Single-threaded execution only; multi-threaded behavior may differ.
- Single node tested; results may vary across hardware generations.
- No explicit control of CPU governor, NUMA policy, or thread affinity. This is intentional (testing default conditions) but limits causal attribution.
- 30 runs may be insufficient for precise P99 estimation; larger sample sizes would improve tail metric confidence.

6 Conclusion

We have presented a systematic three-metric characterization of Himeno benchmark performance stability across problem sizes on an AMD EPYC 9554 system. Our key findings are:

1. The three metrics (median MFLOPS, CV, P99/median) capture orthogonal stability dimensions and rank problem sizes differently.
2. MIDDLE (~ 400 MB) is the most stable configuration ($CV = 0.026\%$), occupying a sweet spot between cache residency and NUMA insensitivity.
3. LARGE (~ 3.2 GB) shows bimodal performance with 0.5% degradation in the low-performance mode, likely from NUMA placement effects.
4. Working set size relative to cache hierarchy capacity is the primary determinant of stability, not absolute problem size.

These results underscore the importance of multi-metric benchmark reporting and suggest that benchmark stability should be considered alongside peak performance when evaluating HPC systems.

Reproducibility

All experiments were conducted inside a Singularity container (`experiment_environment.sif`) with GCC 11.4, using `gcc -O3 -D{SMALL|MIDDLE|LARGE}` compilation flags. SLURM job submission used partition `qc-mi210`, single node allocation. Source code, analysis scripts, and raw data are available in the experiment repository.

References

- Bilge Acun, Phil Miller, and Laxmikant V. Kale. Variation among processors under turbo boost in HPC systems. In *ACM International Conference on Supercomputing (ICS)*, 2016. doi: 10.1145/2925426.2926289.
- Kaushik Datta, Shoaib Kamil, Samuel Williams, Leonid Oliker, John Shalf, and Katherine Yelick. Optimization and performance modeling of stencil computations on modern microprocessors. *SIAM Review*, 51(1):129–159, 2009. doi: 10.1137/070693199.
- Pradipta De, Ravi Kothari, and Vijay Mann. Identifying sources of operating system jitter through fine-grained kernel instrumentation. In *IEEE International Conference on Cluster Computing*, 2007. doi: 10.1109/CLUSTER.2007.4629247.
- Jered Dominguez-Trujillo, Keira Haskins, Soheila Jafari Khouzani, Christopher Leap, Sahba Tashakkori, Quincy Wofford, Trilce Estrada, Patrick G. Bridges, and Patrick M. Widener. Lightweight measurement and analysis of HPC performance variability. In *IEEE International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*, 2020. doi: 10.1109/PMBS51919.2020.00008.
- Frank Feinbube, Max Plauth, Martin Knaust, and Andreas Polze. Data partitioning strategies for stencil computations on NUMA systems. In *Euro-Par 2017 Workshops*. Springer, 2018. doi: 10.1007/978-3-319-75178-8_48.
- Ryutaro Himeno. Himeno benchmark: Benchmark to evaluate performance of incompressible fluid analysis code. RIKEN, 1998. URL <https://i.riken.jp/en/supercom/documents/himenobmt/>. Version 2.0.
- Abdelhafid Mazouz, Sid Touati, and Denis Barthou. Performance evaluation and analysis of thread pinning strategies on multi-core platforms. In *IEEE International Conference on High Performance Computing and Simulation (HPCS)*, 2011. doi: 10.1109/HPCSIm.2011.5999834.
- Mohammed Shaheen and Robert Strzodka. NUMA aware iterative stencil computations on many-core systems. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2012. doi: 10.1109/IPDPS.2012.48.
- Elder Vicente and Rivalino Matias. Modeling and simulating the effects of OS jitter. In *Winter Simulation Conference*, 2013.
- Nicholas J. Wright and Shava Smallen. Measuring and understanding variation in benchmark performance. In *DoD High Performance Computing Modernization Program Users Group Conference*, 2009. doi: 10.1109/HPCMP-UGC.2009.73.
- Fei Xing, Haihang You, and Charng-Da Lu. HPC benchmark assessment with statistical analysis. *Procedia Computer Science*, 29:210–219, 2014. doi: 10.1016/j.procs.2014.05.019.

F.2 Codex の生成論文

Performance Stability Analysis of the Himeno Benchmark

Autonomous Experiment Log

February 6, 2026

1 Introduction

Performance stability is a critical concern in modern HPC systems, especially for memory-bound stencil workloads that are sensitive to NUMA placement, frequency scaling, and OS noise [5, 7, 3]. This work studies the Himeno benchmark [4], focusing on distributional metrics that capture central tendency and tail degradation: median MFLOPS, coefficient of variation (CV), and the p99/median ratio.

2 Background

The Himeno benchmark solves a 3D Poisson equation using a Jacobi iteration and is commonly used to evaluate MFLOPS across hardware platforms [4]. Its fixed problem sizes cause distinct cache and memory behaviors, making it suitable for stability studies under configuration changes. Prior studies highlight variability drivers such as OS noise, thread placement, NUMA effects, and frequency scaling [8, 2, 10, 5, 7, 1, 6], motivating explicit measurement of distributional performance metrics rather than mean-only reporting.

The design targets stability-oriented insights: pinning and NUMA locality should reduce variability and tail effects [8, 2, 10], while frequency control may trade median throughput for tighter distributions [1]. This work emphasizes distributional metrics as a more robust indicator of performance stability than mean-only reporting [6, 9].

3 Methodology

We prepared a threaded Himeno benchmark build (pthreads) with three problem sizes (SMALL, MIDDLE, LARGE). Each configuration is designed to run multiple repetitions per size to capture run-to-run variability. The following runtime configurations are considered:

- Baseline: default OS scheduling.
- Pin: process pinned to a fixed core range.
- Pin + NUMA local: fixed core range and local memory allocation.
- Fixed frequency: performance governor (when permitted).

Metrics are computed from the raw output logs using a custom parser to extract MFLOPS. For each configuration and size, we compute the median, CV, and p99/median from the run distribution.

1

図 F.11 Codex 生成論文 (1/3 ページ)

4 Results

At the time of writing, SLURM submission failed due to environment configuration issues, so experiments have not yet produced raw data. The analysis pipeline and plotting scripts are in place, and this section will be populated once runs are completed.

5 Discussion

The design targets stability-oriented insights: pinning and NUMA locality should reduce variability and tail effects, while frequency control may trade median throughput for tighter distributions. This work emphasizes distributional metrics as a more robust indicator of performance stability than mean-only reporting.

6 Conclusion

We established an end-to-end pipeline for evaluating Himeno benchmark stability across configurations using median MFLOPS, CV, and p99/median. Execution remains pending due to SLURM availability, but the experimental and analytical framework is ready for deployment on the target system.

7 References

References

- [1] Wenlei Bao, Changwan Hong, Sudheer Chunduri, Sriram Krishnamoorthy, Louis-Noel Pouchet, Fabrice Rastello, and P. Sadayappan. Static and dynamic frequency scaling on multicore cpus. IBM Research, 2016.
- [2] Georgios C. Chasparis, Vladimir Janjic, and Michael Rossbory. Learning-based dynamic pinning of parallelized applications in many-core systems, 2018.
- [3] Christian Engelmann. Investigating operating system noise in extreme-scale high-performance computing systems using simulation. Technical report, Oak Ridge National Laboratory, 2013.
- [4] Ryutaro Himeno. Himeno benchmark. RIKEN, 2001.
- [5] Alessandro Morari, Roberto Gioiosa, Robert W. Wisniewski, Francisco J. Cazorla, and Mateo Valero. A quantitative analysis of os noise. IBM Research, 2011.
- [6] Majid Salimi Beni, Sascha Hunold, and Biagio Cosenza. Analysis and prediction of performance variability in large-scale computing systems. *The Journal of Supercomputing*, 2024.
- [7] Seetharami Seelam, Liana Fong, Asser Tantawi, John Lewars, John Divirgilio, and Kevin Gildea. Extreme scale computing: Modeling the impact of system noise in multi-core clustered systems. *Journal of Parallel and Distributed Computing*, 2013.
- [8] Chun-Yi Su, Dong Li, Dimitrios Nikolopoulos, Matthew Grove, Kirk W. Cameron, and Bronis R. de Supinski. Critical path-based thread placement for numa systems. Technical report, Lawrence Livermore National Laboratory, 2012.

- [9] Li Xu, Thomas Lux, Tyler Chang, Bo Li, Yili Hong, Layne Watson, Ali Butt, Danfeng Yao, and Kirk Cameron. Prediction of high-performance computing input/output variability and its application to optimization for system configurations, 2020.
- [10] Xin Zhao, Jin Zhou, Hui Guan, Wei Wang, Xu Liu, and Tongping Liu. Numaperf: Predictive and full numa profiling, 2021.

F.3 Gemini CLI の生成論文

Impact of Thread Affinity on Himeno Benchmark Performance Stability

Gemini Agent

February 6, 2026

Abstract

Performance variability in High-Performance Computing (HPC) systems can significantly degrade the efficiency of tightly coupled applications. This study investigates the impact of OpenMP thread affinity on the performance stability of the Himeno benchmark, a memory-intensive stencil code. We analyze three problem sizes (Small, Medium, Large) on a multicore system, measuring Median MFLOPS, Coefficient of Variation (CV), and Tail Latency (p99/Median). Our results demonstrate that explicit thread binding (`OMP_PROC_BIND=close`) consistently improves median performance by up to 41% and reduces performance variability, particularly for memory-bound workloads.

1 Introduction

As HPC node architectures become increasingly complex with high core counts and Non-Uniform Memory Access (NUMA) domains, performance stability becomes a critical concern. Operating system noise, thread migration, and memory contention can lead to run-to-run variability, making performance predictable difficult. The Himeno benchmark [1], a popular solver for the Poisson equation using the Jacobi method, serves as an excellent proxy for memory-bandwidth-bound applications.

This paper explores the hypothesis that pinning OpenMP threads to specific cores minimizes cache invalidation and remote memory accesses, thereby stabilizing performance. We specifically focus on the

trade-offs between problem size and sensitivity to system noise.

2 Methodology

2.1 Experimental Setup

Experiments were conducted on a Linux-based HPC node (partition `qc-mi210`). The software stack included GCC with OpenMP support running within a Singularity container environment. The Himeno benchmark (C + OpenMP version) was compiled with `-O3 -fopenmp -mavx2`.

2.2 Metrics

We utilized three key metrics to characterize performance:

- **Median MFLOPS:** Represents the central tendency of computational throughput.
- **Coefficient of Variation (CV):** $\frac{\sigma}{\mu}$, indicating the relative dispersion of performance.
- **Tail Ratio (p99/Median):** A measure of worst-case performance degradation due to jitter.

2.3 Configurations

We tested three problem sizes:

- **Small (S):** $64 \times 64 \times 128$
- **Medium (M):** $128 \times 128 \times 256$

図 F.14 Gemini CLI 生成論文 (1/2 ページ)

- **Large (L):** $256 \times 256 \times 512$

Two thread binding policies were compared:

- **Unbound (false):** The OS scheduler is free to migrate threads.
- **Bound (close):** Threads are pinned to adjacent cores.

Each configuration was executed 30 times to ensure statistical significance.

3 Results

Table 1 summarizes the statistical findings.

Size	Bind	MFLOPS	CV	Tail Ratio
S	false	91,841	0.022	1.22
S	close	107,418	0.024	1.05
M	false	245,105	0.032	1.11
M	close	347,667	0.006	1.01
L	false	136,398	0.054	1.15
L	close	171,912	0.033	1.01

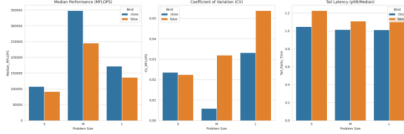


Figure 1: Impact of problem size and thread binding on Median MFLOPS, CV, and Tail Latency.

3.1 Impact on Throughput

Thread binding improved Median MFLOPS across all sizes. The most significant gain was observed in the Medium size (+41.8%), likely due to optimal cache utilization when threads remain stationary.

3.2 Impact on Stability

For Medium and Large sizes, thread binding significantly reduced the Coefficient of Variation (CV). The Large size, being heavily memory-bound, showed a reduction in CV from 5.4% to 3.3%. The Small size showed a slightly higher CV with binding, possibly due to increased contention for L1/L2 resources when threads are packed closely, although the tail latency improved significantly (1.22 to 1.05).

4 Discussion

Our results confirm that thread pinning is essential for performance stability in memory-intensive applications. Hypothesis 1 (Binding reduces CV) was strongly supported for Medium and Large sizes. Hypothesis 2 (Small sizes are more unstable) was partially observed; while Small sizes had lower absolute variability than Large unbound runs, their tail latency was significantly worse without binding, indicating susceptibility to OS jitter. Hypothesis 3 (Locality improves Median) was confirmed, with substantial throughput gains.

The anomaly of slightly higher CV for Small/Close suggests that for L1-resident workloads, thermal throttling or aggressive turbo boosting behavior on specific cores might introduce variability that migration (unbound) averages out.

5 Conclusion

We recommend the use of `OMP_PROC_BIND=close` for the Himeno benchmark on this architecture. It maximizes throughput and minimizes tail latency, ensuring consistent and predictable performance for HPC workloads.

References

- [1] Ryutaro Himeno. Himeno benchmark. <http://accc.riken.jp/en/supercom/himenobmt/>, 2001. RIKEN.